5 year experience Full-Stack Developer, MongoDB expectations go beyond basics—you're expected to understand **data modeling, performance, and backend integration deeply**.

1. MongoDB Fundamentals

- What is NoSQL
- What is a document database
- JSON vs BSON
- Collections vs Documents
- Schema-less design
- When to use MongoDB vs SQL

MongoDB itself:

- MongoDB

2. CRUD Operations (Core Skills)

- `insertOne`, `insertMany`
- `find`, `findOne`
- `updateOne`, `updateMany`
- `replaceOne`
- `deleteOne`, `deleteMany`

Filtering:

- Comparison operators (`$gt`, `$lt`, `$eq`)
- Logical operators (`$and`, `$or`, `$not`)
- Element operators (`$exists`, `$type`)

3. Querying & Projection

- Field selection (projection)
- Sorting (`sort`)
- Pagination (`limit`, `skip`)
- Cursor behavior

4. Indexing (VERY IMPORTANT)

- Single field index
- Compound index
- Multikey index (arrays)
- Text index
- TTL index
- Unique index

Concepts:

- Index selection
- Covered queries
- Index performance trade-offs

5. Data Modeling (Most Important for MongoDB)

Unlike SQL, this is critical.

- Embedding vs Referencing
- One-to-One
- One-to-Many
- Many-to-Many
- Denormalization
- Schema design patterns:
    - Bucket pattern
    - Subset pattern
    - Extended reference pattern

6. Aggregation Framework (High Priority)

- `$match`
- `$group`
- `$project`
- `$sort`
- `$limit`
- `$skip`
- `$lookup` (JOIN equivalent)
- `$unwind`
- `$facet`

Advanced:

- Pipeline optimization
- Aggregation performance

7. Relationships & Joins

- Manual references
- `$lookup`
- Embedding strategy
- Trade-offs vs SQL joins

8. Transactions (Modern MongoDB)

- Multi-document transactions
- ACID properties in MongoDB
- Session handling
- When to use transactions (and when NOT)

9. Schema Validation

- JSON schema validation
- Enforcing structure in collections
- Required fields, types, constraints

10. Performance Optimization

- Query optimization
- Index tuning
- Avoiding large documents
- Document size limits (16MB)
- Read/write patterns
- Caching strategies

11. Replication

- Replica sets
- Primary vs Secondary
- Automatic failover
- Read preferences

12. Sharding (Scalability)

- Horizontal scaling
- Shard key selection
- Balancing data
- Chunk distribution

13. MongoDB Architecture

- Storage engine (WiredTiger)
- Journaling
- Memory usage
- Write concern
- Read concern

14. Security

- Authentication
- Authorization (roles)
- Data encryption
- Network security
- Preventing injection attacks

15. Backup & Restore

- `mongodump`
- `mongorestore`
- Cloud backups
- Point-in-time recovery

16. MongoDB Tools & Ecosystem

- MongoDB Compass (GUI)
- Mongo Shell (`mongosh`)
- Atlas (cloud platform)

17. Backend Integration (Full Stack MUST)

Node.js integration:

- Connecting MongoDB with backend
- Connection pooling
- Error handling
- Transactions in code

Popular ODM:

- Mongoose

Concepts:

- Schema definition
- Models
- Middleware (hooks)
- Validation

18. Advanced Features

- Change Streams (real-time updates)
- TTL collections

- Capped collections
- Geospatial queries
- Full-text search

19. DevOps & Production Usage

- Monitoring (CPU, memory, ops/sec)
- Logging
- Scaling strategies
- Deployment best practices

20. Real-World Design Skills

You should be able to:

- Design scalable MongoDB schemas
- Decide embedding vs referencing
- Optimize slow queries
- Use aggregation pipelines
- Handle production-level data

21. When to Use MongoDB vs SQL

You should understand:

Use MongoDB when:

- Flexible schema needed
- High write throughput
- Nested data

Use SQL when:

- Complex joins
- Strong consistency
- Structured relationships

✅ **What companies REALLY test (0–5 years):**

- CRUD + Querying
- Aggregation (`$group`, `$lookup`)
- Indexing
- Data modeling (embedding vs referencing)
- Performance optimization
- Mongoose / backend integration

💡 If you want next step, I can give you:

- **Top MongoDB interview questions (with answers)**
- **MongoDB vs SQL (interview comparison)**
- **Real-world schema design examples (E-commerce, Chat App, etc.)**

MongoDB Fundamentals: Complete Theory Notes for Senior Developers

1. What is NoSQL?

Theoretical Foundation

NoSQL (Not Only SQL) represents a paradigm shift from traditional relational database management systems (RDBMS). It emerged to address limitations of SQL databases in handling modern web-scale applications, big data workloads, and agile development practices.

Key Characteristics of NoSQL:

1. **Schema Flexibility**: Data can be stored without a predefined schema
2. **Horizontal Scalability**: Designed to scale out across distributed systems
3. **High Availability**: Built-in replication and automatic failover
4. **Eventually Consistent**: Many systems relax ACID properties for better performance
5. **Polyglot Persistence**: Different data models for different use cases

The CAP Theorem in NoSQL Context

The CAP theorem states that a distributed system can only guarantee two of three properties:

```text
Consistency (C): All nodes see the same data simultaneously
Availability (A): Every request receives a response (success/failure)
Partition Tolerance (P): System continues operating despite network failures
```

MongoDB's Position: MongoDB is typically **CP** (Consistent, Partition Tolerant) by default but can be configured for eventual consistency when needed.

NoSQL Database Types:

Type	Description	Examples	Use Cases
Document Stores	Store data as documents (JSON/BSON)	MongoDB, Couchbase	Catalogs, user profiles, content management
Key-Value Stores	Simple key-value pairs	Redis, DynamoDB	Caching, session management
Column-Family Stores	Data stored in column families	Cassandra, HBase	Time-series data, analytics
Graph Databases	Nodes and relationships	Neo4j, JanusGraph	Social networks, recommendation engines

When NoSQL Excels:

- **Massive scale** with millions of read/write operations per second
- **Rapid development** with changing data requirements
- **Unstructured or semi-structured data** that doesn't fit relational models
- **Geographically distributed** applications
- **Real-time analytics** on big data

---

2. What is a Document Database?

Core Concept

A document database stores data in documents rather than tables with rows and columns. Each document is a self-contained unit that encapsulates both data and its structure.

Document Structure Philosophy

Self-Contained Documents:
Instead of normalizing data across multiple tables (like in SQL), related information is often stored together in a single document. This aligns with how applications typically consume data.

Comparison: SQL vs Document Model

SQL Approach (Normalized) :

sql

```sql
-- Customers table
id | name | email
1  | John | john@email.com

-- Orders table
id | customer_id | date | total
100| 1           | 2024-01-01 | 500

-- Order_Items table
id | order_id | product | price
1  | 100      | Laptop  | 500
```

Document Approach (Denormalized) :

javascript

```javascript
// Single document contains all related data
{
  "customerId": 1,
```

```
  "customerName": "John",
  "customerEmail": "john@email.com",
  "orders": [
    {
      "orderId": 100,
      "date": "2024-01-01",
      "total": 500,
      "items": [
        {
          "product": "Laptop",
          "price": 500
        }
      ]
    }
  ]
}
```

Benefits of Document Model:

1. **Locality**: Related data accessed together is stored together
2. **Natural Mapping**: Documents map directly to objects in code
3. **Flexible Schema**: Each document can have different fields
4. **Reduced Joins**: Embedded documents eliminate need for expensive joins
5. **Easier Scaling**: Documents are natural units for sharding

---

3. JSON vs BSON

JSON (JavaScript Object Notation)

Characteristics:

- Human-readable text format
- Supports basic data types: string, number, boolean, null, object, array
- Self-describing structure

- Used widely for API communication

JSON Example:

```
json
{
  "_id": "507f1f77bcf86cd799439011",
  "name": "Product Catalog",
  "price": 29.99,
  "tags": ["electronics", "sale"],
  "metadata": {
    "created": "2024-01-01T10:00:00Z",
    "inStock": true
  }
}
```

BSON (Binary JSON)

Characteristics:

- Binary-encoded serialization of JSON-like documents
- Extends JSON with additional data types
- Designed for efficiency in storage and traversal
- Supports type prefixes for quick field lookup
- Includes length prefixes for fast document scanning

BSON Extensions to JSON:

Data Type	Description	Example in BSON
ObjectId	12-byte unique identifier	`ObjectId("507f1f77bcf86cd799439011`
Date	64-bit integer timestamp	`ISODate("2024-01-01T10:00:00Z")`
Binary Data	Raw binary content	`BinData(0, "SGVsbG8=")`

Data Type	Description	Example in BSON
Regular Expression	Native regex support	`/^MongoDB$/i`
32-bit Integer	Explicit integer type	`NumberInt(42)`
64-bit Integer	Long integer	`NumberLong(4294967295)`
Decimal128	High-precision decimal	`NumberDecimal("10.99")`
Timestamp	Internal MongoDB timestamp	`Timestamp(1672531200, 1)`

BSON Internal Structure

```text
Document = [Element1, Element2, ..., ElementN, Terminator]

Element = [Type, FieldName, Value]

Example BSON for { "price": 29.99 }:

[1]        ["price"]     [double: 29.99]    [0x00]
 ↑           ↑                ↑                 ↑
 Type: 0x01  Field name   Value            Terminator
 (Double)    (null-terminated)
```

Why MongoDB Uses BSON:

1. **Traversal Speed**: Type prefixes allow skipping fields quickly
2. **Rich Data Types**: Supports dates, binaries, and custom types
3. **Space Efficiency**: Binary format reduces storage overhead
4. **Performance**: Direct memory access without parsing text

---

4. Collections vs Documents

Collections

A collection is a grouping of MongoDB documents, analogous to a table in relational databases. However, unlike tables, collections do not enforce a schema.

Collection Types:

javascript

```javascript
// 1. Standard Collections
db.users.insertOne({ name: "John", age: 30 })
db.users.insertOne({ name: "Jane", email: "jane@email.com", address: {...} })

// 2. Capped Collections (Fixed-size, circular queues)
db.createCollection("logs", {
  capped: true,
  size: 1000000,  // 1MB max
  max: 5000       // Maximum 5000 documents
})

// 3. Time Series Collections (MongoDB 5.0+)
db.createCollection("weather", {
  timeseries: {
    timeField: "timestamp",
    metaField: "metadata",
    granularity: "hours"
  }
})

// 4. Clustered Collections (Store documents in index order)
db.createCollection("events", {
  clusteredIndex: { "key": { _id: 1 }, "unique": true }
})
```

Documents

A document is a single record in MongoDB, represented as a BSON object. Documents are the basic unit of data in MongoDB.

Document Structure Deep Dive:

javascript

```javascript
{
  // _id is the only required field - MongoDB creates it if missing
  "_id": ObjectId("507f191e810c19729de860ea"),

  // Nested objects (sub-documents)
  "name": {
    "first": "John",
    "last": "Doe"
  },

  // Arrays can contain mixed types
  "tags": ["developer", "mongodb", 42, true],

  // Array of documents
  "addresses": [
    {
      "type": "home",
      "street": "123 Main St",
      "city": "Boston"
    },
    {
      "type": "work",
      "street": "456 Market St",
      "city": "Cambridge"
    }
  ],

  // Dates
  "createdAt": ISODate("2024-01-01T10:00:00Z"),

  // GeoJSON for geospatial queries
  "location": {
```

```
    "type": "Point",
    "coordinates": [-71.06, 42.36]
  },

  // Binary data
  "profilePicture": BinData(0, "base64EncodedString..."),

  // References to other documents
  "companyId": ObjectId("507f1f77bcf86cd799439012")
}
```

Document Size and Depth Limitations:

Constraint	Maximum	Notes
Document Size	16MB	Includes all BSON overhead
Document Depth	100 levels	Nested objects/arrays
Field Name Length	No limit	But impacts storage size
Number of Fields	No limit	Practical: thousands
Element Count in Array	No limit	Depends on memory

Working with Collections and Documents:

```javascript
// Create/use a collection (implicit creation)
db.myCollection.insertOne({ x: 1 })

// List collections
db.getCollectionNames()

// Get collection stats
db.myCollection.stats()
```

```mongo
// Rename collection
db.myCollection.renameCollection("newCollectionName")

// Drop collection (permanently deletes all documents and metadata)
db.myCollection.drop()

// Document validation rules
db.createCollection("validatedDocs", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "age"],
      properties: {
        name: {
          bsonType: "string",
          description: "must be a string and is required"
        },
        age: {
          bsonType: "int",
          minimum: 0,
          maximum: 150,
          description: "must be an integer between 0-150"
        }
      }
    }
  },
  validationAction: "error"  // or "warn"
})
```

---

5. Schema-less Design

Understanding Schema-less

Schema-less (or schema-flexible) means documents in the same collection can have different fields, data types, and structures. This doesn't mean "no structure" - it means the structure is defined by the application rather than the database.

Schema Flexibility Levels:

javascript

```javascript
// Collection can contain completely different document structures
db.flexibleCollection.insertMany([
  { type: "user", name: "John", email: "john@email.com" },
  { type: "product", sku: "ABC123", price: 29.99, inStock: true },
  { type: "order", orderId: 1001, items: ["A", "B"], total: 50.00 },
  { type: "log", level: "ERROR", message: "Something happened", timestamp: new Dat
e() }
])
```

Evolution of Schema Over Time:

javascript

```javascript
// Version 1: Initial schema (simple)
db.users.insertOne({ name: "John", email: "john@email.com" })

// Version 2: Add optional fields
db.users.insertOne({
  name: "Jane",
  email: "jane@email.com",
  phone: "555-0123",  // New optional field
  preferences: {      // New embedded object
    newsletter: true,
    theme: "dark"
  }
})

// Version 3: Change field type
db.users.insertOne({
  name: "Bob",
  email: "bob@email.com",
  phone: 5550123,  // Changed from string to number
  "preferences.theme": "light",

  // Version tracking
  schemaVersion: 3
})
```

Handling Schema Evolution:

javascript

```javascript
// 1. Application-level versioning
db.users.insertOne({
  _id: ObjectId(),
  schemaVersion: 2,  // Track version
  name: "Alice",
  email: "alice@email.com",
  // Fields for version 2
  profile: {
    bio: "New user"
  }
})

// 2. Query across versions
db.users.aggregate([
  {
    $addFields: {
      displayName: {
        $cond: {
          if: { $eq: ["$schemaVersion", 1] },
          then: "$name",  // Version 1 used 'name'
          else: "$profile.displayName"  // Version 2 uses nested field
        }
      }
    }
  }
])

// 3. Data migration script
db.users.find({ schemaVersion: { $exists: false } }).forEach(function(doc) {
  // Migrate old documents to new schema
  db.users.updateOne(
    { _id: doc._id },
    {
      $set: {
        schemaVersion: 1,
        "profile.displayName": doc.name,
        "profile.created": doc.registered || new Date()
      },
      $unset: { name: "", registered: "" }
```

```
    }
  )
})
```

Benefits of Schema-less Design:

1. **Agile Development**: Add features without database migrations
2. **Polymorphic Data**: Handle varying data shapes naturally
3. **Reduced Downtime**: No schema changes requiring maintenance windows
4. **Simplified Code**: No ORM mapping complexities
5. **Natural for JSON APIs**: Direct mapping to/from application objects

Challenges and Mitigations:

Challenge	Mitigation Strategy
Missing fields	Application defaults, `$exists` queries
Type inconsistencies	Validation rules, application checks
Query complexity	Handle variations in application logic
Index management	Sparse/partial indexes
Data quality	Schema validation, ETL processes

---

6. When to use MongoDB vs SQL

Decision Framework

Use this framework to evaluate whether MongoDB or a SQL database is right for your use case:

MongoDB is Ideal When:

1. Rapid Development & Iteration

javascript

```javascript
// Start with basic fields, add complexity later
// No ALTER TABLE statements, no downtime
db.products.insertOne({
  name: "New Product",
  price: 99.99
})

// Add features without schema changes
db.products.insertOne({
  name: "Advanced Product",
  price: 199.99,
  specifications: {  // Just add nested object
    weight: "2kg",
    dimensions: "10x20x30"
  },
  variants: [  // Add array without altering schema
    { color: "red", sku: "RED001" },
    { color: "blue", sku: "BLUE001" }
  ]
})
```

2. Hierarchical Data

javascript

```javascript
// One-to-many relationships in single document
db.orders.insertOne({
  orderId: "ORD001",
  customer: {
```

```
    id: 1001,
    name: "John Doe",
    email: "john@email.com"
  },
  items: [
    { productId: "P1", name: "Laptop", quantity: 1, price: 999.99 },
    { productId: "P2", name: "Mouse", quantity: 2, price: 29.99 }
  ],
  shipping: {
    address: "123 Main St",
    city: "Boston",
    status: "shipped",
    trackingNumber: "TRACK123"
  },
  payments: [
    { method: "credit_card", amount: 1059.97, status: "completed" }
  ]
})

// Single query retrieves everything
db.orders.findOne({ orderId: "ORD001" })
```

3. High Write Throughput

```javascript
// MongoDB can handle 1000s of writes/second on commodity hardware
// Bulk writes for efficiency
db.events.bulkWrite([
  { insertOne: { document: { type: "click", userId: 1, timestamp: new Date() } } }
,
  { insertOne: { document: { type: "click", userId: 2, timestamp: new Date() } } }
,
  { insertOne: { document: { type: "purchase", userId: 3, amount: 50 } } }
  // ... thousands more
])
```

4. Geographically Distributed Applications

```javascript
// MongoDB's replication and sharding handle distribution naturally
// Configure zones for data locality
sh.addShardTag("shard01", "US East")
```

```javascript
sh.addShardTag("shard02", "EU West")

// Route data to specific regions
sh.updateZoneKeyRange(
  "myDB.users",
  { region: "US" },
  { region: "US" },
  "US East"
)
```

5. Real-time Analytics & Aggregation

```javascript
// Complex aggregations on operational data
db.sales.aggregate([
  { $match: { date: { $gte: ISODate("2024-01-01") } } },
  { $group: {
    _id: {
      product: "$productId",
      month: { $month: "$date" }
    },
    totalSales: { $sum: "$amount" },
    avgPrice: { $avg: "$price" },
    count: { $sum: 1 }
  }},
  { $sort: { totalSales: -1 } },
  { $limit: 10 }
])
```

SQL is Ideal When:

1. Complex Transactions Across Multiple Entities

```sql
-- SQL excels at multi-table transactions
BEGIN TRANSACTION;
    UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
    UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
    INSERT INTO transfers (from_account, to_account, amount, timestamp)
    VALUES (1, 2, 100, NOW());
COMMIT;
```
javascript

```
// MongoDB multi-document transactions (possible but more complex)
const session = db.getMongo().startSession();
session.startTransaction();
try {
  db.accounts.updateOne(
    { account_id: 1 },
    { $inc: { balance: -100 } },
    { session }
  );
  db.accounts.updateOne(
    { account_id: 2 },
    { $inc: { balance: 100 } },
    { session }
  );
  db.transfers.insertOne(
    { from: 1, to: 2, amount: 100, timestamp: new Date() },
    { session }
  );
  session.commitTransaction();
} catch (error) {
  session.abortTransaction();
} finally {
  session.endSession();
}
```

2. Strict Data Integrity Requirements

sql

```
-- SQL enforces referential integrity
CREATE TABLE orders (
    id INT PRIMARY KEY,
    customer_id INT,
    FOREIGN KEY (customer_id) REFERENCES customers(id),
    CHECK (total_amount >= 0)
);

CREATE INDEX idx_customer_orders ON orders(customer_id, order_date);
```

3. Complex Joins and Reporting

sql

```
-- Complex SQL queries with multiple joins
```

```sql
SELECT
    c.name,
    COUNT(DISTINCT o.id) as order_count,
    SUM(oi.quantity * oi.price) as total_spent,
    AVG(oi.price) as avg_item_price
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
LEFT JOIN order_items oi ON o.id = oi.order_id
WHERE o.order_date >= DATE_SUB(NOW(), INTERVAL 1 YEAR)
GROUP BY c.id, c.name
HAVING total_spent > 1000
ORDER BY total_spent DESC;
```

Decision Matrix

Factor	Choose MongoDB	Choose SQL
Schema	Evolving, unpredictable	Stable, well-defined
Relationships	Mostly one-to-many, tree structures	Many-to-many, complex jo
Data Volume	Terabytes to petabytes	Gigabytes to terabytes
Scaling	Horizontal (sharding)	Vertical (bigger servers)
Transactions	Single-document ACID	Multi-record ACID
Consistency	Tunable, eventual options	Strong consistency by defa
Development Speed	Fast, iterative	Slower, requires planning
Query Complexity	Simple to moderate	Very complex
Use Case Examples	CMS, catalogs, real-time	ERP, banking, inventory

Hybrid Approach: Polyglot Persistence

Many modern applications use both MongoDB and SQL databases for different parts:

```javascript
// Application architecture with multiple databases
class OrderService {
  async createOrder(orderData) {
    // Store order in MongoDB for fast access
    const order = await mongodb.orders.insertOne({
      ...orderData,
      status: 'pending',
      createdAt: new Date()
    });

    // Store financial transaction in PostgreSQL for integrity
    await postgres.query(
      'INSERT INTO financial_transactions (order_id, amount, status) VALUES ($1, $2, $3)',
      [order.insertedId, orderData.total, 'pending']
    );

    return order;
  }
}
```

---

7. MongoDB Itself: Deep Architecture

MongoDB Architecture Overview

MongoDB's architecture is designed for scalability, high performance, and ease of use. Here's a comprehensive look at its internal components:

Core Components

```text
```

```
┌──────────────────────────────────────────────────────┐
│                    Application                       │
├──────────────────────────────────────────────────────┤
│                  Driver (Language API)               │
├──────────────────────────────────────────────────────┤
│  ┌──────────────────────────────────────────────┐   │
│  │                 mongos (Router)              │   │
│  │  (For sharded clusters - routes queries)     │   │
│  └──────────────────────────────────────────────┘   │
├──────────────────────────────────────────────────────┤
│  ┌──────────────────────────────────────────────┐   │
│  │              Config Servers                  │   │
│  │  (Store cluster metadata - 3 replica set)    │   │
│  └──────────────────────────────────────────────┘   │
├──────────────────────────────────────────────────────┤
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  │
│  │   Shard 1    │  │   Shard 2    │  │   Shard N    │  │
│  │ (Replica     │  │ (Replica     │  │ (Replica     │  │
│  │   Set)       │  │   Set)       │  │   Set)       │  │
│  └──────────────┘  └──────────────┘  └──────────────┘  │
└──────────────────────────────────────────────────────┘
```

Storage Engine: WiredTiger

MongoDB's default storage engine since version 3.2, WiredTiger provides:

1. MVCC (Multi-Version Concurrency Control)

```javascript
// Each write creates a new version
// Readers see consistent snapshot
db.collection.updateOne(
  { _id: 1 },
  { $set: { status: "active" } }
)
// WiredTiger creates new version, marks old for cleanup
```

2. Document-Level Concurrency

```text
Multiple clients can modify different documents simultaneously
```

```
Write A: Document 1 ──┐
                      ├── Concurrent execution
Write B: Document 2 ──┘

Write C: Document 1 ──┐
                      ├── Sequential (same document)
Write D: Document 1 ──┘
```

3. Compression

javascript

```javascript
// WiredTiger compresses data and indexes
// Prefix compression for indexes
// Dictionary compression for documents

// Check compression stats
db.collection.stats().wiredTiger
```

Memory Management

javascript

```javascript
// WiredTiger cache configuration
storage:
  wiredTiger:
    engineConfig:
      cacheSizeGB: 4  // 50% of RAM minus 1GB by default
      journalCompressor: snappy
    collectionConfig:
      blockCompressor: snappy
    indexConfig:
      prefixCompression: true

// Monitor cache usage
db.serverStatus().wiredTiger.cache
{
  "bytes currently in the cache": 34359738368,  // 32GB
  "maximum bytes configured": 68719476736,      // 64GB
  "bytes read into cache": 1099511627776,
  "bytes written from cache": 549755813888
}
```

Journaling and Durability

javascript

```javascript
// Write concern levels
db.collection.insertOne(
  { name: "test" },
  { writeConcern: { w: "majority", j: true, wtimeout: 5000 } }
)

// Journal files (WiredTiger)
// /data/db/journal/WiredTigerLog.0000000001
// Each journal entry contains group commit of multiple writes

// Checkpoint process (every 60 seconds or 2GB of journal data)
db.runCommand({ getLastError: 1, j: true })
```

Replication Internals

javascript

```javascript
// Oplog (operations log) - capped collection
use local
db.oplog.rs.find().sort({$natural: -1}).limit(1).pretty()
{
  "ts": Timestamp(1672531200, 1),  // Timestamp
  "t": NumberLong(1),               // Term
  "h": NumberLong("7280111994143649393"), // Hash
  "v": 2,                           // Version
  "op": "i",                        // Operation: i=insert, u=update, d=delete
  "ns": "test.users",                // Namespace
  "o": {                             // Operation data
    "_id": ObjectId("63a0f17e8e4f8d4f8d4f8d4f"),
    "name": "John"
  }
}

// Oplog size calculation
// Typically 5% of free disk space (minimum 990MB, maximum 50GB)
// For high-write workloads, consider increasing
db.adminCommand({ replSetResizeOplog: 1, size: 16000 })  // 16GB
```

Sharding Architecture Deep Dive

1. Chunk Splitting and Balancing

javascript

```javascript
// View chunks distribution
db.adminCommand({ shardingState: 1 })

// Manual chunk split
sh.splitAt("myDB.users", { userId: 5000 })

// Move chunk
sh.moveChunk("myDB.users", { userId: 5000 }, "shard02")

// Check balancer status
sh.getBalancerState()
sh.status()
```

2. Query Routing (mongos)

javascript

```javascript
// mongos determines which shards to query
// For sharded collection: users, shard key: { userId: 1 }

// Query with shard key - targeted
db.users.find({ userId: 123, status: "active" })
// mongos routes directly to shard containing userId=123

// Query without shard key - broadcast
db.users.find({ status: "active" })
// mongos broadcasts to ALL shards, then merges results
```

3. Config Servers Data

javascript

```javascript
// Config database structure
use config
show collections
// chunks, collections, databases, shards, tags, version

// Chunk distribution
db.chunks.find({ ns: "myDB.users" }).pretty()
```

```
{
  "_id": "myDB.users-userId_MinKey",
  "ns": "myDB.users",
  "min": { "userId": { "$minKey": 1 } },
  "max": { "userId": 1000 },
  "shard": "shard01",
  "lastmod": Timestamp(1, 0)
}
```

Index Internals

B-Tree Structure in MongoDB

text

```
Root Page
    │
    ├─ Internal Page (userId: 100-200)
    │      ├─ Leaf: doc1 (userId:101) → Disk location
    │      ├─ Leaf: doc2 (userId:150) → Disk location
    │      └─ Leaf: doc3 (userId:199) → Disk location
    │
    └─ Internal Page (userId: 201-300)
           ├─ Leaf: doc4 (userId:201) → Disk location
           ├─ Leaf: doc5 (userId:250) → Disk location
           └─ Leaf: doc6 (userId:299) → Disk location
```

Index Types Deep Dive

javascript

```javascript
// 1. Single Field Index
db.users.createIndex({ email: 1 })

// 2. Compound Index (ESR rule: Equality, Sort, Range)
db.orders.createIndex(
  { status: 1, customerId: 1, orderDate: -1 }
)

// 3. Multikey Index (on array fields)
db.articles.createIndex({ tags: 1 })
// Handles array values automatically
```

```javascript
// 4. Text Index
db.articles.createIndex(
  { title: "text", content: "text" },
  { weights: { title: 10, content: 1 } }
)

// 5. Geospatial Index
db.places.createIndex({ location: "2dsphere" })

// 6. Hashed Index (for sharding)
db.users.createIndex({ userId: "hashed" })

// 7. Wildcard Index
db.products.createIndex({ "metadata.$**": 1 })
// Indexes all fields under metadata
```

Query Execution Pipeline

```javascript
// Explain output shows query planning
db.users.find({ age: { $gt: 25 }, status: "active" })
  .sort({ name: 1 })
  .explain("executionStats")
{
  "queryPlanner": {
    "winningPlan": {
      "stage": "FETCH",
      "inputStage": {
        "stage": "IXSCAN",  // Index scan
        "keyPattern": { "age": 1, "status": 1 },
        "indexName": "age_1_status_1"
      }
    }
  },
  "executionStats": {
    "nReturned": 1000,
    "totalDocsExamined": 1000,
    "totalKeysExamined": 1200,
    "executionTimeMillis": 50
  }
}
```

Internal Locking Mechanism

javascript

```javascript
// Database-level locks (prior to 4.0)
// WiredTiger: Document-level concurrency

// Check lock information
db.serverStatus().locks
{
  "Global": { "acquireCount": { "r": NumberLong(1000), "w": NumberLong(500) } },
  "Database": { "acquireCount": { "r": NumberLong(800), "w": NumberLong(300) } },
  "Collection": { "acquireCount": { "r": NumberLong(700), "w": NumberLong(200) } }
}

// Intent locks for multi-document transactions
// IS (Intent Shared): Read lock on higher level
// IX (Intent Exclusive): Write lock on higher level
```

Network Protocol (OP_MSG)

javascript

```javascript
// MongoDB wire protocol message format
// OP_MSG (since 3.6) replaces OP_QUERY/OP_INSERT etc.

Message {
  messageLength: int32,
  requestId: int32,
  responseTo: int32,
  opCode: int32 (2013 for OP_MSG),
  flags: int32,
  sections: [ Section... ]
}

// Example: Insert operation
// Section type 0 (single document)
// Contains BSON document to insert
```

Background Processes

```javascript
```

```javascript
// 1. Checkpoint thread (every 60s)
// Writes dirty pages to disk

// 2. Cleanup thread (eviction)
// Removes old MVCC versions

// 3. Stats collection thread
db.adminCommand({ serverStatus: 1 })

// 4. Balancer thread (sharded clusters)
// Moves chunks every few seconds

// 5. TTL monitor (every 60s)
// Deletes expired documents
db.events.createIndex({ createdAt: 1 }, { expireAfterSeconds: 86400 })
```

Practical Architecture Decisions

Memory Sizing Formula

text

```text
Total Working Set = (Hot Data Size + Index Size) * 1.2

For a typical application:
- Documents: 100GB
- Indexes: 30GB
- Working Set (20% hot): 26GB
- Recommended RAM: 32-40GB
```

Connection Pooling

javascript

```javascript
// Node.js driver configuration
const client = new MongoClient(uri, {
  maxPoolSize: 100,        // Max connections per server
  minPoolSize: 10,         // Maintain minimum connections
  maxIdleTimeMS: 10000,    // Close idle connections after 10s
  waitQueueTimeoutMS: 5000 // Fail if no connection available
});
```

Read/Write Splitting

javascript

```javascript
// Primary: All writes, strong consistency reads
// Secondaries: Distributed reads, eventual consistency

// Read preferences
db.collection.find().readPref("secondary")  // Read from secondaries
db.collection.find().readPref("primary")     // Read from primary (default)
db.collection.find().readPref("nearest")     // Lowest latency node

// Write concern combinations
{ w: 1, j: false }     // Acknowledged (fast, risk of rollback)
{ w: "majority", j: true }  // Journaled majority (slow, durable)
```

MongoDB CRUD & Querying: Complete Theory Notes

---

2. CRUD Operations (Core Skills)

Insert Operations - Theory

What is an Insert Operation?

An insert operation is the fundamental way to add new data to MongoDB. When you insert a document, MongoDB performs several critical steps that distinguish it from traditional SQL databases.

The Philosophy of Inserts in MongoDB

Unlike SQL databases where you must define a table structure before inserting data, MongoDB's document model allows you to insert data without a predefined schema. This is based on the principle of **schema-on-read** versus schema-on-write.

Schema-on-read means:

- The database doesn't validate the structure when writing
- Structure is interpreted when reading the data
- Applications must handle varying document structures

insertOne() - Deep Theory

Purpose: Insert a single document into a collection.

When to use:

- Adding a new user registration
- Creating a single order
- Logging an individual event

- Any operation where you need the inserted document's ID immediately

Atomicity Guarantee:

- `insertOne()` is atomic at the document level
- If the document is large (>16MB), the entire insert fails
- Either the complete document is inserted or nothing is inserted

Internal Process:

1. **ID Generation**: If no `_id` is provided, MongoDB generates an ObjectId (12 bytes: 4-byte timestamp, 5-byte random value, 3-byte increment counter)
2. **Size Validation**: Document must not exceed 16MB (BSON limit)
3. **Index Updates**: Each index must be updated for the new document
4. **Write Concern Processing**: Waits for acknowledgment based on write concern
5. **Journal Commit**: If journaling is enabled, writes to journal before returning

Performance Theory:

- Providing your own `_id` eliminates server-side generation time
- Smaller documents insert faster (less network transfer, less index update)
- Each insert requires at least one network round-trip
- Batch operations reduce network overhead

insertMany() - Deep Theory

Purpose: Insert multiple documents in a single operation.

Network Efficiency Theory:

- Each network round-trip has latency (typically 1-100ms)
- Sending 1000 documents individually = 1000 round trips
- Sending 1000 documents in one batch = 1 round trip
- Throughput increases dramatically with batch size until other limits are hit

Ordered vs Unordered - Theoretical Comparison:

Ordered Inserts (default):

- Documents inserted in the sequence provided
- If an error occurs, the operation stops
- Earlier documents are preserved, later ones are not attempted
- Use when document order matters or when you want to fail fast

Unordered Inserts:

- MongoDB can parallelize inserts across documents
- Errors don't stop processing of remaining documents
- Maximum throughput but potential partial success
- Use when you want to insert as many as possible regardless of individual failures

Memory Implications:

- Large batches consume client memory
- Large batches consume server memory for processing
- MongoDB has a maximum message size of 48MB
- Optimal batch size typically 100-1000 documents depending on document size

---

Read Operations - Theory

findOne() - Deep Theory

Purpose: Retrieve a single document matching the query criteria.

Theoretical Foundation:

- `findOne()` is a convenience wrapper around `find().limit(1)`
- Returns the first document from the result set
- Immediately closes the cursor after retrieving the document
- If no document matches, returns `null`

Use Cases:

- User login validation (find user by email)
- Fetching a single product by ID
- Getting configuration settings
- Any operation expecting exactly one or zero results

Performance Characteristics:

- Stops scanning after finding first match
- Uses indexes most efficiently when query includes indexed fields
- Does not benefit from covered queries if projection is used (still needs to fetch document to verify match for non-indexed fields in query)

find() - Deep Theory

Purpose: Retrieve multiple documents matching query criteria.

Cursor Theory:
A cursor is not the result set itself, but a pointer to the result set. This is a critical concept for understanding MongoDB's memory management.

Why Cursors Exist:

1. **Memory Efficiency**: Results can be larger than available memory
2. **Network Efficiency**: Results can be streamed in batches
3. **Processing Pipeline**: Allows client to process documents as they arrive
4. **Resource Management**: Server can clean up resources when cursor is closed

Cursor Lifecycle Theory:

- **Creation**: When `find()` is called, a cursor is created but no data is retrieved
- **First Batch**: On first access, first batch (default 101 documents or 1MB) is retrieved
- **Iteration**: As client iterates, more batches are retrieved as needed
- **Timeout**: If inactive for 10 minutes, cursor times out to free server resources
- **Exhaustion**: When all documents retrieved, cursor automatically closes
- **Explicit Close**: Can be closed early to free resources

Batch Size Theory:

```text
Default batch size = min(101 documents, 1MB of data)
```

- Small batches: Quick first result, more network round trips
- Large batches: Slower first result, fewer round trips, more memory on client
- The ideal batch size balances memory usage against network latency

---

Update Operations - Theory

The Update Philosophy

Updates in MongoDB can modify documents in place, which is fundamentally different from SQL's approach of replacing entire rows. This is possible because of MongoDB's document storage format.

updateOne() and updateMany() - Deep Theory

Purpose: Modify existing documents without replacing them entirely.

Atomicity Theory:

- Updates are atomic at the document level
- Even with multiple field updates, either all changes apply or none
- Other operations see either the old version or the new version, never a partial update

Update Operators Theory:

$set Operator:

- Adds new fields or modifies existing fields
- Does not remove unspecified fields (unlike replace)
- Creates nested objects automatically if path doesn't exist
- Efficient for partial updates

$unset Operator:

- Removes fields from documents
- Does not free disk space immediately (marks for overwrite)
- More efficient than setting field to null for queries using $exists

$inc Operator:

- Atomic increment/decrement operation
- Perfect for counters, inventory levels, scores
- Avoids read-modify-write race conditions
- Uses optimistic locking internally

Array Update Operators Theory:

$push:

- Adds elements to arrays
- With modifiers: $each, $slice, $sort, $position
- Can create array if it doesn't exist
- Order of elements is preserved

$pull:

- Removes elements matching criteria
- Removes all matches, not just first
- Efficient for removing specific values

$addToSet:

- Adds only if element doesn't exist
- Maintains set semantics (unique elements)
- More expensive than $push because it must check existence

Update with Aggregation Pipeline (MongoDB 4.2+):

This feature allows updates based on existing field values:

- Can reference other fields in the same document
- Can perform complex transformations
- Eliminates need for read-modify-write cycles
- Reduces network round trips

Write Concern Theory:

Write concern defines the level of acknowledgment requested:

```text
w: 0              → Fire and forget (fastest, risk of data loss)
w: 1              → Acknowledged by primary (default, balanced)
w: majority       → Acknowledged by majority of replica set members
j: true           → Written to journal (durable across restarts)
wtimeout: <ms>    → Timeout if acknowledgment takes too long
```

The trade-off is always: **Durability vs. Performance**

---

replaceOne() - Deep Theory

Purpose: Replace an entire document while preserving the `_id`.

Theoretical Distinction from updateOne():

Aspect	updateOne()	replaceOne()
Operation	Modifies specific fields	Replaces entire document
Schema	Preserves unspecified fields	Removes unspecified fields
Performance	Updates in place	May rewrite document
Use Case	Partial updates	Complete document change

Aspect	updateOne()	replaceOne()
Index Impact	Updates index entries	May require index updates

When to Use replaceOne():

- Complete schema migration
- Document format completely changes
- You want to reset all fields
- After major refactoring of data model

When NOT to Use replaceOne():

- Only changing a few fields (use $set instead)
- Race conditions are possible (updateOne is more atomic for partial changes)
- Large documents (rewriting is expensive)

Concurrency Theory with replaceOne:

The replaceOne operation can be used for optimistic locking:

1. Read document with version field
2. Modify document in application
3. Replace with condition that version hasn't changed
4. If version changed, someone else modified it - retry

This pattern ensures data consistency without locks.

Delete Operations - Theory

deleteOne() and deleteMany() - Deep Theory

Purpose: Permanently remove documents from a collection.

Theoretical Implications of Deletion:

Storage Impact:

- Documents are not immediately removed from disk
- Space is marked as available for reuse
- Fragmentation increases over time
- `compact` command or `repairDatabase` reclaims space

Index Impact:

- Index entries are removed immediately
- Index size doesn't shrink immediately
- Empty index buckets may be reused

Performance Theory:

Deletes are expensive operations because:

1. Must find documents to delete (query execution)
2. Must acquire write locks on affected documents
3. Must remove from all indexes
4. Must update oplog (in replica sets)
5. Must journal the operation

deleteMany() vs drop():

```text
deleteMany({})            drop()
──────────────────────────────────────────────
Preserves indexes         Removes indexes
Preserves collection      Removes collection
O(n) operation            O(1) operation
Can be rolled back        Cannot be rolled back
Can be filtered           Cannot be filtered
Space not reclaimed       Space reclaimed
```

Capped Collection Exception:

In capped collections, you cannot delete individual documents. The collection is

designed as a circular queue where oldest documents are automatically overwritten when space is needed.

TTL Indexes - Automatic Deletion Theory:

TTL (Time-To-Live) indexes provide automatic document deletion based on date fields:

```text
db.events.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })
```

How TTL works:

1. Background thread runs every 60 seconds
2. Finds documents where date field < current time - expireAfterSeconds
3. Deletes them in batches
4. Sleeps for a short interval, repeats

This is more efficient than application-level deletion because:

- No application code needed
- Batched for efficiency
- Runs during idle periods

Filtering - Deep Theory

Why Filtering is Fundamental

Filtering is how you tell MongoDB which documents to operate on. The efficiency of your filters determines query performance.

Comparison Operators Theory

Range Query Theory:
Operators like `$gt`, `$lt`, `$gte`, `$lte` define ranges. MongoDB's B-tree indexes are

optimized for range queries - they can navigate to the start of the range and scan sequentially.

Equality vs Range Performance:

- Equality matches (`$eq`, `$in`) are fastest - direct index lookup
- Range queries are next - sequential scan within index
- Negation (`$ne`, `$nin`) are slowest - must scan everything not matching

The ESR Rule (Equality, Sort, Range):

When designing compound indexes, field order should be:

1. **Equality fields first** - exact matches
2. **Sort fields next** - fields used in sort()
3. **Range fields last** - fields with $gt, $lt, etc.

Why? Because after an equality match, the index is already filtered to a subset. Sorting on the next field is efficient. Range queries at the end can still use the index but may need to scan multiple values.

Logical Operators Theory

$and Operator:

- Implicit AND (comma-separated in query) is usually sufficient
- Explicit $and needed when same field appears multiple times
- MongoDB can use different indexes for different parts of $and

$or Operator:

- Can use multiple indexes (one per $or clause)
- Results from each clause are deduplicated
- More expensive than $and because of deduplication
- Consider $in instead of multiple $or on same field

Query Optimization Theory:

MongoDB's query optimizer:

1. Tries multiple query plans
2. Runs them in parallel for a short time
3. Picks the winning plan based on performance
4. Caches the winning plan for similar queries
5. Reevaluates after certain number of writes or index changes

Element Operators Theory

$exists Operator:

- Checks for field presence, not value
- Can use sparse indexes
- Sparse indexes only include documents with the indexed field

$type Operator:

- Checks BSON data type
- Useful for schema validation
- Essential for mixed-type collections
- Helps with debugging data inconsistencies

Why Element Operators Matter:

In a schema-less database, you often need to:

- Find documents missing required fields
- Locate data with incorrect types
- Clean up inconsistent data
- Validate data quality

These operators give you the tools to manage schema-less data effectively.

---

3. Querying & Projection - Theory

Field Selection (Projection) Theory

What is Projection?

Projection is the mechanism to control which fields MongoDB returns in query results. It's about reducing data transfer and processing.

Theoretical Benefits of Projection:

1. **Network Efficiency**: Sending less data over the network
2. **Memory Efficiency**: Smaller result sets on client
3. **Processing Speed**: Less data to parse and process
4. **Security**: Can exclude sensitive fields (passwords, tokens)

Inclusion vs Exclusion Theory

The Rule: You cannot mix inclusions and exclusions except for `_id`.

Why? Because if you specify:

- `{ field1: 1, field2: 1 }` → Return only these fields
- `{ field1: 0, field2: 0 }` → Return everything except these
- Mixing would be ambiguous - should field3 be included or excluded?

Special Case - _id Field:

- `_id` is included by default
- Can be explicitly excluded even when including other fields
- This exception exists because `_id` is universally needed but sometimes you really don't want it

Projection with Arrays Theory

When projecting on array fields, MongoDB provides special operators:

Positional Operator `$`:

- Returns the first array element matching the query condition
- Useful for "find document and show only the matching subdocument"
- Only returns first match, not all matches

Array Projection Operators:

- `$elemMatch`: Returns array elements matching criteria
- `$slice`: Returns a subset of array elements
- These allow fine-grained control over array data returned

Sorting (sort) Theory

The Importance of Sort Order

Sorting determines the sequence of returned documents. It's not just about presentation - sort order affects query performance and pagination strategies.

Index and Sort Relationship Theory

MongoDB can return sorted results in two ways:

1. **Using an Index** (Efficient):
   - Index already stores data in sorted order
   - MongoDB can traverse index in order
   - No actual sorting operation needed
   - Results returned as quickly as index is traversed
2. **In-Memory Sort** (Inefficient):
   - MongoDB retrieves all matching documents
   - Loads them into memory
   - Sorts them using quicksort or similar algorithm
   - Limited to 32MB for sort operations (throws error if exceeded)

Sort Direction Theory

- `1` for ascending (smallest to largest)
- `-1` for descending (largest to smallest)

For single-field sorts, direction doesn't matter for index usage.
For multi-field sorts, index must match all sort directions exactly.

Example:

```javascript
sort({ a: 1, b: -1 })  // Requires index { a: 1, b: -1 }
                       // Index { a: 1, b: 1 } won't work efficiently
```

Memory and Performance Theory

Sort operations require memory:

- Indexed sorts: Minimal memory (just traversing index)
- In-memory sorts: All result fields must fit in memory
- Sort memory limit: 32MB per operation (in MongoDB 4.2+)
- Exceeding limit throws error or spills to disk (if allowed)

Pagination (limit, skip) Theory

The Pagination Problem

Pagination is the art of returning results in manageable chunks. It's essential for:

- User interfaces showing "next page"
- API endpoints limiting result size
- Batch processing large datasets

skip/limit Pagination Theory

How it works:

```text
```

```
Page 1: skip(0).limit(20)   → First 20 documents
Page 2: skip(20).limit(20)  → Next 20 documents
Page 3: skip(40).limit(20)  → Next 20 documents
```

Performance Degradation Theory:

As `skip` increases, performance degrades because:

1. MongoDB must find all skipped documents
2. It walks through them one by one
3. They must be sorted to know which to skip
4. Skipped documents are still processed, just not returned

The Math of Skip Inefficiency:

- To return page 1000 (20 documents per page)
- MongoDB finds, sorts, and processes 19,980 documents
- Only the last 20 are sent to client
- 99.9% of work is wasted

Cursor-Based Pagination Theory (The Solution)

Also called "keyset pagination" or "seek method":

How it works:

```javascript
// Instead of: .skip(19980).limit(20)
// Do: .find({ $or: [...] }).limit(20)
```

The Theory:

- Use the last seen value to find the next page
- Requires a unique, sortable field combination
- No skipped documents are processed
- Each page query is equally efficient
- Works great for infinite scroll

Trade-offs:

- Cannot jump to arbitrary pages
- More complex application logic
- Requires stable sort criteria

Cursor Behavior - Deep Theory

What is a Cursor, Really?

A cursor is a server-side resource that maintains the state of a query result set. It's not the data itself, but a pointer to where MongoDB is in the result set.

The Client-Server Dance

text

```
Client                    Server
  │                         │
  │  db.find()              │
  │ ──────────────────────> │
  │                         │──┐ Create cursor
  │                         │<─┘
  │  Cursor ID + First Batch │
  │ <────────────────────── │
  │                         │
  │  getMore(cursorId)      │
  │ ──────────────────────> │
  │                         │──┐ Fetch next batch
  │  Next Batch             │<─┘
  │ <────────────────────── │
  │                         │
  │  (implicit close)       │
  │                         │──┐ Clean up after
  │                         │<─┘ timeout
```

Batch Retrieval Theory

Batches balance two competing concerns:

- **Network Round Trips**: Fewer batches = fewer round trips
- **Memory Usage**: Larger batches = more memory on client
- **Time to First Result**: Smaller batches = faster first result

The default batch size (101 documents) is a heuristic based on:

- Typical application scrolling patterns
- Network latency considerations
- Memory constraints of typical clients

Cursor Timeout Theory

Cursors timeout after 10 minutes of inactivity to:

1. Prevent resource leaks from abandoned cursors
2. Free server memory for active operations
3. Avoid infinite cursor accumulation

When to use `noCursorTimeout()`:

- Processing very slowly (one document per minute)
- User walks away and comes back later
- Long-running batch jobs

Risks:

- Must manually close cursor
- Can accumulate many open cursors
- Server resources tied up indefinitely

Cursor Types Theory

Tailable Cursors:

- Only work on capped collections
- Remain open even after no results
- Wait for new data to arrive

- Perfect for tailing logs or real-time feeds

AwaitData Cursors:

- Variation of tailable cursor
- Waits briefly for data before returning
- Reduces polling frequency
- Better for real-time applications

Cursor Cleanup Theory

Always close cursors explicitly because:

1. Server holds resources until timeout
2. Each open cursor consumes memory
3. Too many cursors can exhaust server resources
4. Network connections may stay open longer

The try/finally pattern ensures cleanup even if errors occur.

Summary: The Mental Model

When working with MongoDB CRUD and queries, maintain this mental model:

1. **Documents are the unit of atomicity** - Everything happens at document level
2. **Cursors are pointers, not data** - Think about streaming, not loading
3. **Indexes determine performance** - Query structure must match index structure
4. **Write concern is about trade-offs** - Durability vs. speed, always
5. **Schema is in the application** - MongoDB stores what you give it
6. **Operations are network calls** - Batch when possible, project when practical

2. CRUD Operations (Core Skills)

Insert Operations

insertOne()

The `insertOne()` method inserts a single document into a collection. At the senior level, understanding the internals and implications is crucial.

Syntax:

javascript

```
db.collection.insertOne(
   <document>,
   {
      writeConcern: <document>,
      ordered: <boolean>  // Always true for insertOne
   }
)
```

Internal Mechanics:

```javascript
// What happens when you insert a document:
// 1. MongoDB generates _id if not provided
// 2. Document is validated against collection schema (if any)
// 3. WiredTiger acquires document-level lock
// 4. Document is written to memory and journal
// 5. Index entries are updated
// 6. Returns write result with insertedId

// Example with detailed analysis
const result = db.users.insertOne({
  _id: ObjectId("507f191e810c19729de860ea"), // Manual ID (faster if you know it)
  username: "johndoe",
  email: "john@example.com",
  profile: {
    firstName: "John",
    lastName: "Doe",
    age: 30
  },
```

```
  tags: ["developer", "mongodb"],
  createdAt: new Date()
});

// Check the result
printjson({
  acknowledged: result.acknowledged,
  insertedId: result.insertedId
});

// Performance considerations:
// - Providing _id avoids server-side generation
// - Keep document size under 16MB
// - Consider pre-validating data to avoid schema validation overhead
```

Advanced Patterns:

```javascript
// Conditional insert with findAndModify
db.users.findAndModify({
  query: { email: "john@example.com" },
  update: {
    $setOnInsert: {
      username: "johndoe",
      createdAt: new Date(),
      status: "pending"
    }
  },
  upsert: true,
  new: true
});

// Insert with versioning
db.users.insertOne({
  ...userData,
  schemaVersion: 2,
  _class: "com.example.User", // For polymorphic mapping
  metadata: {
    source: "registration",
    ip: "192.168.1.1",
    userAgent: "Mozilla/5.0..."
  }
```

```
});
```

insertMany()

Syntax and Options:

javascript

```javascript
db.collection.insertMany(
   [ <document1>, <document2>, ... ],
   {
      writeConcern: <document>,
      ordered: <boolean>  // Default: true
   }
)
```

Ordered vs Unordered Inserts:

```javascript
// Ordered insert (default) - Stops on first error
try {
  db.products.insertMany([
    { _id: 1, name: "Laptop", price: 999 },
    { _id: 1, name: "Mouse", price: 29 }, // Will fail (duplicate _id)
    { _id: 2, name: "Keyboard", price: 79 } // Never inserted
  ], { ordered: true });
} catch(e) {
  print("Error at index:", e.writeErrors[0].index);
  // Check which documents were inserted
  db.products.find({ _id: { $in: [1, 2] } }); // Only _id:1 exists
}

// Unordered insert - Continues on error
db.products.insertMany([
  { _id: 1, name: "Laptop", price: 999 },
  { _id: 1, name: "Mouse", price: 29 }, // Fails but continues
  { _id: 2, name: "Keyboard", price: 79 } // Successfully inserted
], { ordered: false });

// Check result
{
  "acknowledged": true,
  "insertedIds": [1, 2], // Note: duplicate _id:1 still reported
```

```
  "writeErrors": [
    {
      "index": 1,
      "code": 11000,
      "errmsg": "E11000 duplicate key error collection"
    }
  ]
}
```

Batch Processing Strategies:

javascript

```javascript
// Large dataset insertion with batching
async function batchInsert(collection, documents, batchSize = 1000) {
  const results = [];

  for (let i = 0; i < documents.length; i += batchSize) {
    const batch = documents.slice(i, i + batchSize);

    try {
      const result = await collection.insertMany(batch, { ordered: false });
      results.push({
        batch: i / batchSize,
        inserted: result.insertedCount,
        errors: result.writeErrors?.length || 0
      });

      // Monitor progress
      print(`Batch ${i/batchSize}: ${result.insertedCount} inserted`);

      // Optional: Throttle to avoid overwhelming the server
      if (i % (batchSize * 10) === 0) {
        await new Promise(resolve => setTimeout(resolve, 1000));
      }
    } catch (error) {
      console.error(`Batch ${i/batchSize} failed:`, error);
    }
  }

  return results;
}

// Usage
const docs = Array.from({ length: 10000 }, (_, i) => ({
  _id: i,
  data: `Document ${i}`,
  timestamp: new Date()
}));

batchInsert(db.testCollection, docs, 500);
```

Read Operations

findOne()

Returns a single document matching the query criteria.

Deep Dive:

javascript

```javascript
// Basic usage
const user = db.users.findOne(
  { email: "john@example.com" },
  { projection: { password: 0, tokens: 0 } }
);

// What happens internally:
// 1. Creates cursor with limit: -1 (negative means single document)
// 2. Returns first document or null
// 3. Automatically closes cursor

// Advanced patterns
db.orders.findOne(
  {
    status: "pending",
    "items.product": "Laptop",
    total: { $gt: 1000 }
  },
  {
    sort: { createdAt: -1 },  // Get most recent pending order
    projection: {
      _id: 1,
      customer: 1,
      total: 1,
      "items.$": 1  // Only first matching array element
    },
    readConcern: "majority",  // Ensure data is committed
    maxTimeMS: 5000           // Timeout after 5 seconds
  }
);
```

find()

Returns a cursor to the matching documents.

Cursor Fundamentals:

javascript

```javascript
// Create cursor (query doesn't execute yet)
const cursor = db.users.find({ active: true });

// Cursor methods
cursor.count();           // Count documents (ignores skip/limit)
```

```javascript
cursor.size();            // Count with skip/limit applied
cursor.explain();         // Get query execution plan
cursor.pretty();          // Format output

// Iteration patterns
// 1. forEach
cursor.forEach(doc => {
  printjson(doc);
});

// 2. Next/Return
while (cursor.hasNext()) {
  const doc = cursor.next();
  processDocument(doc);
}

// 3. Array conversion
const allDocs = cursor.toArray(); // Warning: Loads all into memory

// 4. Batch processing
cursor.batchSize(100).forEach(doc => {
  // Process in batches of 100
  if (someCondition) {
    cursor.objsLeftInBatch(); // Check remaining in current batch
  }
});
```

Advanced Query Patterns:

javascript

```javascript
// Cursor with tailable option (for capped collections)
const cursor = db.logs.find(
  { level: "error" },
  { tailable: true, awaitData: true }
);

// Real-time log tailing
while (cursor.hasNext()) {
  const log = cursor.next();
  processError(log);
}

// Query with hints
db.orders.find({ status: "shipped" })
  .hint({ status: 1, createdAt: -1 }) // Force index usage
  .min({ status: "shipped", createdAt: ISODate("2024-01-01") }) // Index bounds
  .max({ status: "shipped", createdAt: ISODate("2024-02-01") })
  .hint(); // Remove hint

// Read preference for replica sets
db.users.find().readPref("secondary", [
  { datacenter: "east" },  // Tag set
  { datacenter: "west" }
]);
```

Update Operations

updateOne() and updateMany()

Update Operators Deep Dive:

javascript

```javascript
// Field Update Operators
db.users.updateOne(
  { _id: 1 },
  {
    $set: { status: "active", lastLogin: new Date() },
    $unset: { temporaryFlag: "" },
    $inc: { loginCount: 1 },
    $mul: { score: 1.5 },
    $rename: { "oldField": "newField" },
    $min: { discount: 0.1 },  // Set if current value > 0.1
    $max: { priority: 5 }      // Set if current value < 5
  }
);

// Array Update Operators
db.orders.updateOne(
  { _id: 1000 },
  {
    // Add to array (no duplicates)
    $addToSet: { tags: "urgent" },

    // Push to array (with modifiers)
    $push: {
      history: {
        $each: [
          { status: "processing", timestamp: new Date() },
          { status: "shipped", timestamp: new Date() }
        ],
        $slice: -10,  // Keep last 10 entries
        $sort: { timestamp: -1 }
      }
    },

    // Remove from array
    $pull: { tags: "cancelled" },
    $pullAll: { oldTags: ["deprecated", "obsolete"] },

    // Update specific array element
    $set: { "items.$[elem].status": "backordered" }
  },
  {
    arrayFilters: [
      { "elem.productId": "LAPTOP123", "elem.quantity": { $gt: 5 } }
    ]
  }
);

// Update with aggregation pipeline (MongoDB 4.2+)
db.orders.updateMany(
```

```
  { status: "pending" },
  [
    {
      $set: {
        total: { $multiply: ["$subtotal", 1.08] }, // Add tax
        processedAt: "$$NOW",
        priority: {
          $switch: {
            branches: [
              { case: { $gte: ["$total", 1000] }, then: "high" },
              { case: { $gte: ["$total", 500] }, then: "medium" }
            ],
            default: "low"
          }
        }
      }
    },
    {
      $unset: "temporaryFields"
    }
  ]
);
```

Write Concern and Performance:

javascript

```javascript
// Different write concern levels
db.users.updateOne(
  { _id: 1 },
  { $set: { lastActive: new Date() } },
  {
    writeConcern: {
      w: "majority",      // Wait for majority of replicas
      j: true,            // Wait for journal commit
      wtimeout: 5000      // Timeout after 5 seconds
    }
  }
);

// Performance comparison:
// w: 1, j: false  - Fastest, risk of rollback
// w: majority     - Balanced, safe from replica rollbacks
// w: majority, j: true - Slowest, maximum durability

// Bulk updates for efficiency
const bulk = db.users.initializeUnorderedBulkOp();
for (let i = 0; i < 1000; i++) {
  bulk.find({ _id: i }).updateOne({ $inc: { visits: 1 } });
}
const result = bulk.execute();
printjson({
  matched: result.nMatched,
  modified: result.nModified,
  upserted: result.nUpserted
});
```

replaceOne()

Completely replaces a document except for the `_id` field.

Key Concepts:

javascript

```javascript
// Original document
{
  _id: 1,
  name: "John Doe",
  email: "john@old.com",
  age: 30,
  preferences: { theme: "dark" }
}

// Replace operation
db.users.replaceOne(
  { _id: 1 },
  {
    name: "John Smith",
    email: "john@new.com",
    phone: "555-0123"  // Note: age and preferences are gone
  }
);

// Result
{
  _id: 1,
  name: "John Smith",
  email: "john@new.com",
  phone: "555-0123"
  // age and preferences no longer exist
}
```

Important Considerations:

javascript

```javascript
// 1. Schema migration with replaceOne
db.products.find({ category: "legacy" }).forEach(function(doc) {
  // Transform to new schema
  const newDoc = {
    _id: doc._id,
    sku: doc.oldSku,
    name: doc.title,
    price: {
      current: doc.price,
      currency: "USD",
      historical: doc.oldPrices || []
    },
    metadata: {
      importedAt: new Date(),
      originalVersion: doc.version
    }
  };
```

```
  db.products.replaceOne({ _id: doc._id }, newDoc);
});

// 2. Conditional replace (optimistic locking)
const currentDoc = db.accounts.findOne({ _id: 1 });
const newBalance = currentDoc.balance - 100;

const result = db.accounts.replaceOne(
  {
    _id: 1,
    balance: currentDoc.balance,  // Ensure no concurrent update
    version: currentDoc.version    // Version field for concurrency
  },
  {
    ...currentDoc,
    balance: newBalance,
    version: currentDoc.version + 1,
    lastTransaction: new Date()
  }
);

if (result.modifiedCount === 0) {
  // Handle concurrent modification
  throw new Error("Account was modified by another process");
}
```

Delete Operations

deleteOne() and deleteMany()

Basic Usage:

javascript

```
// Delete single document
db.orders.deleteOne(
  { _id: 1000 },
  { writeConcern: { w: "majority", wtimeout: 5000 } }
);

// Delete multiple documents
db.logs.deleteMany(
  {
    timestamp: { $lt: new Date(Date.now() - 30 * 24 * 60 * 60 * 1000) },
    level: "debug"
  }
);
```

Advanced Delete Patterns:

```
javascript

// 1. Capped collection behavior (can't delete)
db.createCollection("system_logs", { capped: true, size: 1000000 });
```

```javascript
// deleteOne() will fail - documents auto-remove when full

// 2. TTL indexes (automatic deletion)
db.sessions.createIndex(
  { lastAccess: 1 },
  { expireAfterSeconds: 3600 } // Delete after 1 hour
);

// 3. Conditional delete with aggregation
const oldOrders = db.orders.aggregate([
  { $match: { status: "delivered", deliveredAt: { $lt: new Date("2023-01-01") } }
},
  { $limit: 1000 },
  { $project: { _id: 1 } }
]).toArray();

if (oldOrders.length > 0) {
  db.orders.deleteMany({
    _id: { $in: oldOrders.map(o => o._id) }
  });
}

// 4. Archive before delete
const archiveCollection = db.getSiblingDB("archive").orders;
const cutoffDate = new Date("2023-01-01");

const pipeline = [
  { $match: { status: "closed", closedAt: { $lt: cutoffDate } } },
  { $limit: 5000 },
  { $out: "archive.orders" } // Write to archive
];

db.orders.aggregate(pipeline);

// Then delete from main
db.orders.deleteMany({
  status: "closed",
  closedAt: { $lt: cutoffDate }
});

// 5. Remove with reference cleanup
const deletedUser = db.users.findOneAndDelete({ email: "user@example.com" });

if (deletedUser) {
  // Clean up related data
  db.posts.deleteMany({ authorId: deletedUser._id });
  db.comments.deleteMany({ userId: deletedUser._id });

  // Or soft delete (mark as inactive)
  db.posts.updateMany(
    { authorId: deletedUser._id },
    { $set: { authorStatus: "deleted" } }
  );
}
```

Performance Considerations:

```javascript
```

```javascript
// 1. Index usage in deletes
db.orders.deleteMany({ status: "cancelled" });
// Uses index on { status: 1 } if exists

// 2. Batch deletes for large datasets
async function batchDelete(collection, query, batchSize = 1000) {
  let deleted = 0;
  let hasMore = true;

  while (hasMore) {
    const docs = await collection.find(query)
      .limit(batchSize)
      .project({ _id: 1 })
      .toArray();

    if (docs.length === 0) {
      hasMore = false;
      break;
    }

    const result = await collection.deleteMany({
      _id: { $in: docs.map(d => d._id) }
    });

    deleted += result.deletedCount;

    // Optional: small delay to reduce impact
    await new Promise(resolve => setTimeout(resolve, 100));
  }

  return deleted;
}

// 3. Drop collection vs deleteMany
db.logs.drop(); // Fastest, but removes indexes and metadata
db.logs.deleteMany({}); // Slower, preserves collection structure

// 4. Truncate alternative (MongoDB 5.0+)
db.logs.deleteMany({}, { hint: { _id: 1 } }); // Use hint for efficiency
```

Filtering: Deep Dive

Comparison Operators

javascript

```javascript
// Comprehensive comparison examples
db.products.find({
  // Numeric comparisons
  price: {
    $gt: 100,     // Greater than
    $gte: 50,      // Greater than or equal
    $lt: 1000,     // Less than
    $lte: 500,     // Less than or equal
    $ne: 0,        // Not equal
    $in: [99, 199, 299],     // In array
    $nin: [999, 1999]        // Not in array
```

```javascript
  },

  // String comparisons
  name: {
    $regex: /^laptop/i,      // Case-insensitive starts with
    $options: "i"            // Alternative syntax
  },

  // Date comparisons
  createdAt: {
    $gt: ISODate("2024-01-01"),
    $lt: ISODate("2024-02-01"),
    $gte: new Date(new Date().setHours(0,0,0,0)) // Today
  },

  // Array comparisons
  tags: {
    $all: ["electronics", "sale"],  // Contains all
    $size: 3,                         // Array length
    $elemMatch: {                     // Element matches criteria
      $gt: 5,
      $lt: 10
    }
  }
});

// Range queries with compound indexes
// Create index for range queries
db.products.createIndex({ category: 1, price: 1 });

// Efficient range query
db.products.find({
  category: "electronics",
  price: { $gte: 100, $lte: 500 }
}).explain("executionStats");
```

Logical Operators

javascript

```javascript
// $and (implicit and explicit)
// Implicit AND (comma-separated)
db.users.find({
  age: { $gte: 18 },
  status: "active",
  "profile.verified": true
});

// Explicit AND (for same field or complex conditions)
db.products.find({
  $and: [
    { price: { $gte: 100 } },
    { price: { $lte: 500 } },
    { $or: [
      { category: "electronics" },
      { category: "computers" }
    ]}
  ]
});
```

```javascript
// $or with different fields
db.orders.find({
  $or: [
    { status: "pending" },
    { priority: "high", amount: { $gt: 1000 } },
    { customerTier: "platinum", createdToday: true }
  ]
});

// $nor (not or)
db.users.find({
  $nor: [
    { status: "banned" },
    { role: "admin" },
    { loginAttempts: { $gt: 5 } }
  ]
});

// $not (negates a condition)
db.products.find({
  price: { $not: { $gte: 100 } } // Same as price < 100
});

// Complex logical combinations
db.transactions.find({
  $and: [
    { amount: { $gt: 0 } },
    { $or: [
      { type: "purchase", status: "completed" },
      { type: "refund", status: "approved" }
    ]},
    { $nor: [
      { flagged: true },
      { retryCount: { $gt: 3 } }
    ]}
  ]
});
```

Element Operators

javascript

```javascript
// $exists - Check field existence
db.users.find({
  email: { $exists: true },        // Has email field
  phone: { $exists: false },       // No phone field
  "profile.bio": { $exists: true } // Nested field
});

// $type - Check BSON type
db.products.find({
  price: { $type: "number" },              // Is number
  sku: { $type: "string" },                 // Is string
  tags: { $type: "array" },                  // Is array
  metadata: { $type: "object" },             // Is object
  createdAt: { $type: "date" },              // Is date
  _id: { $type: "objectId" },                // Is ObjectId
  quantity: { $type: ["int", "long"] }       // Multiple types allowed
```

```javascript
});

// BSON type codes and aliases
const BSON_TYPES = {
  1: "double",
  2: "string",
  3: "object",
  4: "array",
  5: "binData",
  6: "undefined",
  7: "objectId",
  8: "bool",
  9: "date",
  10: "null",
  11: "regex",
  12: "dbPointer",
  13: "javascript",
  14: "symbol",
  15: "javascriptWithScope",
  16: "int",
  17: "timestamp",
  18: "long",
  19: "decimal",
  127: "minKey",
  255: "maxKey"
};

// Practical examples
// Find documents where field was set to null explicitly
db.users.find({ middleName: { $type: "null" } });

// Find numeric fields (excluding strings that look like numbers)
db.products.find({
  $expr: { $eq: [{ $type: "$price" }, "number"] }
});

// Schema validation with $type
db.createCollection("validated", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      properties: {
        name: { bsonType: "string" },
        age: { bsonType: "int", minimum: 0 },
        email: { bsonType: "string", pattern: "^.+@.+$" }
      }
    }
  }
});
```

3. Querying & Projection

Field Selection (Projection)

```javascript
```

```mongo
// Include specific fields
db.users.find(
  { active: true },
  {
    name: 1,
    email: 1,
    "profile.avatar": 1,
    _id: 0  // Exclude _id (included by default)
  }
);

// Exclude specific fields
db.products.find(
  { category: "electronics" },
  {
    description: 0,
    internalNotes: 0,
    supplierInfo: 0
  }
);

// Mixed inclusion/exclusion rules
// Cannot mix 1 and 0 except for _id
db.orders.find(
  {},
  {
    _id: 0,      // Can exclude _id with other inclusions
    orderId: 1,
    customer: 1,
    items: 1
  }
);

// Projection with array elements
db.orders.find(
  { "items.product": "Laptop" },
  {
    "items.$": 1,              // First matching array element
    "items.$[].name": 1,       // All names from array
    "items.slice": { $slice: [0, 3] } // First 3 elements
  }
);

// Computed fields in projection (aggregation style)
db.orders.aggregate([
  { $match: { status: "completed" } },
  {
    $project: {
      orderId: 1,
      date: 1,
      totalWithTax: { $multiply: ["$total", 1.08] },
      itemCount: { $size: "$items" },
      firstItem: { $arrayElemAt: ["$items", 0] },
      hasExpensiveItems: {
        $gt: [{ $max: "$items.price" }, 1000]
      }
    }
  }
]);
```

Sorting (sort)

javascript

```javascript
// Basic sorting
db.products.find()
  .sort({ price: 1 })           // Ascending
  .sort({ price: -1 });         // Descending

// Multi-field sort
db.orders.find()
  .sort({
    status: 1,                   // First by status
    createdAt: -1,                // Then by date desc
    total: -1                     // Then by total desc
  });

// Sort with index consideration
// Create compound index for sort
db.orders.createIndex({ status: 1, createdAt: -1, total: -1 });

// Natural order (insertion order)
db.logs.find().sort({ $natural: -1 }); // Most recent first

// Sort with meta (text search score)
db.articles.find(
  { $text: { $search: "mongodb performance" } },
  { score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } });

// Collation for language-specific sorting
db.users.find()
  .collation({ locale: "de", strength: 1 }) // German, case-insensitive
  .sort({ name: 1 });
```

Pagination (limit, skip)

javascript

```javascript
// Basic pagination
const pageSize = 20;
const page = 3;

db.products.find()
  .sort({ createdAt: -1 })
  .skip(pageSize * (page - 1))  // Skip 40 documents
  .limit(pageSize);              // Return 20 documents

// Performance issues with large skip values
// For page 1000: .skip(19980).limit(20) - Inefficient!

// Cursor-based pagination (more efficient)
db.products.find({
  $or: [
    { createdAt: { $lt: lastCreatedAt } },
    {
      createdAt: lastCreatedAt,
```

```
      _id: { $gt: lastId }
    }
  ]
})
.sort({ createdAt: -1, _id: 1 })
.limit(20);

// Pagination with count
const total = db.products.countDocuments({ category: "electronics" });
const pages = Math.ceil(total / pageSize);

// Efficient pagination with aggregation
db.products.aggregate([
  { $match: { category: "electronics" } },
  { $sort: { price: -1 } },
  {
    $facet: {
      metadata: [{ $count: "total" }],
      data: [
        { $skip: 20 },
        { $limit: 10 },
        { $project: { name: 1, price: 1 } }
      ]
    }
  }
]);
```

Cursor Behavior

javascript

```
// Cursor lifecycle
const cursor = db.users.find({ active: true });

// 1. Cursor created - query not executed
// 2. First batch retrieved on first access
// 3. Subsequent batches retrieved as needed
// 4. Cursor timeout after 10 minutes of inactivity

// Cursor methods
cursor.batchSize(50);              // Set batch size
cursor.maxTimeMS(5000);             // Query timeout
cursor.noCursorTimeout();           // Disable timeout (use with caution)

// Cursor flags
db.users.find().addOption(DBQuery.Option.noTimeout);
db.users.find().addOption(DBQuery.Option.tailable);
db.users.find().addOption(DBQuery.Option.awaitData);

// Monitor cursor activity
db.currentOp({ "command.getMore": { $exists: true } });
db.serverStatus().metrics.cursor;

// Close cursors explicitly
const cursor = db.collection.find();
try {
  while (cursor.hasNext()) {
    process(cursor.next());
```

```javascript
  }
} finally {
  cursor.close();  // Release server resources
}

// Cursor conversion to array (caution with large datasets)
const smallResult = db.users.find({}).limit(1000).toArray();
// Bad: const allUsers = db.users.find({}).toArray(); // May exceed memory

// Iteration patterns for large datasets
// 1. Cursor stream
const stream = db.users.find({}).stream();
stream.on('data', doc => {
  process(doc);
});
stream.on('end', () => console.log('Done'));

// 2. Batch processing with cursor
const cursor = db.largeCollection.find({});
while (cursor.hasNext()) {
  const batch = [];
  for (let i = 0; i < 1000 && cursor.hasNext(); i++) {
    batch.push(cursor.next());
  }
  processBatch(batch);
}

// 3. Cursor with snapshot (prevent duplicate documents)
db.collection.find().snapshot(); // Deprecated, use hint on _id instead
db.collection.find().hint({ _id: 1 });
```

Query Optimization Examples

```javascript
// 1. Covered query (all fields in index)
db.users.createIndex({ email: 1, name: 1, status: 1 });
db.users.find(
  { email: "user@example.com" },
  { name: 1, status: 1, _id: 0 }
);
// Query never touches documents, only index

// 2. Query with explain analysis
const explain = db.orders.find({
  status: "pending",
  total: { $gt: 100 }
})
.sort({ createdAt: -1 })
.explain("executionStats");

printjson({
  winningPlan: explain.queryPlanner.winningPlan,
  nReturned: explain.executionStats.nReturned,
  totalDocsExamined: explain.executionStats.totalDocsExamined,
  executionTimeMillis: explain.executionStats.executionTimeMillis
});
```

```
// 3. Index intersection (using multiple indexes)
db.orders.createIndex({ status: 1 });
db.orders.createIndex({ total: 1 });

db.orders.find({
  status: "pending",
  total: { $gt: 1000 }
});
// MongoDB may use both indexes and intersect results

// 4. Query with hint
db.orders.find({ status: "pending" })
  .hint({ status: 1, createdAt: -1 }); // Force index usage

// 5. Suboptimal queries to avoid
db.users.find({ $where: "this.age > this.loginCount" }); // Slow, no index
db.orders.find({ description: /.*urgent.*/ }); // No index for regex starting with
.*
db.logs.find({ timestamp: { $ne: null } }); // Better: { timestamp: { $exists: tru
e } }
```

MongoDB CRUD & Querying: Complete Theory Notes

2. CRUD Operations (Core Skills)

Insert Operations - Theory

What is an Insert Operation?

An insert operation is the fundamental way to add new data to MongoDB. When you insert a document, MongoDB performs several critical steps that distinguish it from traditional SQL databases.

The Philosophy of Inserts in MongoDB

Unlike SQL databases where you must define a table structure before inserting data, MongoDB's document model allows you to insert data without a predefined schema. This is based on the principle of **schema-on-read** versus schema-on-write.

Schema-on-read means:

- The database doesn't validate the structure when writing

- Structure is interpreted when reading the data
- Applications must handle varying document structures

insertOne() - Deep Theory

Purpose: Insert a single document into a collection.

When to use:

- Adding a new user registration
- Creating a single order
- Logging an individual event
- Any operation where you need the inserted document's ID immediately

Atomicity Guarantee:

- `insertOne()` is atomic at the document level
- If the document is large (>16MB), the entire insert fails
- Either the complete document is inserted or nothing is inserted

Internal Process:

1. **ID Generation**: If no `_id` is provided, MongoDB generates an ObjectId (12 bytes: 4-byte timestamp, 5-byte random value, 3-byte increment counter)
2. **Size Validation**: Document must not exceed 16MB (BSON limit)
3. **Index Updates**: Each index must be updated for the new document
4. **Write Concern Processing**: Waits for acknowledgment based on write concern
5. **Journal Commit**: If journaling is enabled, writes to journal before returning

Performance Theory:

- Providing your own `_id` eliminates server-side generation time
- Smaller documents insert faster (less network transfer, less index update)
- Each insert requires at least one network round-trip
- Batch operations reduce network overhead

insertMany() - Deep Theory

Purpose: Insert multiple documents in a single operation.

Network Efficiency Theory:

- Each network round-trip has latency (typically 1-100ms)
- Sending 1000 documents individually = 1000 round trips
- Sending 1000 documents in one batch = 1 round trip
- Throughput increases dramatically with batch size until other limits are hit

Ordered vs Unordered - Theoretical Comparison:

Ordered Inserts (default):

- Documents inserted in the sequence provided
- If an error occurs, the operation stops
- Earlier documents are preserved, later ones are not attempted
- Use when document order matters or when you want to fail fast

Unordered Inserts:

- MongoDB can parallelize inserts across documents
- Errors don't stop processing of remaining documents
- Maximum throughput but potential partial success
- Use when you want to insert as many as possible regardless of individual failures

Memory Implications:

- Large batches consume client memory
- Large batches consume server memory for processing
- MongoDB has a maximum message size of 48MB
- Optimal batch size typically 100-1000 documents depending on document size

---

Read Operations - Theory

findOne() - Deep Theory

Purpose: Retrieve a single document matching the query criteria.

Theoretical Foundation:

- `findOne()` is a convenience wrapper around `find().limit(1)`
- Returns the first document from the result set
- Immediately closes the cursor after retrieving the document
- If no document matches, returns `null`

Use Cases:

- User login validation (find user by email)
- Fetching a single product by ID
- Getting configuration settings
- Any operation expecting exactly one or zero results

Performance Characteristics:

- Stops scanning after finding first match
- Uses indexes most efficiently when query includes indexed fields
- Does not benefit from covered queries if projection is used (still needs to fetch document to verify match for non-indexed fields in query)

find() - Deep Theory

Purpose: Retrieve multiple documents matching query criteria.

Cursor Theory:
A cursor is not the result set itself, but a pointer to the result set. This is a critical concept for understanding MongoDB's memory management.

Why Cursors Exist:

1. **Memory Efficiency**: Results can be larger than available memory

2. **Network Efficiency**: Results can be streamed in batches
3. **Processing Pipeline**: Allows client to process documents as they arrive
4. **Resource Management**: Server can clean up resources when cursor is closed

Cursor Lifecycle Theory:

- **Creation**: When `find()` is called, a cursor is created but no data is retrieved
- **First Batch**: On first access, first batch (default 101 documents or 1MB) is retrieved
- **Iteration**: As client iterates, more batches are retrieved as needed
- **Timeout**: If inactive for 10 minutes, cursor times out to free server resources
- **Exhaustion**: When all documents retrieved, cursor automatically closes
- **Explicit Close**: Can be closed early to free resources

Batch Size Theory:

```text
Default batch size = min(101 documents, 1MB of data)
```

- Small batches: Quick first result, more network round trips
- Large batches: Slower first result, fewer round trips, more memory on client
- The ideal batch size balances memory usage against network latency

---

Update Operations - Theory

The Update Philosophy

Updates in MongoDB can modify documents in place, which is fundamentally different from SQL's approach of replacing entire rows. This is possible because of MongoDB's document storage format.

updateOne() and updateMany() - Deep Theory

Purpose: Modify existing documents without replacing them entirely.

Atomicity Theory:

- Updates are atomic at the document level
- Even with multiple field updates, either all changes apply or none
- Other operations see either the old version or the new version, never a partial update

Update Operators Theory:

$set Operator:

- Adds new fields or modifies existing fields
- Does not remove unspecified fields (unlike replace)
- Creates nested objects automatically if path doesn't exist
- Efficient for partial updates

$unset Operator:

- Removes fields from documents
- Does not free disk space immediately (marks for overwrite)
- More efficient than setting field to null for queries using $exists

$inc Operator:

- Atomic increment/decrement operation
- Perfect for counters, inventory levels, scores
- Avoids read-modify-write race conditions
- Uses optimistic locking internally

Array Update Operators Theory:

$push:

- Adds elements to arrays
- With modifiers: $each, $slice, $sort, $position
- Can create array if it doesn't exist
- Order of elements is preserved

$pull:

- Removes elements matching criteria
- Removes all matches, not just first
- Efficient for removing specific values

$addToSet:

- Adds only if element doesn't exist
- Maintains set semantics (unique elements)
- More expensive than $push because it must check existence

Update with Aggregation Pipeline (MongoDB 4.2+):

This feature allows updates based on existing field values:

- Can reference other fields in the same document
- Can perform complex transformations
- Eliminates need for read-modify-write cycles
- Reduces network round trips

Write Concern Theory:

Write concern defines the level of acknowledgment requested:

```text
w: 0              → Fire and forget (fastest, risk of data loss)
w: 1              → Acknowledged by primary (default, balanced)
w: majority       → Acknowledged by majority of replica set members
j: true           → Written to journal (durable across restarts)
wtimeout: <ms>    → Timeout if acknowledgment takes too long
```

The trade-off is always: **Durability vs. Performance**

---

replaceOne() - Deep Theory

Purpose: Replace an entire document while preserving the `_id`.

Theoretical Distinction from updateOne():

Aspect	updateOne()	replaceOne()
Operation	Modifies specific fields	Replaces entire document
Schema	Preserves unspecified fields	Removes unspecified fields
Performance	Updates in place	May rewrite document
Use Case	Partial updates	Complete document change
Index Impact	Updates index entries	May require index updates

When to Use replaceOne():

- Complete schema migration
- Document format completely changes
- You want to reset all fields
- After major refactoring of data model

When NOT to Use replaceOne():

- Only changing a few fields (use $set instead)
- Race conditions are possible (updateOne is more atomic for partial changes)
- Large documents (rewriting is expensive)

Concurrency Theory with replaceOne:

The replaceOne operation can be used for optimistic locking:

1. Read document with version field
2. Modify document in application
3. Replace with condition that version hasn't changed
4. If version changed, someone else modified it - retry

This pattern ensures data consistency without locks.

Delete Operations - Theory

deleteOne() and deleteMany() - Deep Theory

Purpose: Permanently remove documents from a collection.

Theoretical Implications of Deletion:

Storage Impact:

- Documents are not immediately removed from disk
- Space is marked as available for reuse
- Fragmentation increases over time
- `compact` command or `repairDatabase` reclaims space

Index Impact:

- Index entries are removed immediately
- Index size doesn't shrink immediately
- Empty index buckets may be reused

Performance Theory:

Deletes are expensive operations because:

1. Must find documents to delete (query execution)
2. Must acquire write locks on affected documents
3. Must remove from all indexes
4. Must update oplog (in replica sets)
5. Must journal the operation

deleteMany() vs drop():

```text
```

```
deleteMany({})            drop()
──────────────────────────────────────────────
Preserves indexes         Removes indexes
Preserves collection      Removes collection
O(n) operation            O(1) operation
Can be rolled back        Cannot be rolled back
Can be filtered           Cannot be filtered
Space not reclaimed       Space reclaimed
```

Capped Collection Exception:
In capped collections, you cannot delete individual documents. The collection is designed as a circular queue where oldest documents are automatically overwritten when space is needed.

TTL Indexes - Automatic Deletion Theory:

TTL (Time-To-Live) indexes provide automatic document deletion based on date fields:

```text
db.events.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })
```

How TTL works:

1. Background thread runs every 60 seconds
2. Finds documents where date field < current time - expireAfterSeconds
3. Deletes them in batches
4. Sleeps for a short interval, repeats

This is more efficient than application-level deletion because:

- No application code needed
- Batched for efficiency
- Runs during idle periods

Filtering - Deep Theory

Why Filtering is Fundamental

Filtering is how you tell MongoDB which documents to operate on. The efficiency of your filters determines query performance.

Comparison Operators Theory

Range Query Theory:
Operators like `$gt`, `$lt`, `$gte`, `$lte` define ranges. MongoDB's B-tree indexes are optimized for range queries - they can navigate to the start of the range and scan sequentially.

Equality vs Range Performance:

- Equality matches (`$eq`, `$in`) are fastest - direct index lookup
- Range queries are next - sequential scan within index
- Negation (`$ne`, `$nin`) are slowest - must scan everything not matching

The ESR Rule (Equality, Sort, Range):

When designing compound indexes, field order should be:

1. **Equality fields first** - exact matches
2. **Sort fields next** - fields used in sort()
3. **Range fields last** - fields with $gt, $lt, etc.

Why? Because after an equality match, the index is already filtered to a subset. Sorting on the next field is efficient. Range queries at the end can still use the index but may need to scan multiple values.

Logical Operators Theory

$and Operator:

- Implicit AND (comma-separated in query) is usually sufficient
- Explicit $and needed when same field appears multiple times

- MongoDB can use different indexes for different parts of $and

$or Operator:

- Can use multiple indexes (one per $or clause)
- Results from each clause are deduplicated
- More expensive than $and because of deduplication
- Consider $in instead of multiple $or on same field

Query Optimization Theory:

MongoDB's query optimizer:

1. Tries multiple query plans
2. Runs them in parallel for a short time
3. Picks the winning plan based on performance
4. Caches the winning plan for similar queries
5. Reevaluates after certain number of writes or index changes

Element Operators Theory

$exists Operator:

- Checks for field presence, not value
- Can use sparse indexes
- Sparse indexes only include documents with the indexed field

$type Operator:

- Checks BSON data type
- Useful for schema validation
- Essential for mixed-type collections
- Helps with debugging data inconsistencies

Why Element Operators Matter:

In a schema-less database, you often need to:

- Find documents missing required fields
- Locate data with incorrect types
- Clean up inconsistent data
- Validate data quality

These operators give you the tools to manage schema-less data effectively.

---

3. Querying & Projection - Theory

Field Selection (Projection) Theory

What is Projection?

Projection is the mechanism to control which fields MongoDB returns in query results. It's about reducing data transfer and processing.

Theoretical Benefits of Projection:

1. **Network Efficiency**: Sending less data over the network
2. **Memory Efficiency**: Smaller result sets on client
3. **Processing Speed**: Less data to parse and process
4. **Security**: Can exclude sensitive fields (passwords, tokens)

Inclusion vs Exclusion Theory

The Rule: You cannot mix inclusions and exclusions except for `_id`.

Why? Because if you specify:

- `{ field1: 1, field2: 1 }` → Return only these fields
- `{ field1: 0, field2: 0 }` → Return everything except these
- Mixing would be ambiguous - should field3 be included or excluded?

Special Case - _id Field:

- `_id` is included by default
- Can be explicitly excluded even when including other fields
- This exception exists because `_id` is universally needed but sometimes you really don't want it

Projection with Arrays Theory

When projecting on array fields, MongoDB provides special operators:

Positional Operator `$`:

- Returns the first array element matching the query condition
- Useful for "find document and show only the matching subdocument"
- Only returns first match, not all matches

Array Projection Operators:

- `$elemMatch`: Returns array elements matching criteria
- `$slice`: Returns a subset of array elements
- These allow fine-grained control over array data returned

Sorting (sort) Theory

The Importance of Sort Order

Sorting determines the sequence of returned documents. It's not just about presentation - sort order affects query performance and pagination strategies.

Index and Sort Relationship Theory

MongoDB can return sorted results in two ways:

1. **Using an Index** (Efficient):
   - Index already stores data in sorted order
   - MongoDB can traverse index in order

- No actual sorting operation needed
- Results returned as quickly as index is traversed

2. **In-Memory Sort** (Inefficient):

- MongoDB retrieves all matching documents
- Loads them into memory
- Sorts them using quicksort or similar algorithm
- Limited to 32MB for sort operations (throws error if exceeded)

Sort Direction Theory

- `1` for ascending (smallest to largest)
- `-1` for descending (largest to smallest)

For single-field sorts, direction doesn't matter for index usage.
For multi-field sorts, index must match all sort directions exactly.

Example:

```javascript
sort({ a: 1, b: -1 })  // Requires index { a: 1, b: -1 }
                       // Index { a: 1, b: 1 } won't work efficiently
```

Memory and Performance Theory

Sort operations require memory:

- Indexed sorts: Minimal memory (just traversing index)
- In-memory sorts: All result fields must fit in memory
- Sort memory limit: 32MB per operation (in MongoDB 4.2+)
- Exceeding limit throws error or spills to disk (if allowed)

Pagination (limit, skip) Theory

The Pagination Problem

Pagination is the art of returning results in manageable chunks. It's essential for:

- User interfaces showing "next page"
- API endpoints limiting result size
- Batch processing large datasets

skip/limit Pagination Theory

How it works:

```text
Page 1: skip(0).limit(20)   → First 20 documents
Page 2: skip(20).limit(20)  → Next 20 documents
Page 3: skip(40).limit(20)  → Next 20 documents
```

Performance Degradation Theory:

As `skip` increases, performance degrades because:

1. MongoDB must find all skipped documents
2. It walks through them one by one
3. They must be sorted to know which to skip
4. Skipped documents are still processed, just not returned

The Math of Skip Inefficiency:

- To return page 1000 (20 documents per page)
- MongoDB finds, sorts, and processes 19,980 documents
- Only the last 20 are sent to client
- 99.9% of work is wasted

Cursor-Based Pagination Theory (The Solution)

Also called "keyset pagination" or "seek method":

How it works:

```javascript
// Instead of: .skip(19980).limit(20)
```

```
// Do: .find({ $or: [...] }).limit(20)
```

The Theory:

- Use the last seen value to find the next page
- Requires a unique, sortable field combination
- No skipped documents are processed
- Each page query is equally efficient
- Works great for infinite scroll

Trade-offs:

- Cannot jump to arbitrary pages
- More complex application logic
- Requires stable sort criteria

Cursor Behavior - Deep Theory

What is a Cursor, Really?

A cursor is a server-side resource that maintains the state of a query result set. It's not the data itself, but a pointer to where MongoDB is in the result set.

The Client-Server Dance

text

```
Client                    Server
  │                         │
  │  db.find()              │
  │ ──────────────────────> │
  │                         │──┐ Create cursor
  │                         │<─┘
  │  Cursor ID + First Batch │
  │ <────────────────────── │
  │                         │
  │  getMore(cursorId)      │
```

```
│ ──────────────────────────> │
│                             │──┐ Fetch next batch
│  Next Batch                 │<─┘
│ <──────────────────────────── │
│                             │
│  (implicit close)           │
│                             │──┐ Clean up after
│                             │<─┘ timeout
```

Batch Retrieval Theory

Batches balance two competing concerns:

- **Network Round Trips**: Fewer batches = fewer round trips
- **Memory Usage**: Larger batches = more memory on client
- **Time to First Result**: Smaller batches = faster first result

The default batch size (101 documents) is a heuristic based on:

- Typical application scrolling patterns
- Network latency considerations
- Memory constraints of typical clients

Cursor Timeout Theory

Cursors timeout after 10 minutes of inactivity to:

1. Prevent resource leaks from abandoned cursors
2. Free server memory for active operations
3. Avoid infinite cursor accumulation

When to use `noCursorTimeout()`:

- Processing very slowly (one document per minute)
- User walks away and comes back later
- Long-running batch jobs

Risks:

- Must manually close cursor
- Can accumulate many open cursors
- Server resources tied up indefinitely

Cursor Types Theory

Tailable Cursors:

- Only work on capped collections
- Remain open even after no results
- Wait for new data to arrive
- Perfect for tailing logs or real-time feeds

AwaitData Cursors:

- Variation of tailable cursor
- Waits briefly for data before returning
- Reduces polling frequency
- Better for real-time applications

Cursor Cleanup Theory

Always close cursors explicitly because:

1. Server holds resources until timeout
2. Each open cursor consumes memory
3. Too many cursors can exhaust server resources
4. Network connections may stay open longer

The try/finally pattern ensures cleanup even if errors occur.

Summary: The Mental Model

When working with MongoDB CRUD and queries, maintain this mental model:

1. **Documents are the unit of atomicity** - Everything happens at document level
2. **Cursors are pointers, not data** - Think about streaming, not loading
3. **Indexes determine performance** - Query structure must match index structure
4. **Write concern is about trade-offs** - Durability vs. speed, always
5. **Schema is in the application** - MongoDB stores what you give it
6. **Operations are network calls** - Batch when possible, project when practical

MongoDB Indexing & Data Modeling: Complete Theory Notes for Senior Developers

---

4. Indexing (VERY IMPORTANT)

The Fundamental Theory of Indexing

What is an Index?

An index is a specialized data structure (B-tree) that stores a small portion of the collection's data in an ordered form, separate from the documents themselves. Think of it as the **table of contents** of a book versus the **actual pages**.

The Book Analogy:

```text
Without Index: To find all pages mentioning "MongoDB", you read every page (Collection Scan)
With Index: Look up "MongoDB" in the index, go directly to those pages (Index Scan)
```

Why Indexes Matter

Without indexes, MongoDB must perform a **collection scan** - examining every document in the collection. This is:

- **O(n) complexity** - time increases linearly with collection size
- **Unacceptable at scale** - a 1TB collection would take hours to query
- **CPU intensive** - every document must be parsed from BSON
- **I/O intensive** - every document must be read from disk

With appropriate indexes:

- **O(log n) complexity** - time increases logarithmically

- **Constant time** lookups for equality matches
- **Efficient range scans** for sorting and filtering
- **Covered queries** possible when all needed fields are in index

The B-Tree Data Structure

MongoDB uses B-Tree (Balanced Tree) indexes:

```text
                    [100]
                   /     \
               [50]       [150]
              /    \      /    \
          [25]   [75]  [125]  [175]
```

Properties of B-Trees:

- Self-balancing (maintains O(log n) search time)
- All leaves at same depth
- Nodes contain multiple keys (unlike binary trees)
- Optimized for disk storage (large node size)
- Supports range scans efficiently

Single Field Index

Theory

A single field index is an index on one field of a document. It's the simplest form of index but incredibly powerful.

Structure:

```text
Index Entry Format: [field value] → [pointer to document on disk]

Example for { age: 1 } index:
```

```
[20] → Document location for age=20
[25] → Document location for age=25
[30] → Document location for age=30
[35] → Document location for age=35
```

Sort Direction:

- `1` for ascending (lowest to highest)
- `-1` for descending (highest to lowest)

The direction matters because the B-Tree stores values in sorted order according to this direction.

When to Use

Use single field indexes when:

- Queries always filter by this single field
- You need to sort by this field
- The field has high cardinality (many unique values)
- This field is used for equality matches

Example with Theory

javascript

```javascript
// Create index on email field (ascending)
db.users.createIndex({ email: 1 })

// Query uses this index
db.users.find({ email: "john@example.com" })

// How MongoDB uses this index:
// 1. Locates "john@example.com" in B-Tree (log n time)
// 2. Follows pointer to document on disk
// 3. Returns document

// Sort also uses index efficiently
db.users.find().sort({ email: 1 })  // Traverses index in order
```

Compound Index

Theory

A compound index is an index on multiple fields. This is where MongoDB indexing gets powerful and complex.

Structure:

```text
Index for { lastname: 1, firstname: 1 }:

[("Smith", "John")] → Document location
[("Smith", "Mary")] → Document location
[("Smith", "Robert")] → Document location
[("Taylor", "James")] → Document location
[("Taylor", "Sarah")] → Document location
```

Key Theoretical Concepts:

1. **Order Matters**: The sequence of fields in a compound index is critical. The index sorts first by the first field, then by the second field within each first field value.
2. **Prefix Principle**: A compound index can support queries on any prefix of its fields:

- Index on `{ a:1, b:1, c:1 }` supports queries on:
  - `{ a }`
  - `{ a, b }`
  - `{ a, b, c }`
- Does NOT support queries on `{ b }` alone or `{ b, c }`

3. **ESR Rule (Equality, Sort, Range)** : The most important rule for compound index design:

- **Equality fields first**: Fields that use exact matches
- **Sort fields next**: Fields used in sort() operations
- **Range fields last**: Fields with $gt, $lt, $in, etc.

Why ESR Works

Theoretical Explanation:

1. **Equality First**: After equality matches, the result set is reduced to a subset. The index has already narrowed down to specific values.
2. **Sort Next**: Within that subset, the data is already sorted by the next field(s) because of how the index is structured.
3. **Range Last**: Range scans within an already-sorted subset are efficient. The index can scan a contiguous range of values.

Example with Theory

javascript

```javascript
// Bad index order (violates ESR)
db.orders.createIndex({ amount: 1, status: 1, date: -1 })

// Query
db.orders.find({
  status: "shipped",           // Equality
  amount: { $gt: 100 }         // Range
}).sort({ date: -1 })          // Sort

// How MongoDB uses the bad index:
// 1. Can't use amount field efficiently because status isn't first
// 2. May need to scan many documents
// 3. Must sort in memory (slow)

// Good index order (follows ESR)
db.orders.createIndex({
  status: 1,    // Equality first
  date: -1,     // Sort next
  amount: 1     // Range last
})

// Now MongoDB:
// 1. Finds all "shipped" documents quickly (equality)
// 2. Within "shipped", data is already sorted by date
// 3. Only checks amount > 100 within this sorted subset
```

Multikey Index (Arrays)

Theory

A multikey index is created when the indexed field contains an array. MongoDB creates index entries for each element in the array.

Structure:

```text
Document: { tags: ["mongodb", "database", "nosql"] }

Multikey index entries:
["mongodb"] → Document location
["database"] → Document location
["nosql"] → Document location
```

Important Theoretical Limitations:

1. **At most one indexed field can be an array per document**. Why? Because if you have two array fields indexed, the number of index entries would multiply:

- Array A with 10 elements × Array B with 10 elements = 100 index entries
- This combinatorial explosion would kill performance

2. **Multikey indexes cannot cover queries** (covered query theory explained later). Why? Because the index contains array elements, not the whole document structure needed to verify all conditions.
3. **Queries on array elements** can use multikey indexes efficiently:

- Find documents where array contains a specific value
- Find documents where any array element matches criteria

Example with Theory

```javascript
// Create multikey index
db.articles.createIndex({ tags: 1 })

// Document
db.articles.insertOne({
  title: "MongoDB Indexing",
  tags: ["database", "mongodb", "performance"]
})

// Query uses multikey index
db.articles.find({ tags: "mongodb" })

// How MongoDB uses this:
// 1. Looks up "mongodb" in index
// 2. Finds pointer to document
// 3. Returns full document

// Special multikey behavior for $elemMatch
db.articles.find({
  tags: {
    $elemMatch: { $eq: "mongodb", $exists: true }
```

```
  }
})
// Uses same index but must verify both conditions
```

Text Index

Theory

A text index is a specialized index type for text search within string content. Unlike regular indexes that match exact values, text indexes understand language and provide sophisticated search capabilities.

How Text Indexes Work:

1. **Tokenization**: Text is split into words (tokens)
2. **Stemming**: Words are reduced to root form (e.g., "running" → "run")
3. **Stop Words**: Common words (the, and, or) are ignored
4. **Term Frequency**: Each term gets a weight based on frequency
5. **Position Tracking**: Word positions enable phrase matching

Internal Structure:

```text
Index entries for "the quick brown fox":
["quick"] → Document location + position info
["brown"] → Document location + position info
["fox"] → Document location + position info
```

Key Theoretical Concepts:

1. **One text index per collection**: You can only have one text index because it needs to know the weighting of all text fields.
2. **Weighted Fields**: You can assign different weights to different fields, making matches in some fields more important than others.
3. **Text Score**: Each matching document gets a relevance score used for sorting.

Example with Theory

```javascript
// Create text index with weights
db.articles.createIndex(
  {
    title: "text",      // Fields to index for text search
    content: "text",
    tags: "text"
  },
  {
    weights: {
      title: 10,        // Matches in title are 10x more important
      content: 5,       // Content matches are 5x important
      tags: 1           // Tags matches are baseline
    },
    name: "TextIndex"
  }
)

// Text search query
db.articles.find(
  { $text: { $search: "mongodb performance" } },
  { score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } })

// How MongoDB processes this:
// 1. Tokenizes "mongodb performance" → ["mongodb", "performance"]
// 2. Stems words (if applicable)
// 3. Searches index for each term
// 4. Calculates score = sum(term frequency × field weight)
// 5. Returns documents sorted by score

// Language-specific search
db.articles.createIndex(
  { content: "text" },
  { default_language: "french" }  // Uses French stemming rules
)
```

TTL Index (Time-To-Live)

Theory

A TTL (Time-To-Live) index automatically removes documents after a specified period. This is MongoDB's built-in data expiration mechanism.

How TTL Works:

1. **Background Thread**: A special thread runs every 60 seconds

2. **Query Execution**: Finds documents where indexed date field < current time - expireAfterSeconds
3. **Batch Deletion**: Deletes found documents in batches
4. **Repeat**: Sleeps, then repeats the process

Theoretical Considerations:

1. **Date Field Required**: The indexed field must be a BSON date type or array of dates
2. **Expiration Precision**: Deletion is not real-time; documents may persist up to 60 seconds past expiration
3. **Performance Impact**: Deletion batches can impact performance; consider scheduling during low-load periods
4. **Replica Sets**: TTL operations propagate through replication; secondaries also delete documents
5. **Limitations**:

- Cannot use compound indexes (must be single field)
- Cannot use with capped collections
- Indexed field must be date type

Example with Theory

javascript

```javascript
// Create TTL index on session data
db.sessions.createIndex(
  { lastActivity: 1 },
  { expireAfterSeconds: 3600 }  // Delete after 1 hour
)

// Insert session
db.sessions.insertOne({
  sessionId: "abc123",
  userId: 12345,
  data: { ... },
  lastActivity: new Date()  // Current timestamp
})

// Document will be deleted after lastActivity + 3600 seconds
// TTL thread wakes up, finds document if lastActivity < now-3600
// Deletes it in next batch

// For sliding expiration, update lastActivity on each request
db.sessions.updateOne(
  { sessionId: "abc123" },
  { $set: { lastActivity: new Date() } }
```

```
)
// TTL timer resets
```

Unique Index

Theory

A unique index ensures that the indexed fields do not store duplicate values. This enforces data integrity at the database level.

How Uniqueness Works:

1. **Insert Time Check**: When inserting, MongoDB checks if the value already exists in the index
2. **Update Time Check**: Similarly checked during updates
3. **Atomic Guarantee**: Uniqueness is maintained even under concurrent operations
4. **Sparse Unique**: Can combine with sparse to only enforce uniqueness on documents that have the field

Important Theoretical Points:

1. **Null is a Value**: If multiple documents miss a unique indexed field, they all have `null` as the value - only one can exist (unless sparse)
2. **Compound Unique**: Uniqueness applies to the combination of fields, not individual fields
3. **_id is Always Unique**: The _id field has a unique index by default
4. **Sharding Implications**: Unique indexes on sharded collections must include the shard key as a prefix

Example with Theory

javascript

```javascript
// Simple unique index
db.users.createIndex({ email: 1 }, { unique: true })

// This works
db.users.insertOne({ email: "john@example.com", name: "John" })

// This fails - duplicate key error
db.users.insertOne({ email: "john@example.com", name: "Jane" })
```

```
// Sparse unique index - only enforces uniqueness if field exists
db.users.createIndex(
  { phoneNumber: 1 },
  { unique: true, sparse: true }
)

// Allowed - multiple documents without phoneNumber
db.users.insertOne({ name: "John" })
db.users.insertOne({ name: "Jane" })

// First phone number set - allowed
db.users.insertOne({ name: "Bob", phoneNumber: "555-1234" })

// Second same phone number - fails
db.users.insertOne({ name: "Alice", phoneNumber: "555-1234" }) // Error

// Compound unique index
db.products.createIndex(
  { sku: 1, warehouse: 1 },
  { unique: true }
)

// All combinations must be unique
db.products.insertOne({ sku: "ABC123", warehouse: "NYC" })  // OK
db.products.insertOne({ sku: "ABC123", warehouse: "LAX" })  // OK (different wareh
ouse)
db.products.insertOne({ sku: "ABC123", warehouse: "NYC" })  // Fails (duplicate co
mbo)
```

Index Concepts - Deep Theory

Index Selection Theory

How MongoDB Chooses Which Index to Use

When you execute a query, MongoDB's query planner evaluates multiple possible indexes and chooses the most efficient one. This process is crucial to understand.

The Query Planning Process:

```text
Query: db.orders.find({ status: "shipped", amount: { $gt: 100 } }).sort({ date: -1
})

Step 1: Query Planner examines available indexes
Step 2: Generates candidate plans using different indexes
Step 3: Runs candidate plans in parallel for a short period
Step 4: Measures performance (docs examined, execution time)
```

```
Step 5: Selects winning plan
Step 6: Caches plan for similar queries
```

Index Selection Criteria:

1. **Selectivity**: How many documents match each indexed field? More selective fields (fewer matches) are preferred.
2. **Sort Avoidance**: Can the index provide results in the requested sort order?
3. **Covered Query Potential**: Can the query be satisfied entirely from the index?
4. **Index Size**: Smaller indexes are generally faster to scan.
5. **Field Order**: Does the index follow ESR for this query?

Example of Index Selection

javascript

```javascript
// Given these indexes:
db.orders.createIndex({ status: 1 })
db.orders.createIndex({ amount: 1 })
db.orders.createIndex({ status: 1, amount: 1, date: -1 })

// Query
db.orders.find({
  status: "shipped",
  amount: { $gt: 100 }
}).sort({ date: -1 })

// Query planner considers:
// Plan A: Use { status:1 } index, filter amount in memory, sort in memory
// Plan B: Use { amount:1 } index, filter status in memory, sort in memory
// Plan C: Use { status:1, amount:1, date:-1 } - can do everything!

// Plan C wins because it:
// 1. Filters on status exactly
// 2. Handles amount range within index
// 3. Returns already sorted by date
```

Covered Queries Theory

What is a Covered Query?

A covered query is a query where **all the fields requested are in the index**, and **all the fields in the query criteria are in the index**. MongoDB can satisfy the query using only the index, without examining documents.

The Magic of Covered Queries:

```text
Without covered query:
Index Lookup → Find document pointers → Fetch documents → Return data
                ↓                        ↓
              (fast)                  (slow - disk I/O)

With covered query:
Index Lookup → Return data directly from index
                ↓
              (fast - no disk I/O for documents)
```

Requirements for a Covered Query:

1. **All query fields in index**: Every field in the filter must be in the index
2. **All projection fields in index**: Every field requested must be in the index
3. **No array fields**: If any indexed field is an array, covered query is impossible
4. **_id exclusion**: Must explicitly exclude `_id` unless it's in the index

Why Covered Queries Matter

Performance Impact:

- **10x-100x faster** than regular queries
- **Reduced disk I/O**: Documents may never be read from disk
- **Lower memory pressure**: Index typically smaller than documents
- **Consistent performance**: Index always in memory (if working set fits)

Example with Theory

```javascript
// Create index covering common queries
db.users.createIndex({ email: 1, name: 1, age: 1 })

// This is a covered query
db.users.find(
  { email: "john@example.com" },        // email in index
  { name: 1, age: 1, _id: 0 }           // name, age in index
)

// Explain confirms covered query
db.users.find(...).explain()
// Look for: "totalDocsExamined": 0  ← Key indicator!
//           "indexOnly": true        ← Covered query
```

```
// This is NOT covered (age not in index)
db.users.find(
  { email: "john@example.com" },
  { name: 1, age: 1, salary: 1, _id: 0 }  // salary not in index
)
// Must fetch documents to get salary field
```

Index Performance Trade-offs Theory

The Cost-Benefit Analysis of Indexes

Every index is a trade-off. Understanding these trade-offs is what separates junior from senior developers.

Benefits of Indexes:

- Faster queries (reads)
- Efficient sorting
- Covered query possibilities
- Reduced resource usage for queries

Costs of Indexes:

- **Write Performance**: Every insert, update, delete must update all indexes
- **Storage Space**: Each index consumes disk space
- **Memory Usage**: Indexes compete for cache space in RAM
- **Query Planner Overhead**: More indexes = more plans to evaluate

The Indexing Trade-off Formula

```text
Total System Cost = (Read Cost × Read Frequency) + (Write Cost × Write Frequency)

Adding an index:
- Decreases Read Cost (faster queries)
- Increases Write Cost (more updates per operation)
- Net benefit if: ΔReadCost × ReadFreq > ΔWriteCost × WriteFreq
```

Index Selection Matrix

Workload Type	Index Strategy	Reasoning
Read-heavy (90% reads)	Many indexes, compound indexes	Reads benefit greatly, writes suffer but ar
Write-heavy (90% writes)	Minimal indexes, single-field only	Every index slows writes; only index essen fields
Mixed	Balanced approach, ESR-designed	Index only what's needed, optimize for co queries
Real-time analytics	Covered query indexes	Maximum read performance, accept write overhead

Practical Examples of Trade-offs

javascript

```javascript
// Scenario 1: User authentication service
// 90% reads (login checks), 10% writes (new users)

// Good strategy - index email heavily
db.users.createIndex({ email: 1 })  // Login queries are fast
// Write penalty: Small (one index update per new user)

// Scenario 2: Logging system
// 5% reads (debugging), 95% writes (application logs)

// Bad strategy - many indexes
db.logs.createIndex({ timestamp: 1 })
db.logs.createIndex({ level: 1 })
db.logs.createIndex({ userId: 1 })
// Each log write must update 3 indexes (slow!)

// Better strategy - minimal indexes
db.logs.createIndex({ timestamp: 1 }, { expireAfterSeconds: 86400 })
// Only one index for TTL and occasional time-based queries

// Scenario 3: E-commerce product search
// Many complex read patterns, moderate writes

// Good strategy - targeted compound indexes
db.products.createIndex({ category: 1, price: 1, inStock: 1 })  // Common search
db.products.createIndex({ tags: 1 })  // Tag search
// Write penalty: Updates affect both indexes, but query patterns justify it
```

5. Data Modeling (Most Important for MongoDB)

The Fundamental Theory of Data Modeling in MongoDB

Why Data Modeling is Different in MongoDB

In SQL databases, data modeling is about **normalization** - eliminating redundancy through relationships. In MongoDB, data modeling is about **application access patterns** - structuring data to match how your application queries it.

The Paradigm Shift:

```text
SQL Mindset: "How do I structure this data to avoid duplication?"
MongoDB Mindset: "How does my application need to access this data?"
```

The Three Fundamental Questions

Before modeling any data in MongoDB, ask:

1. **What are the access patterns?**
   - Which queries run most often?
   - What data is accessed together?
   - What are the read/write ratios?
2. **What are the data relationships?**
   - Is it one-to-one, one-to-many, many-to-many?
   - How large are the related data sets?
   - Does the relationship need to be bidirectional?
3. **What are the performance requirements?**
   - How fast must reads be?
   - How fast must writes be?
   - What consistency is required?

Embedding vs Referencing Theory

The Core Decision

Embedding: Storing related data inside the same document
Referencing: Storing related data in separate documents, linked by IDs

When to Embed - The Theory

Embedding is optimal when:

1. **Data is accessed together**: You always need the related data when querying the parent
2. **One-to-few relationships**: The related data set is small and bounded
3. **No independent access**: Related data is never queried on its own
4. **Atomic operations needed**: You need to update parent and child atomically
5. **Data locality matters**: Performance is critical and you want to minimize disk seeks

The "One-to-Few" Rule:

```text
Embed if cardinality is in the "few" range (< 100-1000)
Example: User's addresses (most users have 1-3 addresses)
```

When to Reference - The Theory

Referencing is optimal when:

1. **Data is accessed independently**: You query related data separately
2. **Many-to-many relationships**: Complex relationships with multiple connections
3. **Large child documents**: Children are large enough to push document size limits
4. **Frequent updates to children**: Updates to child data shouldn't affect parent
5. **One-to-many with "many" being large**: Thousands or millions of related documents

Example with Theory

```javascript
// EMBEDDING - Good for addresses (one-to-few)
```

```
{
  _id: ObjectId("user123"),
  name: "John Doe",
  addresses: [
    {
      type: "home",
      street: "123 Main St",
      city: "Boston",
      primary: true
    },
    {
      type: "work",
      street: "456 Market St",
      city: "Cambridge"
    }
  ]
}
// Why this works:
// - User always logs in with addresses
// - Small set (1-3 addresses)
// - Addresses not queried independently
// - Atomic updates when adding addresses

// REFERENCING - Good for orders (one-to-many large)
{
  _id: ObjectId("user123"),
  name: "John Doe",
  orderIds: [
    ObjectId("order456"),
    ObjectId("order789")
  ]
}

// Separate orders collection
{
  _id: ObjectId("order456"),
  userId: ObjectId("user123"),
  total: 299.99,
  items: [...],
  createdAt: ISODate("2024-01-15")
}
// Why this works:
// - Users have many orders (hundreds+)
// - Orders queried separately (order history)
// - Orders large documents
// - Updates to orders don't affect user document
```

Relationship Patterns - Deep Theory

One-to-One Theory

Definition: Each entity in A corresponds to exactly one entity in B.

Modeling Options:

1. **Embedding** (Preferred):
   - Keep related data in the same document
   - No joins needed
   - Atomic operations
2. **Referencing** (When needed):
   - Separate documents with reference
   - Used when data is large or accessed separately

Theoretical Decision Tree:

```text
Is the related data always accessed with parent?
├─ YES → Embed
└─ NO → Is the related data large (> 16MB)?
    ├─ YES → Reference
    └─ NO → Consider both, prefer embedding
```

Example - One-to-One

```javascript
// One-to-One: User and Profile

// OPTION 1: Embedding (Recommended)
{
  _id: ObjectId("user123"),
  username: "johndoe",
  email: "john@example.com",
  profile: {
    firstName: "John",
    lastName: "Doe",
    avatar: "avatar.jpg",
    bio: "MongoDB enthusiast",
    preferences: {
      theme: "dark",
      notifications: true
    }
  }
}
// All user data in one place, efficient access

// OPTION 2: Referencing (Only if profile is huge)
// users collection
{
  _id: ObjectId("user123"),
  username: "johndoe",
  email: "john@example.com",
  profileId: ObjectId("profile456")
}
```

```
// profiles collection
{
  _id: ObjectId("profile456"),
  userId: ObjectId("user123"),
  bio: "Very long bio with history...",
  avatar: "large_image_data...",
  settings: { /* complex settings */ }
}
// Two queries or $lookup to get all data
```

One-to-Many Theory

Definition: One entity in A corresponds to many entities in B.

Three Cardinality Cases:

1. **One-to-Few** (up to ~100): Embed
2. **One-to-Many** (up to ~1000): Reference with array of IDs
3. **One-to-Squillions** (unbounded): Reference with parent reference in child

Theoretical Framework:

text

```
Cardinality          Strategy               Example
_____________________________________________________

Few (<100)          Embed                 User's credit cards
Many (100-1000)     Array of IDs          User's orders
Squillions (∞)      Parent reference      Product reviews
```

Examples - One-to-Many

javascript

```
// ONE-TO-FEW: Blog posts with tags (max 10 tags)
{
  _id: ObjectId("post123"),
  title: "MongoDB Indexing Deep Dive",
  content: "...",
  tags: ["mongodb", "database", "performance", "indexing"]
}
// Tags rarely exceed 5-10, perfect for embedding

// ONE-TO-MANY: User with orders (hundreds of orders)
// User document
{
  _id: ObjectId("user123"),
  name: "John Doe",
  orderIds: [
    ObjectId("order456"),
```

```
    ObjectId("order457"),
    // ... up to ~1000 orders
  ]
}

// Order documents (separate collection)
{
  _id: ObjectId("order456"),
  userId: ObjectId("user123"),
  total: 299.99,
  items: [...],
  date: ISODate("2024-01-15")
}
// Efficient: User document stays small, orders can grow

// ONE-TO-SQUILLIONS: Product with reviews (millions)
// Product document
{
  _id: ObjectId("product123"),
  name: "MongoDB: The Definitive Guide",
  price: 49.99
  // No array of review IDs - would overflow 16MB!
}

// Review documents (with parent reference)
{
  _id: ObjectId("review456"),
  productId: ObjectId("product123"),  // Reference to parent
  userId: ObjectId("user789"),
  rating: 5,
  comment: "Excellent book!",
  createdAt: ISODate("2024-01-15")
}
// Query: db.reviews.find({ productId: product123 })
```

Many-to-Many Theory

Definition: Multiple entities in A correspond to multiple entities in B.

Modeling Approaches:

1. **Two-Way Embedding**: Each document references the other (for small sets)
2. **Separate Join Collection**: Dedicated collection for relationships (for large sets)
3. **Embedding with References**: Embed array of IDs in both documents

Theoretical Considerations:

- **Symmetry**: Is the relationship queried from both sides?
- **Cardinality**: How many on each side?
- **Additional Data**: Does the relationship itself have attributes (like "since date")?

Examples - Many-to-Many

javascript

```javascript
// MANY-TO-MANY: Students and Courses

// APPROACH 1: Two-way embedding (for small sets)
// Student document
{
  _id: ObjectId("student123"),
  name: "John Doe",
  courses: [
    { courseId: ObjectId("course456"), enrolledDate: ISODate("2024-01-10") },
    { courseId: ObjectId("course789"), enrolledDate: ISODate("2024-01-12") }
  ]
}

// Course document
{
  _id: ObjectId("course456"),
  title: "MongoDB Data Modeling",
  students: [
    { studentId: ObjectId("student123"), enrolledDate: ISODate("2024-01-10") },
    { studentId: ObjectId("student456"), enrolledDate: ISODate("2024-01-11") }
  ]
}
// Works for small classes (<100 students), but duplicate data

// APPROACH 2: Join collection (for large scale)
// Students collection
{
  _id: ObjectId("student123"),
  name: "John Doe"
}

// Courses collection
{
  _id: ObjectId("course456"),
  title: "MongoDB Data Modeling"
}

// Enrollments collection (the "join" table)
{
  _id: ObjectId("enrollment789"),
  studentId: ObjectId("student123"),
  courseId: ObjectId("course456"),
  enrolledDate: ISODate("2024-01-10"),
  grade: null,
  status: "active"
}
// Best for:
// - Large datasets (10,000+ students)
// - Relationship has its own data (grades, status)
// - Need to query from both sides efficiently
```

Denormalization Theory

What is Denormalization?

Denormalization is the intentional duplication of data across multiple documents to improve read performance. It's a trade-off: **consistency overhead for read performance**.

The Denormalization Spectrum:

```text
Normalized                      Denormalized
(No duplication)    <----->     (Heavy duplication)
    |                               |
    |                               |
Less reads,                 More reads,
more joins                  fewer joins
```

When to Denormalize

Denormalization is beneficial when:

1. **Read-to-write ratio is high**: Data is read often, written rarely
2. **Performance is critical**: Joins are too slow for your use case
3. **Data is mostly static**: The duplicated data doesn't change often
4. **You need history snapshots**: You want to preserve how things looked at a point in time

The Denormalization Formula:

```text
Benefit = ReadFrequency × ReadComplexity
Cost = WriteFrequency × (Number of Duplicates)

Denormalize if: Benefit > Cost
```

Examples with Theory

```javascript
// NORMALIZED APPROACH (No duplication)
// Products
```

```
{
  _id: ObjectId("product123"),
  name: "Laptop",
  price: 999.99
}

// Orders
{
  _id: ObjectId("order456"),
  productId: ObjectId("product123"),
  quantity: 2,
  orderDate: ISODate("2024-01-15")
}
// To display order with product name:
// Need to join or second query

// DENORMALIZED APPROACH (With duplication)
// Orders with embedded product info
{
  _id: ObjectId("order456"),
  product: {
    _id: ObjectId("product123"),
    name: "Laptop",
    price: 999.99
  },
  quantity: 2,
  orderDate: ISODate("2024-01-15")
}
// Single query gets everything!
// BUT: If product price changes, old orders show new price

// Historical denormalization (preserve snapshot)
{
  _id: ObjectId("order456"),
  product: {
    _id: ObjectId("product123"),
    name: "Laptop",
    priceAtPurchase: 999.99  // Fixed at order time
  },
  quantity: 2,
  orderDate: ISODate("2024-01-15")
}
// Best of both: single query, historical accuracy
// Price changes don't affect old orders
```

Schema Design Patterns - Deep Theory

Bucket Pattern

Theory

The Bucket Pattern groups related data that would naturally be many separate documents into "buckets" - single documents containing an array of related sub-documents.

Why Buckets Matter:

Time-series data presents a challenge: millions of tiny documents. Each document has overhead (12 bytes for _id, indexes, etc.). Bucketing reduces:

- Document count overhead
- Index size
- Number of round trips for time-range queries

The Math:

```text
Without Bucketing:
1,000,000 readings = 1,000,000 documents
Overhead: ~100 bytes per document = 100MB overhead

With Bucketing (100 readings per bucket):
1,000,000 readings = 10,000 documents
Overhead: ~100 bytes × 10,000 = 1MB overhead
99% reduction in overhead!
```

Example with Theory

```javascript
// WITHOUT BUCKETING (Bad for time-series)
// Each reading is a document
{
  _id: ObjectId(),
  sensorId: "temp-001",
  timestamp: ISODate("2024-01-15T10:00:00Z"),
  value: 22.5
}
{
  _id: ObjectId(),
  sensorId: "temp-001",
  timestamp: ISODate("2024-01-15T10:01:00Z"),
  value: 22.6
}
// 1440 documents per day per sensor!
// Index on {sensorId, timestamp} huge

// WITH BUCKETING (Optimal)
```

```
{
  _id: ObjectId(),
  sensorId: "temp-001",
  bucketDate: ISODate("2024-01-15"),  // Date for this bucket
  readings: [
    { timestamp: ISODate("2024-01-15T10:00:00Z"), value: 22.5 },
    { timestamp: ISODate("2024-01-15T10:01:00Z"), value: 22.6 },
    // ... up to 1440 readings per day
  ],
  metadata: {
    unit: "celsius",
    location: "room-101",
    model: "sensor-v2"
  },
  stats: {
    min: 18.2,
    max: 26.7,
    avg: 22.4,
    count: 1440
  }
}
// One document per sensor per day!
// Index on {sensorId, bucketDate} tiny
// Queries for time range: find bucket, filter array
```

When to Use:

- IoT sensor data
- Stock tickers
- Application logs
- Any high-frequency time-series data

Subset Pattern

Theory

The Subset Pattern addresses the problem of loading large documents when you only need a small portion of the data. It splits documents into "hot" (frequently accessed) and "cold" (rarely accessed) subsets.

The Problem:

```text
Full document (1MB) includes:
- Frequently accessed fields: name, price (1KB)
- Rarely accessed fields: description, reviews, specs (1023KB)
```

```
Loading full document for every list view wastes:
- Memory
- Network bandwidth
- Processing time
```

The Solution:

Split into two collections:

- Main collection: Frequently accessed fields (small)
- Detail collection: Full data (large)

Example with Theory

```javascript
// WITHOUT SUBSET (Inefficient)
// Products collection with all data
{
  _id: ObjectId("product123"),
  name: "MongoDB Guide",
  price: 49.99,
  description: "Very long description...",  // Rarely read
  reviews: [ ... ],                         // Rarely read
  specs: { ... },                           // Rarely read
  images: [ ... ]                           // Rarely read
}
// Product listing page loads 100 products = 100MB data!

// WITH SUBSET PATTERN (Optimal)
// products collection (hot data)
{
  _id: ObjectId("product123"),
  name: "MongoDB Guide",
  price: 49.99,
  thumbnail: "thumb.jpg",
  reviewCount: 128,
  avgRating: 4.5
}
// Product listing: 100 products = 100KB data (1000x less!)

// product_details collection (cold data)
{
  _id: ObjectId("product123"),  // Same _id as product
  description: "Very long description...",
  reviews: [ ... ],
  specs: { ... },
  images: [ ... ]
}
// Only loaded when user clicks "Details"
```

When to Use:

- Product catalogs
- User profiles with extensive data
- Any list-detail pattern in UI
- Large documents with infrequently accessed fields

Extended Reference Pattern

Theory

The Extended Reference pattern denormalizes just enough frequently accessed data from related documents to avoid joins for common queries, while maintaining the ability to update source data.

The Problem:

text

```
Orders need product names and prices, but:
- If you embed full product, updates are hard
- If you reference product, you need a join
- If you duplicate name/price, it's outdated when product changes
```

The Solution:

Embed a snapshot of the most frequently accessed fields from the related document. These fields rarely change, or changes are acceptable to propagate slowly.

Example with Theory

javascript

```javascript
// Products collection (source of truth)
{
  _id: ObjectId("product123"),
  name: "MongoDB Guide",
  price: 49.99,
  description: "...",
  inventory: 100,
  supplier: "Publisher Inc"
}

// Orders collection with extended reference
{
```

```
  _id: ObjectId("order456"),
  userId: ObjectId("user789"),
  items: [
    {
      productId: ObjectId("product123"),
      name: "MongoDB Guide",        // Denormalized from product
      priceAtPurchase: 49.99,       // Snapshot price
      quantity: 2
    }
  ],
  total: 99.98,
  orderDate: ISODate("2024-01-15")
}

// When product price changes:
// - New orders use new price
// - Old orders preserve historical price
// - No need to update old orders

// For fields that should update (like product name typo fix):
// Run periodic batch update
db.orders.updateMany(
  { "items.productId": productId },
  { $set: { "items.$[elem].name": newName } },
  { arrayFilters: [{ "elem.productId": productId }] }
)
```

When to Use:

- E-commerce orders (product name/price snapshots)
- Activity feeds (denormalize actor names)
- Comments (denormalize author name)
- Any read-heavy scenario with slowly changing reference data

---

Summary: The Data Modeling Decision Framework

The Complete Decision Tree

When approaching any data modeling problem in MongoDB, follow this framework:

```text
START: What are the access patterns?
│
├─ Are documents always accessed together?
```

```
│   ├─ YES → Consider embedding
│   └─ NO → Consider referencing
│
├─ What's the cardinality?
│   ├─ Few (1-100) → Embed
│   ├─ Many (100-1000) → Array of IDs
│   └─ Squillions (∞) → Parent reference
│
├─ What's the read/write ratio?
│   ├─ Read-heavy → Consider denormalization
│   └─ Write-heavy → Prefer normalization
│
├─ Are there large, rarely used fields?
│   ├─ YES → Consider Subset Pattern
│   └─ NO → Proceed
│
├─ Is this time-series data?
│   ├─ YES → Consider Bucket Pattern
│   └─ NO → Proceed
│
└─ Do you need snapshots of related data?
    ├─ YES → Consider Extended Reference
    └─ NO → Standard embedding/referencing is fine
```

The Golden Rules of MongoDB Data Modeling

1. **Model for your application's queries, not for the data structure**
2. **Data that's accessed together should be stored together**
3. **Consider both read and write patterns, not just one**
4. **Plan for growth - today's "few" may be tomorrow's "many"**
5. **Denormalization is a tool, not a crutch - use it intentionally**
6. **Document size limits are real - respect the 16MB boundary**
7. **Indexes are your friends, but they have costs**
8. **There's no single right answer - trade-offs everywhere**

MongoDB Aggregation Framework & Relationships: Complete Theory Notes for Senior Developers

---

6. Aggregation Framework (High Priority)

The Fundamental Theory of Aggregation

What is the Aggregation Framework?

The Aggregation Framework is MongoDB's built-in data processing pipeline. Think of it as a **data processing assembly line** where documents enter one end, pass through various processing stages, and transformed results emerge from the other end.

The Pipeline Concept:

```text
Collection → Stage 1 → Stage 2 → Stage 3 → ... → Result
             [$match]  [$group]  [$sort]

Each stage:
- Receives documents from previous stage
- Transforms them in some way
- Passes results to next stage
```

Why Aggregation Exists

Before aggregation, MongoDB had:

- **Simple find()** : Basic filtering and projection
- **map-reduce** : Powerful but complex and slow
- **$where** : JavaScript evaluation (very slow)

The Aggregation Framework was created to provide:

1. **Performance**: Native C++ implementation, not JavaScript
2. **Expressiveness**: Complex transformations in readable pipeline
3. **Optimization**: Automatic pipeline optimization
4. **Streaming**: Process documents as they flow through pipeline

The Mental Model

Think of aggregation like a **Unix pipeline**:

```text
cat collection | grep "active" | sort | group | print
    [$match]      [$match]   [$sort] [$group] [$project]
```

Each stage:

- **Takes input** (documents from previous stage)
- **Performs an operation** (filter, transform, group)
- **Produces output** (documents for next stage)

Core Aggregation Stages - Deep Theory

$match - The Filtering Stage

Theory

`$match` filters documents, allowing only those that match specified conditions to pass through to the next stage. It's the most important stage for performance optimization.

Why $match First is Critical:

The aggregation pipeline processes documents sequentially. If `$match` is early:

- Fewer documents flow through subsequent stages
- Less memory and CPU used

- Can use indexes (if `$match` is first)

The Optimization Rule: Always put `$match` as early as possible in the pipeline.

Example with Theory

```javascript
// GOOD - $match first
db.orders.aggregate([
  { $match: { status: "shipped" } },           // Uses index on status
  { $group: { _id: "$customerId", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }
])

// BAD - $match after expensive operations
db.orders.aggregate([
  { $group: { _id: "$customerId", total: { $sum: "$amount" } } },
  { $match: { total: { $gt: 1000 } } },        // Already grouped all orders!
  { $sort: { total: -1 } }
])

// How MongoDB optimizes:
// 1. Recognizes $match at beginning
// 2. Uses index on status if available
// 3. Only passes 10,000 matching docs instead of 1,000,000 total
```

$group - The Grouping Stage

Theory

`$group` divides documents into groups and performs aggregation operations on each group. This is the most complex and powerful stage.

Conceptual Model:

```text
Input Documents → Group by _id expression → One output document per group
                        ↓
               Accumulators track values
               ($sum, $avg, $push, etc.)
```

The _id Field:

- Defines what constitutes a group
- Can be a single field: `{ _id: "$status" }`

- Can be compound: `{ _id: { status: "$status", year: { $year: "$date" } } }`
- Can be null: `{ _id: null }` (all documents in one group)

Accumulators Theory:

Accumulators maintain state as they process each document in the group:

Accumulator	What it does	Memory Usage
`$sum`	Running total	Single number
`$avg`	Running average	Count + running total
`$min`/$max`	Track min/max	Single value
`$push`	Build array	Grows with group size
`$addToSet`	Build unique array	Grows, checks uniquen

Memory Warning: `$push` on large groups can exceed 16MB document limit!

Example with Theory

javascript

```javascript
// Simple grouping by status
db.orders.aggregate([
  {
    $group: {
      _id: "$status",                     // Group by status field
      count: { $sum: 1 },                  // Count documents per group
      totalAmount: { $sum: "$amount" },    // Sum of amounts
      avgAmount: { $avg: "$amount" },       // Average amount
      firstOrder: { $min: "$orderDate" },  // Earliest order
      lastOrder: { $max: "$orderDate" },   // Latest order
      orderIds: { $push: "$_id" }          // Array of all order IDs (careful!)
    }
  }
])

// Compound grouping
db.sales.aggregate([
  {
    $group: {
```

```
      _id: {
        year: { $year: "$date" },
        month: { $month: "$date" },
        product: "$productId"
      },
      totalSales: { $sum: "$amount" }
    }
  }
])

// How $group processes internally:
// 1. Creates a group object in memory for each unique _id
// 2. For each document, finds its group, updates accumulators
// 3. After all documents, outputs one document per group
// 4. If groups exceed 100MB, writes to disk (slow!)
```

$project - The Reshaping Stage

Theory

`$project` reshapes documents by including, excluding, adding, or transforming fields. It's like SQL's SELECT statement but much more powerful.

Capabilities:

1. **Field selection**: Include/exclude fields (1 or 0)
2. **Field renaming**: `newName: "$oldName"`
3. **Computed fields**: Create new fields using expressions
4. **Type conversions**: Convert strings to dates, etc.
5. **String operations**: Concatenate, substring, etc.
6. **Arithmetic**: Add, multiply, etc.
7. **Conditional logic**: $cond for if-then-else

The _id Special Case:

- `_id: 1` (include - default)
- `_id: 0` (exclude)
- Can exclude `_id` even when including other fields

Example with Theory

javascript

```
// Basic projection - include/exclude
```

```
db.users.aggregate([
  {
    $project: {
      name: 1,                    // Include name
      email: 1,                    // Include email
      _id: 0,                      // Exclude _id
      password: 0                   // Exclude password (if existed)
    }
  }
])

// Field transformation and creation
db.orders.aggregate([
  {
    $project: {
      orderId: "$_id",               // Rename _id to orderId
      customerName: { $concat: ["$firstName", " ", "$lastName"] },  // Combine fie
lds
      totalWithTax: { $multiply: ["$subtotal", 1.08] },  // Calculate
      orderYear: { $year: "$orderDate" },                  // Extract from date
      orderMonth: { $month: "$orderDate" },
      status: { $toUpper: "$status" },                     // Transform string
      isHighValue: {                                        // Conditional
        $cond: {
          if: { $gte: ["$total", 1000] },
          then: "PREMIUM",
          else: "STANDARD"
        }
      },
      itemCount: { $size: { $ifNull: ["$items", []] } }    // Array length
    }
  }
])

// How $project processes:
// 1. For each input document, creates a new document
// 2. Evaluates each expression in order
// 3. Previous expressions can be used in later ones
// 4. Output document is built in memory
```

$sort - The Ordering Stage

Theory

`$sort` reorders documents based on specified fields. It can be a performance bottleneck if not used correctly.

Sort Mechanics:

- **Indexed sort**: Uses index if possible (fast)
- **In-memory sort**: Loads all into memory (slow, limited to 100MB)

- **Disk-based sort**: Spills to disk when memory exceeded (very slow)

Memory Limits:

- Default: 100MB for sort operations
- Can allow disk use: `{ allowDiskUse: true }`
- Disk use is SLOW - avoid if possible

Optimization Principle: Sort should come after stages that reduce document count, before stages that increase it.

Example with Theory

javascript

```javascript
// Simple sort
db.products.aggregate([
  { $sort: { price: -1, name: 1 } }  // Desc price, asc name
])

// Sort with memory management
db.orders.aggregate([
  { $match: { status: "shipped" } },              // Reduce documents first
  { $sort: { total: -1 } },                         // Sort reduced set
  { $limit: 10 }                                     // Return top 10
], { allowDiskUse: true })                          // If sort exceeds 100MB

// How sort optimization works:
// GOOD: Uses index
db.orders.aggregate([
  { $match: { status: "shipped" } },    // Uses index on status
  { $sort: { orderDate: -1 } }           // Uses index on orderDate
])

// BAD: Cannot use index
db.orders.aggregate([
  { $group: { _id: "$customerId", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }               // Must sort computed field in memory
])
```

$limit and $skip - The Pagination Stages

Theory

`$limit` restricts the number of documents passed to the next stage. `$skip` discards a specified number of documents from the beginning.

Performance Reality:

- `$limit` is efficient - stops processing after N documents
- `$skip` is inefficient - must process and discard skipped documents
- Large skips in aggregation are as bad as in find()

Optimization Strategy:

- Put `$match` before `$skip` to minimize skipped documents
- Use range-based pagination instead of skip for large offsets

Example with Theory

javascript

```javascript
// Basic pagination (inefficient for large skips)
db.products.aggregate([
  { $match: { category: "electronics" } },
  { $sort: { price: -1 } },
  { $skip: 1000 },        // Must process first 1000 docs and discard them
  { $limit: 20 }
])

// More efficient: Use range-based pagination
db.products.aggregate([
  { $match: {
    category: "electronics",
    $or: [
      { price: { $lt: lastPrice } },                    // Price less than last
      { price: lastPrice, _id: { $gt: lastId } }        // Or same price, higher I
D
    ]
  }},
  { $sort: { price: -1, _id: 1 } },
  { $limit: 20 }
])
```

$lookup - The Join Stage

Theory

`$lookup` performs a left outer join with another collection. It's MongoDB's answer to SQL JOINs, but with important differences.

How $lookup Works:

1. For each document in the input pipeline
2. Takes the value of `localField`

3. Searches the foreign collection for documents where `foreignField` equals that value
4. Creates an array field containing matching documents
5. Outputs the original document with the new array

Performance Reality:

- $lookup is expensive - it's essentially a nested loop join
- Requires indexes on foreignField
- Can't use indexes on localField (it's the source document's field)
- Each document triggers a query on the foreign collection

Example with Theory

javascript

```javascript
// Basic lookup - orders with customer details
db.orders.aggregate([
  {
    $lookup: {
      from: "customers",                    // Foreign collection
      localField: "customerId",              // Field in orders
      foreignField: "_id",                   // Field in customers
      as: "customer"                         // Output array field
    }
  }
])

// Result structure:
{
  _id: ObjectId("order123"),
  customerId: ObjectId("cust456"),
  amount: 299.99,
  customer: [                                // Array of matching customers
    {
      _id: ObjectId("cust456"),
      name: "John Doe",
      email: "john@example.com"
    }
  ]
}

// Better: Unwind the array for easier access
db.orders.aggregate([
  {
    $lookup: {
      from: "customers",
      localField: "customerId",
      foreignField: "_id",
      as: "customer"
    }
  },
  { $unwind: "$customer" },                  // Convert array to object
```

```
  {
    $project: {
      amount: 1,
      "customer.name": 1,
      "customer.email": 1
    }
  }
])

// Complex lookup with pipeline (MongoDB 3.6+)
db.orders.aggregate([
  {
    $lookup: {
      from: "products",
      let: { productIds: "$items.productId" },     // Variables from order
      pipeline: [
        {
          $match: {
            $expr: { $in: ["$_id", "$$productIds"] },
            status: "active"                        // Additional filter
          }
        },
        { $project: { name: 1, price: 1 } }          // Only needed fields
      ],
      as: "productDetails"
    }
  }
])

// How $lookup executes:
// 1. Orders collection scanned (or index used from earlier stages)
// 2. For each order, queries customers collection
// 3. If customers._id indexed, each query is fast O(log n)
// 4. But still N queries for N orders!
```

$unwind - The Array Deconstruction Stage

Theory

`$unwind` deconstructs an array field, creating a separate document for each array element. It's like "flattening" nested data.

Why $unwind is Needed:

- Arrays are great for storage but hard for aggregation
- To group by array elements, you need to unwind first
- To join based on array elements, you need to unwind

The Cardinality Explosion:

```text
```

```
Document with array of 10 items → 10 output documents
If you unwind multiple arrays: 10 × 10 = 100 documents
This can cause massive pipeline growth!
```

Example with Theory

javascript

```javascript
// Original document
{
  _id: ObjectId("order123"),
  customer: "John Doe",
  items: [
    { product: "Laptop", price: 999, qty: 1 },
    { product: "Mouse", price: 29, qty: 2 }
  ]
}

// After $unwind
db.orders.aggregate([
  { $unwind: "$items" }
])

// Result: Two documents
{ _id: "order123", customer: "John Doe", items: { product: "Laptop", price: 999, q
ty: 1 } }
{ _id: "order123", customer: "John Doe", items: { product: "Mouse", price: 29, qty
: 2 } }

// With options
db.orders.aggregate([
  {
    $unwind: {
      path: "$items",
      preserveNullAndEmptyArrays: true,    // Keep documents with no items
      includeArrayIndex: "itemIndex"        // Add index position
    }
  }
])

// Common pattern: Unwind, group, reconstruct
db.orders.aggregate([
  { $unwind: "$items" },
  {
    $group: {
      _id: "$items.product",
      totalSold: { $sum: "$items.qty" },
      revenue: { $sum: { $multiply: ["$items.price", "$items.qty"] } }
    }
  }
])

// How $unwind scales:
// Input: 1000 orders, each with 10 items = 10,000 documents after unwind
// Be careful with multiple unwinds!
```

$facet - The Multi-Faceted Aggregation Stage

Theory

`$facet` processes multiple aggregation pipelines within a single stage on the same set of input documents. It's like running parallel aggregations.

Use Cases:

- Dashboard with multiple metrics
- Search results with facets (categories, prices ranges)
- Paginated results with total count
- Multiple aggregations in one query

Performance Reality:

- All facet pipelines run in parallel
- Each pipeline processes the SAME input documents
- Results are combined into a single output document
- Can be memory intensive with large inputs

Example with Theory

javascript

```javascript
// E-commerce product dashboard
db.products.aggregate([
  { $match: { category: "electronics" } },
  {
    $facet: {
      // Paginated results
      products: [
        { $skip: 20 },
        { $limit: 10 },
        { $project: { name: 1, price: 1 } }
      ],
      // Total count
      totalCount: [
        { $count: "count" }
      ],
      // Price facets
      priceRanges: [
        {
          $bucket: {
            groupBy: "$price",
            boundaries: [0, 100, 500, 1000, 5000],
            default: "Other",
```

```
            output: {
              count: { $sum: 1 },
              avgPrice: { $avg: "$price" }
            }
          }
        }
      ],
      // Top brands
      topBrands: [
        { $group: { _id: "$brand", count: { $sum: 1 } } },
        { $sort: { count: -1 } },
        { $limit: 5 }
      ]
    }
  }
])

// Result structure:
{
  products: [ { name: "Laptop", price: 999 }, ... ],
  totalCount: [ { count: 1567 } ],
  priceRanges: [
    { _id: 0, count: 45, avgPrice: 67.50 },
    { _id: 100, count: 234, avgPrice: 299.99 },
    ...
  ],
  topBrands: [ { _id: "Apple", count: 156 }, ... ]
}

// How $facet processes:
// 1. $match produces 1567 documents
// 2. These 1567 documents are fed to ALL four pipelines
// 3. Each pipeline runs independently
// 4. Results combined into one document
```

Advanced Aggregation Concepts

Pipeline Optimization Theory

How MongoDB Optimizes Pipelines

The aggregation engine automatically reorders stages when safe to improve performance:

Automatic Optimizations:

1. **Projection Pushdown**: Moves $project before other stages when possible
2. **Match Coalescing**: Combines multiple $match stages
3. **Limit Pushdown**: Moves $limit earlier when safe

4. **Skip/Limit Reordering**: Swaps skip and limit when beneficial

Example of Optimization:

```javascript
// Your pipeline:
db.orders.aggregate([
  { $sort: { orderDate: -1 } },
  { $match: { status: "shipped" } },
  { $limit: 10 }
])

// MongoDB may optimize to:
db.orders.aggregate([
  { $match: { status: "shipped" } },   // Match first (uses index)
  { $sort: { orderDate: -1 } },          // Sort smaller set
  { $limit: 10 }                          // Limit last
])
```

Manual Optimization Strategies

1. Early Filtering:

```javascript
// BAD
db.orders.aggregate([
  { $unwind: "$items" },
  { $match: { "items.product": "Laptop" } }  // Unwind first = huge pipeline
])

// GOOD
db.orders.aggregate([
  { $match: { "items.product": "Laptop" } },  // Filter first
  { $unwind: "$items" },
  { $match: { "items.product": "Laptop" } }   // Final filter after unwind
])
```

2. Project Before Group:

```javascript
// BAD
db.orders.aggregate([
  { $group: {
    _id: "$customerId",
    total: { $sum: "$amount" },
    month: { $first: { $month: "$date" } }  // Computing in group
  }}
])

// GOOD
db.orders.aggregate([
  { $project: {
    customerId: 1,
```

```
    amount: 1,
    month: { $month: "$date" }                // Compute once
  }},
  { $group: {
    _id: "$customerId",
    total: { $sum: "$amount" },
    months: { $addToSet: "$month" }
  }}
])
```

3. Use Indexes:

```javascript
// Create index for common aggregation pattern
db.orders.createIndex({ status: 1, orderDate: -1, amount: 1 })

// Pipeline uses this index for:
db.orders.aggregate([
  { $match: { status: "shipped" } },        // Equality
  { $sort: { orderDate: -1 } },               // Sort
  { $group: { _id: "$customerId", total: { $sum: "$amount" } } }
])
```

Aggregation Performance Theory

Memory Management

The 100MB Limit:

- Each stage has a 100MB memory limit
- Exceeded stages must spill to disk (slow)
- Use `allowDiskUse: true` but avoid when possible

Memory-Intensive Operations:

- `$group` with many distinct keys
- `$sort` on large datasets
- `$bucket` with many boundaries
- `$facet` with multiple pipelines

Performance Monitoring

```javascript
// Explain aggregation pipeline
db.orders.explain("executionStats").aggregate([
  { $match: { status: "shipped" } },
```

```
  { $group: { _id: "$customerId", total: { $sum: "$amount" } } }
])

// Look for:
// - "totalDocsExamined" vs "nReturned"
// - "executionTimeMillis"
// - "stage" names and their costs
// - "inputStage" information
```

Common Performance Issues and Solutions

Issue	Cause	Solution
Slow $lookup	No index on foreignField	Create index on foreignField
Memory error in $group	Too many groups	Filter first, or allow disk use
Slow $sort	Sorting large dataset	Index sort fields, sort after $mat
Pipeline huge after $unwind	Large arrays	Filter before $unwind
Slow $facet	Processing large input	$match before $facet

---

7. Relationships & Joins

The Fundamental Theory of Relationships in MongoDB

The Document Model vs Relational Model

Relational Databases:

- Data is normalized across tables
- Relationships are defined by foreign keys
- Joins reconstruct related data at query time
- Consistency is enforced at database level

MongoDB Document Model:

- Data can be embedded or referenced
- Relationships are defined by application logic
- Joins ($lookup) are expensive and optional
- Consistency is managed by application

The Key Insight: In MongoDB, you design relationships based on how data is **accessed**, not how data is **structured**.

Manual References

Theory

Manual references are the simplest form of relationship - storing an ID in one document that refers to another document. The application is responsible for resolving the reference.

How It Works:

```text
User Document:                      Order Document:
{                                  {
  _id: ObjectId("user123"),          _id: ObjectId("order456"),
  name: "John Doe"                    userId: ObjectId("user123"),  ← Manual refer
ence
}                                     amount: 299.99
                                    }
```

Types of Manual References:

1. **One-to-One**: Single ID field
2. **One-to-Many**: Array of IDs in parent
3. **Many-to-One**: Single ID in child
4. **Many-to-Many**: Arrays of IDs in both

Advantages and Disadvantages

Advantages:

- Simple and intuitive
- No database overhead
- Flexible - can reference any document
- No referential integrity constraints

Disadvantages:

- Requires multiple queries (or $lookup)
- No automatic consistency
- Application must manage relationships
- Can lead to N+1 query problem

Example with Theory

javascript

```javascript
// MANUAL REFERENCES - One-to-Many

// User document
{
  _id: ObjectId("user123"),
  name: "John Doe",
  email: "john@example.com"
}

// Order documents (each references user)
{
  _id: ObjectId("order456"),
  userId: ObjectId("user123"),  // Manual reference
  amount: 299.99,
  items: [...]
}

// To get user with orders (two queries):
const user = db.users.findOne({ _id: user123 });
const orders = db.orders.find({ userId: user._id }).toArray();

// Or with aggregation (one query but slower):
db.users.aggregate([
  { $match: { _id: user123 } },
  {
    $lookup: {
      from: "orders",
      localField: "_id",
      foreignField: "userId",
      as: "orders"
    }
  }
```

```
])

// MANUAL REFERENCES - Many-to-Many

// Student document
{
  _id: ObjectId("student123"),
  name: "John Doe",
  courseIds: [  // Array of references
    ObjectId("course456"),
    ObjectId("course789")
  ]
}

// Course document
{
  _id: ObjectId("course456"),
  title: "MongoDB Data Modeling",
  studentIds: [  // Also array of references
    ObjectId("student123"),
    ObjectId("student456")
  ]
}

// Application must maintain both arrays on enrollment!
```

$lookup - The Join Operation

Theory

`$lookup` is MongoDB's left outer join operation, available in the aggregation framework. It performs a join between documents in two collections.

The Join Algorithm:

```text
For each document in input collection:
    value = document[localField]
    Find documents in foreign collection where foreignField = value
    Add found documents as array to output document
```

Performance Characteristics:

1. **Nested Loop Join**: For N input documents, performs N queries on foreign collection
2. **Index Dependency**: Requires index on foreignField for acceptable performance
3. **Memory Usage**: Results stored in memory before output
4. **No Query Optimization**: Can't use indexes on localField

Types of $lookup

1. Basic $lookup (equality match):

```javascript
{
  $lookup: {
    from: "orders",
    localField: "_id",
    foreignField: "userId",
    as: "userOrders"
  }
}
```

2. Pipeline $lookup (complex conditions):

```javascript
{
  $lookup: {
    from: "orders",
    let: { userId: "$_id", minAmount: 100 },
    pipeline: [
      {
        $match: {
          $expr: {
            $and: [
              { $eq: ["$userId", "$$userId"] },
              { $gte: ["$amount", "$$minAmount"] }
            ]
          }
        }
      },
      { $sort: { amount: -1 } },
      { $limit: 5 }
    ],
    as: "recentOrders"
  }
}
```

Example with Theory

```javascript
// E-commerce database with $lookup

// Customers collection
{
  _id: ObjectId("cust123"),
  name: "John Doe",
  email: "john@example.com",
  tier: "gold"
}

// Orders collection
{
```

```
  _id: ObjectId("order456"),
  customerId: ObjectId("cust123"),
  amount: 299.99,
  date: ISODate("2024-01-15"),
  status: "shipped"
}

// Reviews collection
{
  _id: ObjectId("review789"),
  customerId: ObjectId("cust123"),
  productId: ObjectId("prod123"),
  rating: 5,
  comment: "Great product!"
}

// Complex report with multiple $lookup
db.customers.aggregate([
  { $match: { tier: "gold" } },
  {
    $lookup: {
      from: "orders",
      localField: "_id",
      foreignField: "customerId",
      as: "orders"
    }
  },
  {
    $lookup: {
      from: "reviews",
      let: { customerId: "$_id" },
      pipeline: [
        {
          $match: {
            $expr: { $eq: ["$customerId", "$$customerId"] }
          }
        },
        { $group: {
          _id: null,
          avgRating: { $avg: "$rating" },
          totalReviews: { $sum: 1 }
        }}
      ],
      as: "reviewStats"
    }
  },
  {
    $project: {
      name: 1,
      email: 1,
      orderCount: { $size: "$orders" },
      totalSpent: { $sum: "$orders.amount" },
      avgRating: { $arrayElemAt: ["$reviewStats.avgRating", 0] }
    }
  }
])

// How this executes:
// 1. Find gold customers (uses index on tier)
```

```
// 2. For each gold customer:
//    - Query orders index on customerId (fast)
//    - Query reviews index on customerId, group in memory
// 3. Project computed fields
```

Embedding Strategy

Theory

Embedding stores related data directly within the document. This is MongoDB's most powerful feature and the primary reason for its performance advantages.

The Embedding Principle: Store together what's queried together.

When to Embed - Deep Analysis

Embed when:

1. **Data is always accessed together**
   - User profile with addresses
   - Blog post with comments
   - Product with specifications
2. **Relationship is one-to-few**
   - User has 1-3 addresses
   - Post has 5-10 tags
   - Order has 1-10 items
3. **Data has no independent lifecycle**
   - Address doesn't exist without user
   - Comment doesn't exist without post
   - Order item doesn't exist without order
4. **Data is read much more than written**
   - Product details (read often, rarely change)
   - User preferences (read on every page load)
   - Configuration settings
5. **Atomic operations needed**

- Update user and address together
- Add item and update order total atomically
- Modify post and comments in one operation

When NOT to Embed

Do NOT embed when:

1. **Data grows without bound**

- User's order history (could be thousands)
- Product reviews (could be millions)
- Log entries (unbounded growth)

2. **Data is accessed independently**

- Need to query orders without user info
- Need to find all users in a city
- Need to analyze comments separately

3. **Data is shared across documents**

- Category referenced by many products
- Author referenced by many posts
- Department referenced by many employees

4. **Document size would exceed 16MB**

- Large arrays of embedded documents
- Binary data like images
- Extensive history or logs

Example with Theory

javascript

```javascript
// GOOD EMBEDDING - User with addresses (one-to-few)
{
  _id: ObjectId("user123"),
  name: "John Doe",
  email: "john@example.com",
  addresses: [
    {
      type: "home",
      street: "123 Main St",
      city: "Boston",
      zip: "02101",
```

```
      isPrimary: true
    },
    {
      type: "work",
      street: "456 Market St",
      city: "Cambridge",
      zip: "02142",
      isPrimary: false
    }
  ]
}
// Benefits:
// - One query gets user with all addresses
// - Addresses never needed without user
// - Limited to few addresses
// - Can update atomically

// GOOD EMBEDDING - Order with items (bounded growth)
{
  _id: ObjectId("order123"),
  customerId: ObjectId("user123"),
  orderDate: ISODate("2024-01-15"),
  status: "shipped",
  items: [
    {
      productId: ObjectId("prod456"),
      name: "MongoDB Guide",
      price: 49.99,
      quantity: 2,
      subtotal: 99.98
    },
    {
      productId: ObjectId("prod789"),
      name: "Database Indexing Explained",
      price: 39.99,
      quantity: 1,
      subtotal: 39.99
    }
  ],
  total: 139.97,
  shipping: {
    address: "123 Main St",
    method: "UPS Ground",
    trackingNumber: "1Z999AA10123456784"
  }
}
// Benefits:
// - Complete order in one document
// - Items rarely exceed reasonable count
// - Order history queries are single document reads

// BAD EMBEDDING - Product with all reviews (unbounded)
{
  _id: ObjectId("product123"),
  name: "MongoDB Guide",
  price: 49.99,
  reviews: [  // Could grow to thousands/millions!
    { userId: "user1", rating: 5, comment: "Excellent!" },
    { userId: "user2", rating: 4, comment: "Very good" },
```

```
    // ... thousands more
  ]
}
// Problems:
// - Document will exceed 16MB
// - Adding review rewrites entire document
// - Can't query reviews efficiently
// - Can't paginate reviews

// SOLUTION: Reference instead
// products collection
{
  _id: ObjectId("product123"),
  name: "MongoDB Guide",
  price: 49.99,
  reviewStats: {
    count: 1543,
    avgRating: 4.5
  }
}

// reviews collection
{
  _id: ObjectId("review1"),
  productId: ObjectId("product123"),
  userId: ObjectId("user1"),
  rating: 5,
  comment: "Excellent!",
  createdAt: ISODate("2024-01-15")
}
// Benefits:
// - Product document stays small
// - Reviews can grow arbitrarily
// - Can query/paginate reviews
// - Can index review fields
```

Trade-offs vs SQL Joins

Theoretical Comparison

Aspect	SQL Joins	MongoDB $lookup
Execution	Optimized by query planner	Nested loop per document
Performance	Can be very fast	O(N) queries on foreign collection
Indexes	Can use indexes on all tables	Only foreignField can use index

Aspect	SQL Joins	MongoDB $lookup
Flexibility	Fixed join types	Pipeline allows complex conditions
Data Locality	Data scattered across tables	Data can be embedded for locality
Consistency	ACID transactions available	Multi-document transactions available but hea
Scalability	Vertical scaling limited	Horizontal scaling easier

The Performance Reality

SQL Join Performance:

- Database optimizes join order
- Can use multiple indexes
- May use hash joins, merge joins
- Optimizer chooses best algorithm
- All data typically on one server

MongoDB $lookup Performance:

- Always nested loop join
- For N documents, N queries on foreign collection
- Each query can use index (O(log n))
- Total complexity: O(N log M) where M is foreign collection size
- Can be distributed across shards (complex)

Decision Framework

```text
Do you need relational capabilities?
├─ NO → Use embedding (best performance)
└─ YES → Is the relationship cardinality high?
    ├─ NO → Embed (still)
    └─ YES → Do you need SQL-like joins?
```

```
        ├─ NO → Use manual references (simpler)
        └─ YES → Use $lookup, but be aware of performance
```

Example: Same Query in Both Paradigms

javascript

```javascript
// SQL: Get customers with their recent orders
SELECT c.*, o.*
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.order_date > '2024-01-01'
ORDER BY o.order_date DESC
LIMIT 100;

// MongoDB: Three approaches

// APPROACH 1: Embed orders (if one-to-few)
// No join needed - orders already in customer document

// APPROACH 2: Manual references + application join
const customers = db.customers.find({
  "orders.orderDate": { $gt: new Date("2024-01-01") }
}).limit(100).toArray();

const customerIds = customers.map(c => c._id);
const orders = db.orders.find({
  customerId: { $in: customerIds },
  orderDate: { $gt: new Date("2024-01-01") }
}).sort({ orderDate: -1 }).toArray();

// Then join in application code

// APPROACH 3: $lookup (closest to SQL)
db.customers.aggregate([
  {
    $lookup: {
      from: "orders",
      let: { customerId: "$_id" },
      pipeline: [
        {
          $match: {
            $expr: { $eq: ["$customerId", "$$customerId"] },
            orderDate: { $gt: new Date("2024-01-01") }
          }
        },
        { $sort: { orderDate: -1 } },
        { $limit: 10 }
      ],
      as: "recentOrders"
    }
  },
  { $match: { "recentOrders.0": { $exists: true } } }, // Has recent orders
  { $limit: 100 }
])
```

Summary: The Complete Mental Model

Aggregation Framework

Think of aggregation as a **data factory assembly line**:

- Raw materials (documents) enter
- Each station (stage) transforms them
- Quality checks ($match) early to reject bad parts
- Assembly ($group) combines parts into products
- Packaging ($project) prepares final output
- Multiple assembly lines ($facet) run in parallel

Relationships in MongoDB

Think in terms of **data gravity**:

- Data that orbits together should be stored together (embedding)
- Data that has independent orbits can reference each other
- Joins ($lookup) are like space-docks - expensive but possible
- Manual references are like navigation coordinates - you know where to go but must travel there yourself

The Golden Rules

1. **Design for queries, not for relationships**
2. **Embed unless there's a reason not to**
3. **References are for data with independent lifecycles**
4. **$lookup is powerful but expensive - use sparingly**
5. **Index foreign fields for any $lookup**
6. **Always $match before expensive operations**
7. **Monitor memory usage in aggregations**
8. **Consider data growth - today's "few" may be tomorrow's "many"**

MongoDB Transactions & Schema Validation: Complete Theory Notes for Senior Developers

---

8. Transactions (Modern MongoDB)

The Fundamental Theory of Transactions

What Are Transactions?

A transaction is a sequence of database operations that must be executed as a single, atomic unit of work. Either **all operations succeed** and are committed, or **none of them take effect** and everything is rolled back.

The Classic Example:

javascript

```
// Bank transfer - must be atomic
1. Debit $100 from Account A
2. Credit $100 to Account B
// If step 1 succeeds but step 2 fails, money disappears!
```

The ACID Properties

ACID is the gold standard for transaction reliability. Understanding each property is crucial for knowing when and how to use transactions.

A - Atomicity (All or Nothing)

- The transaction is an indivisible unit
- Either all operations are applied, or none are
- Partial updates are impossible
- If any operation fails, the entire transaction is rolled back

C - Consistency (Data Integrity)

- Transactions bring the database from one valid state to another
- All defined rules (constraints, validations) are maintained
- No transaction can violate database integrity
- Application-level invariants are preserved

I - Isolation (Concurrency Control)

- Concurrent transactions don't interfere with each other
- Each transaction executes as if it were alone
- Intermediate states are not visible to other transactions
- MongoDB uses snapshot isolation by default

D - Durability (Persistence)

- Once committed, changes survive permanently
- Even after system failure or restart
- Data is written to journal before acknowledging commit
- Ensured by write concern and journaling

MongoDB's Transaction Evolution

text

```
Pre-4.0: Only single-document ACID transactions
4.0:     Multi-document ACID transactions (replica sets)
4.2:     Multi-document ACID transactions (sharded clusters)
4.4+:    Enhanced performance and features
```

Multi-document Transactions - Deep Theory

How Multi-document Transactions Work

A multi-document transaction in MongoDB allows multiple operations across multiple collections and databases to be executed atomically.

The Transaction Lifecycle:

```text
Session Start → Transaction Start → Operations → Commit/Rollback → Session End
      ↓                ↓                  ↓              ↓                ↓
  Creates        Initializes         All writes     Makes all       Cleans up
  session        transaction         are staged     changes         resources
                                                          permanent
```

Internal Mechanics:

1. **Snapshot Isolation**: When a transaction starts, MongoDB takes a snapshot of the data at that point in time. All reads within the transaction see this consistent snapshot.
2. **Write Staging**: All writes within the transaction are held in memory and not visible outside the transaction.
3. **Conflict Detection**: MongoDB tracks read/write conflicts. If two transactions modify the same document, one will fail.
4. **Two-Phase Commit**: On commit, MongoDB ensures all nodes have the writes, then commits atomically.

Example with Theory

```javascript
// Complex transaction across multiple collections
const session = db.getMongo().startSession();
session.startTransaction({
  readConcern: { level: "snapshot" },
  writeConcern: { w: "majority" },
  readPreference: "primary"
});

try {
  const accounts = session.getDatabase("bank").accounts;
  const transactions = session.getDatabase("bank").transactions;
  const audit = session.getDatabase("logs").audit;

  // Step 1: Debit from source account
  const debitResult = accounts.updateOne(
    { accountId: "A123", balance: { $gte: 1000 } },
    { $inc: { balance: -1000 } }
  );

  if (debitResult.modifiedCount !== 1) {
    throw new Error("Insufficient funds or account not found");
  }

  // Step 2: Credit to destination account
  const creditResult = accounts.updateOne(
```

```
    { accountId: "B456" },
    { $inc: { balance: 1000 } }
  );

  if (creditResult.modifiedCount !== 1) {
    throw new Error("Destination account not found");
  }

  // Step 3: Record transaction
  transactions.insertOne({
    from: "A123",
    to: "B456",
    amount: 1000,
    timestamp: new Date(),
    type: "transfer"
  });

  // Step 4: Audit log (different database)
  audit.insertOne({
    event: "transfer",
    accounts: { from: "A123", to: "B456" },
    amount: 1000,
    timestamp: new Date()
  });

  // All operations succeeded, commit
  await session.commitTransaction();
  print("Transfer completed successfully");

} catch (error) {
  // Any operation failed, rollback everything
  await session.abortTransaction();
  print("Transfer failed:", error);
} finally {
  session.endSession();
}

// What happens internally:
// 1. Snapshot taken at transaction start
// 2. All reads see consistent data
// 3. Writes held in transaction buffer
// 4. On commit, all writes applied atomically
// 5. On abort, buffer discarded
```

Session Handling - Deep Theory

What is a Session?

A session is a logical container for a sequence of operations that share the same context. Sessions enable:

- Causal consistency across operations
- Transaction grouping

- Retryable writes
- Operation ordering

Session States

```text
                 ┌──────────────┐
                 │   Started    │
                 └──────┬───────┘
                        │
                 ┌──────▼───────┐
         ┌───────┤ Transaction │
         │       │   Active     │
         │       └──────┬───────┘
         │              │
    ┌────▼────┐   ┌────▼────┐
    │ Commit  │   │ Abort   │
    │ Pending │   │ Pending │
    └────┬────┘   └────┬────┘
         │              │
    ┌────▼────┐   ┌────▼────┐
    │Committed│   │ Aborted │
    └────┬────┘   └────┬────┘
         │              │
         └──────┬───────┘
                ▼
           ┌──────────────┐
           │Session Ended│
           └──────────────┘
```

Session Operations

```javascript
// 1. Starting a session
const session = db.getMongo().startSession({
  causalConsistency: true,  // Default: ensures causal consistency
  retryWrites: true         // Enable retryable writes
});

// 2. Session with options
const session = client.startSession({
  defaultTransactionOptions: {
    readConcern: { level: "majority" },
    writeConcern: { w: "majority" },
    readPreference: "primary",
```

```
    maxCommitTimeMS: 10000  // Timeout after 10 seconds
  }
});

// 3. Using session for operations (without transaction)
const collection = session.getDatabase("test").users;
collection.insertOne({ name: "John" });  // Part of session context
collection.updateOne({ name: "John" }, { $set: { age: 30 } });

// 4. Session with causal consistency
session.advanceOperationTime(session.getOperationTime());
// All subsequent operations will see previous writes

// 5. Session pooling and reuse
async function processBatch(items, session) {
  for (const item of items) {
    await processItem(item, session);  // Reuse same session
  }
}

// 6. Cleaning up
session.endSession();  // Always call this!
```

Session Management Best Practices

DO:

- Always end sessions (use try/finally)
- Reuse sessions for related operations
- Set appropriate timeouts
- Monitor active sessions

DON'T:

- Leave sessions open indefinitely
- Share sessions across threads
- Mix unrelated operations in same session

When to Use Transactions (and When NOT)

The Transaction Decision Framework

```text
```

```
Do you need atomicity across multiple documents?
├─ NO → Use single-document operations (always first choice)
└─ YES → Can you model with embedded documents?
    ├─ YES → Use single-document operations (better)
    └─ NO → Do you need ACID across collections?
        ├─ YES → Use transactions (necessary)
        └─ NO → Consider eventual consistency
```

When to Use Transactions - The Theory

Use transactions when:

1. **Financial Operations**

```javascript
// Money transfers MUST be atomic
session.startTransaction();
try {
    db.accounts.updateOne({ _id: fromId }, { $inc: { balance: -amount } });
    db.accounts.updateOne({ _id: toId }, { $inc: { balance: amount } });
    db.transactions.insertOne({ from: fromId, to: toId, amount });
    session.commitTransaction();
} catch {
    session.abortTransaction();
}
```

2. **Inventory Management**

```javascript
// Order fulfillment must be consistent
session.startTransaction();
try {
    // Check and reserve inventory
    const product = db.products.findOne({ _id: productId });
    if (product.stock < quantity) throw new Error("Out of stock");

    db.products.updateOne(
        { _id: productId },
        { $inc: { stock: -quantity, reserved: quantity } }
    );

    db.orders.insertOne(orderData);
    session.commitTransaction();
} catch {
    session.abortTransaction();
}
```

3. **User Registration with Dependencies**

```javascript
```

```
// User creation across multiple collections
session.startTransaction();
try {
    db.users.insertOne(userData);
    db.profiles.insertOne(profileData);
    db.settings.insertOne(userSettings);
    db.auditLog.insertOne({ action: "user_created", userId });
    session.commitTransaction();
} catch {
    session.abortTransaction();
}
```

When NOT to Use Transactions - The Theory

Avoid transactions when:

1. **Single Document Operations Suffice**

```
javascript

// WRONG: Using transaction for single document
session.startTransaction();
db.users.updateOne({ _id }, { $set: { lastLogin: new Date() } });
session.commitTransaction();

// RIGHT: Direct update
db.users.updateOne({ _id }, { $set: { lastLogin: new Date() } });
```

2. **High-Volume Operations**

```
javascript

// WRONG: Transactions in high-throughput logging
session.startTransaction();
db.logs.insertMany(batch);
session.commitTransaction();  // Massive performance hit

// RIGHT: Batch insert without transaction
db.logs.insertMany(batch, { ordered: false });
```

3. **Operations That Don't Need Atomicity**

```
javascript

// WRONG: Using transaction for non-critical updates
session.startTransaction();
db.analytics.updateOne({ _id }, { $inc: { pageViews: 1 } });
db.userStats.updateOne({ userId }, { $inc: { visits: 1 } });
session.commitTransaction();

// RIGHT: Accept eventual consistency
db.analytics.updateOne({ _id }, { $inc: { pageViews: 1 } });
db.userStats.updateOne({ userId }, { $inc: { visits: 1 } });
// Small chance of inconsistency, but 100x faster
```

4. **Long-Running Operations**

```javascript
// WRONG: Transaction processing 1M records
session.startTransaction();
for (const doc of largeDataset) {  // Takes hours!
    process(doc);
}
session.commitTransaction();  // Will timeout or conflict

// RIGHT: Process in batches with checkpoints
let processed = 0;
while (processed < total) {
    const batch = getNextBatch(1000);
    processBatch(batch);  // Each batch can be retried
    updateCheckpoint(processed);
    processed += 1000;
}
```

Performance Impact Analysis

Transaction Overhead:

Operation Type	Without Transaction	With Transaction	Slowdown
Single doc update	1ms	2ms	2x
10 doc updates	10ms	30ms	3x
100 doc updates	100ms	500ms	5x
Complex multi-collection	N/A	1000ms	-

Why Transactions Are Slower:

1. **Snapshot creation**: Must capture point-in-time view
2. **Lock tracking**: Must track all read/write locks
3. **Conflict detection**: Must detect concurrent modifications
4. **Two-phase commit**: Must coordinate across replicas

5. **Journal synchronization**: Must ensure durability

---

9. Schema Validation

The Fundamental Theory of Schema Validation

What is Schema Validation?

Schema validation allows MongoDB to enforce rules about document structure at the database level. It brings some of the rigor of SQL schemas while maintaining MongoDB's flexibility.

The Paradigm Shift:

```text
Traditional MongoDB:           Schema-Validated MongoDB:
Any document goes in           Only valid documents accepted
Validation in application       Validation in database
Runtime errors only            Insert-time rejection
Schema evolves organically     Controlled schema evolution
```

Why Schema Validation Exists

1. **Data Quality**: Prevent invalid data from entering
2. **Application Safety**: Assume consistent structure
3. **Documentation**: Schema serves as living documentation
4. **Migration Safety**: Ensure old data meets new requirements
5. **API Contracts**: Guarantee response structure

Validation Levels

MongoDB provides three validation levels:

Level	Behavior	Use Case
strict (default)	Validate all inserts and updates	Production systems
moderate	Validate existing documents only on update, new docs always	Gradual enforcement
off	No validation	Development, migrations

Validation Actions

Action	Behavior	Use Case
error (default)	Reject invalid documents	Strict enforcement
warn	Log warning but accept	Testing, gradual adoption

JSON Schema Validation - Deep Theory

What is JSON Schema?

JSON Schema is a declarative language for validating the structure of JSON documents. MongoDB implements a subset of JSON Schema (draft-04).

The Schema as a Contract:

javascript

```javascript
// Schema defines what's allowed
{
  $jsonSchema: {
    bsonType: "object",
    required: ["name", "email"],
    properties: {
      name: { bsonType: "string" },
      email: { bsonType: "string", pattern: "^.+@.+$" }
    }
  }
```

```
}

// Document must satisfy this contract
{ name: "John", email: "john@example.com" }  // ✅ Valid
{ name: "John" }  // ❌ Invalid (missing email)
{ name: 123, email: "john@example.com" }  // ❌ Invalid (name wrong type)
```

Schema Validation Structure

javascript

```
{
  $jsonSchema: {
    bsonType: "object",                    // Document must be object
    required: ["field1", "field2"],          // Required fields
    properties: {                            // Field definitions
      field1: {
        bsonType: "string",
        description: "Must be a string"
      },
      field2: {
        bsonType: "int",
        minimum: 0,
        maximum: 100
      }
    },
    additionalProperties: false,              // No extra fields allowed
    patternProperties: {                       // Dynamic field names
      "^meta_": { bsonType: "string" }
    },
    dependencies: {                           // Conditional requirements
      "creditCard": ["billingAddress"]
    }
  }
}
```

Enforcing Structure in Collections

Creating Validation Rules

javascript

```
// 1. Create collection with validation
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["email", "createdAt"],
      properties: {
        _id: { bsonType: "objectId" },
        email: {
          bsonType: "string",
          pattern: "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$",
          description: "Must be a valid email format"
        },
        name: {
```

```
          bsonType: "string",
          minLength: 2,
          maxLength: 100,
          description: "Must be between 2 and 100 characters"
        },
        age: {
          bsonType: "int",
          minimum: 0,
          maximum: 150,
          description: "Must be a reasonable age"
        },
        status: {
          enum: ["active", "inactive", "pending"],
          description: "Must be valid status"
        },
        createdAt: {
          bsonType: "date",
          description: "Must be a date"
        },
        address: {
          bsonType: "object",
          required: ["city", "country"],
          properties: {
            street: { bsonType: "string" },
            city: { bsonType: "string" },
            country: { bsonType: "string" },
            zipCode: {
              bsonType: "string",
              pattern: "^[0-9]{5}$"
            }
          }
        },
        tags: {
          bsonType: "array",
          minItems: 0,
          maxItems: 20,
          uniqueItems: true,
          items: {
            bsonType: "string",
            enum: ["premium", "verified", "beta"]
          }
        }
      },
      additionalProperties: false  // No extra fields allowed
    }
  },
  validationLevel: "strict",
  validationAction: "error"
});

// 2. Add validation to existing collection
db.runCommand({
  collMod: "users",
  validator: {
    $jsonSchema: { /* schema */ }
  },
  validationLevel: "moderate"  // Start with moderate
});
```

```
// 3. Multiple validators with $and/$or
db.createCollection("products", {
  validator: {
    $and: [
      {
        $jsonSchema: {
          required: ["name", "price"],
          properties: {
            name: { bsonType: "string" },
            price: { bsonType: "decimal", minimum: 0 }
          }
        }
      },
      {
        $expr: { $lt: ["$price", "$msrp"] }  // Custom condition
      }
    ]
  }
});
```

Testing Validation

javascript

```
// Test valid document - should work
db.users.insertOne({
  email: "john@example.com",
  name: "John Doe",
  age: 30,
  status: "active",
  createdAt: new Date(),
  address: {
    city: "Boston",
    country: "USA",
    zipCode: "02101"
  },
  tags: ["premium"]
});

// Test invalid document - should fail
db.users.insertOne({
  email: "not-an-email",  // Invalid email
  name: "J",               // Too short
  age: 200,                // Too high
  status: "superuser",     // Not in enum
  createdAt: "yesterday",  // Not a date
  // Missing address.city
  extraField: "not allowed"  // additionalProperties: false
});
// WriteError: Document failed validation
```

Required Fields, Types, and Constraints - Deep Theory

Field Requirements

required specifies fields that must exist in every document:

```javascript
{
  $jsonSchema: {
    required: ["name", "email"],  // Must have these fields
    properties: {
      name: { bsonType: "string" },
      email: { bsonType: "string" },
      phone: { bsonType: "string" }  // Optional
    }
  }
}

// Valid:
{ name: "John", email: "john@example.com" }
{ name: "John", email: "john@example.com", phone: "555-1234" }

// Invalid:
{ name: "John" }  // Missing email
{ email: "john@example.com" }  // Missing name
```

Data Types (bsonType)

MongoDB supports all BSON types in validation:

Type	Description	Example
object	Nested document	`{ bsonType: "object" }`
array	Array of values	`{ bsonType: "array" }`
string	Text string	`{ bsonType: "string" }`
int	32-bit integer	`{ bsonType: "int" }`
long	64-bit integer	`{ bsonType: "long" }`
double	Floating point	`{ bsonType: "double" }`
decimal	High-precision decimal	`{ bsonType: "decimal" }`

Type	Description	Example
bool	Boolean	`{ bsonType: "bool" }`
date	ISODate	`{ bsonType: "date" }`
null	Null value	`{ bsonType: "null" }`
objectId	MongoDB ObjectId	`{ bsonType: "objectId" }`
binData	Binary data	`{ bsonType: "binData" }`
regex	Regular expression	`{ bsonType: "regex" }`

```javascript
// Multiple allowed types
{
  bsonType: ["string", "null"]  // Can be string or null
}

// Specific numeric types
{
  price: { bsonType: "decimal" },  // Use decimal for money
  count: { bsonType: "int" },       // Use int for whole numbers
  rating: { bsonType: "double" }     // Use double for averages
}
```

String Constraints

```javascript
{
  username: {
    bsonType: "string",
    minLength: 3,
    maxLength: 20,
    pattern: "^[a-zA-Z0-9_]+$",  // Alphanumeric + underscore
    description: "Username must be 3-20 alphanumeric characters"
  },
  email: {
    bsonType: "string",
    pattern: "^[^@]+@[^@]+\\.[^@]+$",  // Basic email validation
    maxLength: 254
  },
  bio: {
    bsonType: "string",
    maxLength: 500
```

```
  }
}
```

Numeric Constraints

```javascript
{
  age: {
    bsonType: "int",
    minimum: 0,
    maximum: 150,
    exclusiveMinimum: false,  // >= 0
    exclusiveMaximum: true     // < 150 (not <= 150)
  },
  score: {
    bsonType: "double",
    minimum: 0.0,
    maximum: 100.0,
    multipleOf: 0.5  // Must be multiple of 0.5
  },
  price: {
    bsonType: "decimal",
    minimum: 0.00
  }
}
```

Array Constraints

```javascript
{
  tags: {
    bsonType: "array",
    minItems: 0,
    maxItems: 10,
    uniqueItems: true,  // No duplicates
    items: {
      bsonType: "string",
      enum: ["tech", "science", "art"]
    }
  },
  scores: {
    bsonType: "array",
    items: {
      bsonType: "double",
      minimum: 0,
      maximum: 100
    },
    additionalItems: false  // No extra items beyond schema
  },
  matrix: {
    bsonType: "array",
    items: {
      bsonType: "array",
      items: { bsonType: "int" }  // 2D array of ints
    }
  }
```

```
}
```

Object (Nested Document) Constraints

javascript

```javascript
{
  address: {
    bsonType: "object",
    required: ["city", "country"],
    properties: {
      street: { bsonType: "string" },
      city: { bsonType: "string" },
      state: {
        bsonType: "string",
        minLength: 2,
        maxLength: 2
      },
      zipCode: {
        bsonType: "string",
        pattern: "^[0-9]{5}(-[0-9]{4})?$"
      },
      country: {
        bsonType: "string",
        enum: ["USA", "Canada", "Mexico"]
      }
    },
    additionalProperties: false  // No extra address fields
  }
}
```

Enum Constraints

javascript

```javascript
{
  status: {
    enum: ["draft", "published", "archived"],
    description: "Must be a valid document status"
  },
  role: {
    enum: ["admin", "editor", "viewer"],
    bsonType: "string"
  },
  priority: {
    enum: [1, 2, 3, 4, 5],  // Can mix types in enum
    bsonType: "int"
  }
}
```

Conditional Validation (Dependencies)

javascript

```javascript
{
  $jsonSchema: {
    properties: {
      paymentMethod: {
```

```
        enum: ["credit", "paypal", "invoice"]
      },
      creditCard: { bsonType: "object" },
      paypalEmail: { bsonType: "string" },
      billingAddress: { bsonType: "object" }
    },
    dependencies: {
      creditCard: ["billingAddress"],  // If creditCard exists, billingAddress req
uired
      paypalEmail: ["billingAddress"]   // If paypalEmail exists, billingAddress r
equired
    }
  }
}
```

Complex Validation with $expr

For validation beyond JSON Schema, use `$expr`:

javascript

```
db.createCollection("orders", {
  validator: {
    $and: [
      {
        $jsonSchema: {
          required: ["items", "subtotal", "tax", "total"],
          properties: {
            items: { bsonType: "array", minItems: 1 },
            subtotal: { bsonType: "decimal", minimum: 0 },
            tax: { bsonType: "decimal", minimum: 0 },
            total: { bsonType: "decimal", minimum: 0 },
            discount: { bsonType: "decimal", minimum: 0 }
          }
        }
      },
      {
        $expr: {
          $and: [
            { $eq: ["$total", { $add: ["$subtotal", "$tax"] }] },
            { $lte: ["$discount", "$subtotal"] }
          ]
        }
      }
    ]
  }
});
```

Advanced Schema Validation Patterns

Pattern 1: Versioned Schemas

javascript

```
// Support multiple schema versions
```

```
{
  $jsonSchema: {
    required: ["schemaVersion"],
    properties: {
      schemaVersion: {
        bsonType: "int",
        enum: [1, 2]  // Support versions 1 and 2
      },
      // Version 1 fields
      name: {
        bsonType: "string",
        // Only required if version 1
      },
      // Version 2 fields
      firstName: {
        bsonType: "string",
        // Only required if version 2
      },
      lastName: {
        bsonType: "string",
        // Only required if version 2
      }
    },
    // Conditional requirements based on version
    allOf: [
      {
        if: {
          properties: { schemaVersion: { const: 1 } }
        },
        then: {
          required: ["name"]
        }
      },
      {
        if: {
          properties: { schemaVersion: { const: 2 } }
        },
        then: {
          required: ["firstName", "lastName"]
        }
      }
    ]
  }
}
```

Pattern 2: Polymorphic Schemas

javascript

```
// Different document types in same collection
{
  $jsonSchema: {
    required: ["type"],
    properties: {
      type: {
        enum: ["user", "product", "order"]
      }
    },
    allOf: [
```

```
      {
        if: {
          properties: { type: { const: "user" } }
        },
        then: {
          required: ["email", "name"],
          properties: {
            email: { bsonType: "string" },
            name: { bsonType: "string" }
          }
        }
      },
      {
        if: {
          properties: { type: { const: "product" } }
        },
        then: {
          required: ["sku", "price"],
          properties: {
            sku: { bsonType: "string" },
            price: { bsonType: "decimal" }
          }
        }
      }
    ]
  }
}
```

Pattern 3: Audit/History Tracking

javascript

```
{
  $jsonSchema: {
    required: ["data", "metadata"],
    properties: {
      data: { bsonType: "object" },  // Any structure allowed
      metadata: {
        bsonType: "object",
        required: ["createdAt", "createdBy", "version"],
        properties: {
          createdAt: { bsonType: "date" },
          createdBy: { bsonType: "objectId" },
          updatedAt: { bsonType: "date" },
          updatedBy: { bsonType: "objectId" },
          version: { bsonType: "int", minimum: 1 },
          tags: {
            bsonType: "array",
            items: { bsonType: "string" }
          }
        }
      }
    }
  }
}
```

Schema Validation Best Practices

DO's and DON'Ts

DO:

1. **Start with moderate validation** when adding to existing collections
2. **Use meaningful descriptions** for error messages
3. **Test validation rules** thoroughly before production
4. **Document your schema** alongside validation rules
5. **Version your schemas** to handle evolution
6. **Consider performance** - complex validation adds overhead
7. **Monitor validation errors** to catch data quality issues

DON'T:

1. **Don't make schemas too restrictive** early on
2. **Don't use validation as primary error handling** - apps should still validate
3. **Don't forget about indexes** when adding required fields
4. **Don't validate fields that change frequently** without need
5. **Don't create circular dependencies** in validation

Migration Strategies

javascript

```javascript
// Phase 1: No validation
// Collection has mixed data

// Phase 2: Warn only
db.runCommand({
  collMod: "users",
  validator: { $jsonSchema: { ... } },
  validationLevel: "moderate",
  validationAction: "warn"
});

// Phase 3: Enforce on new docs, warn on updates
db.runCommand({
  collMod: "users",
  validationLevel: "moderate",
  validationAction: "error"
});

// Phase 4: Full enforcement
```

```javascript
db.runCommand({
  collMod: "users",
  validationLevel: "strict",
  validationAction: "error"
});

// Phase 5: Clean up old data
db.users.find({
  $nor: [
    { email: { $exists: true } },
    { name: { $exists: true } }
  ]
}).forEach(function(doc) {
  // Migrate or archive invalid documents
  db.users_archive.insertOne(doc);
  db.users.deleteOne({ _id: doc._id });
});
```

Monitoring Validation

```javascript
// Check validation rules
db.getCollectionInfos({ name: "users" });

// Find documents that would fail validation
db.users.find({
  $nor: [
    { email: { $exists: true, $type: "string" } },
    { name: { $exists: true, $type: "string" } }
  ]
}).count();

// Monitor validation errors in logs
// Look for: "Document failed validation" in mongod.log

// Get validation error statistics
db.serverStatus().metrics.documentValidation;
```

Summary: The Complete Mental Model

Transactions

Think of transactions as **bank vaults**:

- Single-document operations are like individual safe deposit boxes (always atomic)
- Multi-document transactions are like moving money between boxes (needs coordination)
- Sessions are like your visit to the bank (temporary, track your activities)
- Use only when you must move money between boxes, not for everyday banking

Schema Validation

Think of schema validation as **building codes**:

- Define what's allowed before construction
- Prevent unsafe structures
- Allow flexibility within bounds
- Enforce quality without dictating everything
- Can be updated as building practices evolve

The Golden Rules

For Transactions:

1. **Single-document operations first** - they're always atomic
2. **Transactions are tools, not defaults** - use sparingly
3. **Keep transactions short** - minimize lock time
4. **Handle transaction failures gracefully** - always have retry logic
5. **Monitor transaction latency** - it's your canary for problems

For Schema Validation:

1. **Validate at the database level** - it's your last line of defense
2. **Start permissive, become strict** - gradual adoption wins
3. **Document your schema** - validation rules are living documentation
4. **Test validation rules** - they're code too
5. **Plan for evolution** - schemas will change

MongoDB Performance Optimization & Replication: Complete Theory Notes for Senior Developers

10. Performance Optimization

The Fundamental Theory of Performance

What is Performance in MongoDB?

Performance is the measure of how efficiently MongoDB handles operations under various conditions. It's not just about speed—it's about **predictable speed at scale**.

The Performance Equation:

```text
System Performance = (Index Efficiency × Query Optimization) - (Resource Contentio
n + Locking Overhead)

Where:
- Index Efficiency: How well indexes serve queries
- Query Optimization: How efficiently queries are written
- Resource Contention: Competition for CPU, RAM, Disk I/O
- Locking Overhead: Time spent managing concurrent access
```

The Performance Pyramid

```text
            ┌──────────────────────┐
            │   Application        │  ← Query patterns, schema design
            ├──────────────────────┤
            │   Query Execution    │  ← Query plans, covered queries
            ├──────────────────────┤
            │   Indexes            │  ← Index design, selectivity
            ├──────────────────────┤
            │   Storage Engine     │  ← WiredTiger, caching, compression
            ├──────────────────────┤
            │   Hardware           │  ← RAM, Disk I/O, Network, CPU
            └──────────────────────┘
```

Each layer builds on the one below. Optimize from the bottom up.

Query Optimization - Deep Theory

The Query Lifecycle

Understanding how MongoDB processes a query is essential for optimization:

text

```
Client → Parsing → Query Planner → Index Selection → Execution → Return

Stage 1: Parsing
- Validate syntax
- Convert to internal representation

Stage 2: Query Planning
- Generate possible query plans
- Evaluate plan costs
- Select winning plan

Stage 3: Index Selection
- Check available indexes
- Determine index eligibility
- Choose best index(es)

Stage 4: Execution
- Fetch documents from index/collection
- Apply filters, projections
- Sort, group, transform

Stage 5: Return
- Format results
- Send to client
```

The Query Optimizer

MongoDB's query optimizer is a cost-based optimizer that:

1. **Generates multiple plans** for each query shape
2. **Runs them in parallel** for a short trial period
3. **Selects the winner** based on:

- Number of documents examined
- Execution time
- Index usage

4. **Caches the winning plan** for similar queries
5. **Reevaluates** after:

- 1000 writes
- Index changes
- Restart

javascript

```javascript
// Understanding the optimizer
db.users.find({ age: { $gt: 25 }, status: "active" })
     .sort({ name: 1 })
     .explain("executionStats");

// Output shows:
// - Winning plan (what was used)
// - Rejected plans (alternatives considered)
// - Execution stats (how it performed)
```

Query Patterns and Anti-Patterns

Good Query Patterns:

javascript

```javascript
// GOOD: Selective query
db.users.find({ email: "user@example.com" })
// Uses index, returns 0-1 documents

// GOOD: Range query with selective first field
db.orders.find({
  status: "shipped",           // High selectivity
  orderDate: { $gt: ISODate("2024-01-01") }
})

// GOOD: Covered query
db.users.find(
  { email: "user@example.com" },
  { name: 1, status: 1, _id: 0 }
)  // All fields in index
```

Bad Query Patterns (Anti-Patterns):

javascript

```
// BAD: Non-selective query
db.users.find({ status: "active" })
// Returns 80% of collection - almost a table scan

// BAD: Negation query
db.products.find({ price: { $ne: 0 } })
// Must scan all documents to find non-matching

// BAD: Regular expression without prefix
db.articles.find({ title: { $regex: /mongodb/i } })
// Cannot use index efficiently (unless leading anchor)

// BAD: $where with JavaScript
db.users.find({ $where: "this.age > this.loginCount" })
// Cannot use indexes, evaluates JavaScript for each document

// BAD: Or conditions on different fields
db.orders.find({
  $or: [
    { status: "pending" },
    { customerId: "123" }
  ]
})  // May need multiple indexes
```

The N+1 Query Problem

javascript

```
// BAD: N+1 queries
const users = db.users.find().limit(10).toArray();
users.forEach(user => {
  const orders = db.orders.find({ userId: user._id }).toArray();
  // For 10 users, this is 11 queries (1 + N)
});

// GOOD: Single query with $lookup
db.users.aggregate([
  { $limit: 10 },
  {
    $lookup: {
      from: "orders",
      localField: "_id",
      foreignField: "userId",
      as: "orders"
    }
  }
]);

// GOOD: Batch loading
const users = db.users.find().limit(10).toArray();
const userIds = users.map(u => u._id);
const orders = db.orders.find({ userId: { $in: userIds } }).toArray();
// Group orders by userId in application code
```

Index Tuning - Deep Theory

Index Selection Theory

The goal of index tuning is to match indexes to your **most important queries**. Not all queries need indexes.

The 80/20 Rule for Indexes:

- 80% of your queries should be covered by 20% of your indexes
- Index the queries that matter most, not every possible query

Index Cardinality

Cardinality refers to the number of unique values in an indexed field:

Cardinality	Example	Index Effectiveness
High	Email, SSN, ObjectId	Excellent - quickly narrows results
Medium	Status (5-10 values)	Good - reduces scan but not to single document
Low	Boolean (true/false)	Poor - may not be worth indexing alone

The Cardinality Rule: Index fields with high cardinality first in compound indexes.

Index Selectivity

Selectivity measures how many documents match a query condition:

text

```
Selectivity = Matching Documents / Total Documents

High Selectivity (< 1%):  Good for indexing
Medium Selectivity (1-10%):  May benefit from index
Low Selectivity (> 10%):  Index may not help (full scan might be faster)
```

Index Tuning Process

```javascript
// Step 1: Identify slow queries
db.setProfilingLevel(1, { slowms: 100 });
// Check system.profile collection

// Step 2: Analyze query pattern
db.orders.find({
  status: "shipped",
  orderDate: { $gt: ISODate("2024-01-01") }
}).sort({ customerId: 1 }).explain("executionStats");

// Look for:
// - totalDocsExamined (should be close to nReturned)
// - totalKeysExamined (index usage)
// - executionTimeMillis

// Step 3: Design optimal index (ESR rule)
db.orders.createIndex(
  { status: 1,           // Equality first
    customerId: 1,        // Sort next
    orderDate: 1          // Range last
  }
);

// Step 4: Test with hint to force index
db.orders.find({ ... })
  .hint({ status: 1, customerId: 1, orderDate: 1 })
  .explain("executionStats");

// Step 5: Compare before/after
```

Index Intersection

MongoDB can use multiple indexes to satisfy a query through index intersection:

```javascript
// Given two separate indexes:
db.orders.createIndex({ status: 1 });
db.orders.createIndex({ customerId: 1 });

// Query can use both:
db.orders.find({
  status: "shipped",
  customerId: "12345"
});
// May use index intersection:
// 1. Get docs with status="shipped" from first index
// 2. Get docs with customerId="12345" from second index
// 3. Intersect the two sets

// But a compound index is better:
db.orders.createIndex({ status: 1, customerId: 1 });
```

Index Maintenance

```javascript
```

```
// Monitor index usage
db.aggregate([
  { $indexStats: {} }
]);

// Find unused indexes
// Look for indexes with low access counts

// Rebuild indexes (after heavy fragmentation)
db.collection.reIndex();

// Background vs Foreground building
db.collection.createIndex({ field: 1 }, { background: true });
// Background: Slower build, doesn't block writes
// Foreground: Faster build, blocks all operations
```

Avoiding Large Documents - Deep Theory

The 16MB Limit - Why It Exists

The 16MB document size limit is not arbitrary—it's a deliberate design decision:

Technical Reasons:

1. **Network Transmission**: Larger documents tie up network connections longer
2. **Memory Usage**: Documents must fit in working memory for operations
3. **Disk I/O**: Reading/writing large documents is expensive
4. **Journaling**: Large documents create large journal entries
5. **Replication**: Oplog entries limited to near 16MB

The Real Limit Isn't 16MB - It's Much Smaller

In practice, documents should be much smaller than 16MB:

javascript

```
// Document size recommendations:
// Ideal:      < 1MB     - Fastest operations, best for caching
// Good:       1-4MB     - Acceptable for most use cases
// Acceptable: 4-8MB     - May need optimization
// Caution:    8-16MB    - Performance risk, consider redesign
// Danger:     >16MB     - Not allowed
```

Detecting Large Documents

javascript

```javascript
// Find document sizes in a collection
db.collection.aggregate([
  {
    $project: {
      size: { $bsonSize: "$$ROOT" }
    }
  },
  { $sort: { size: -1 } },
  { $limit: 10 }
]);

// Get size statistics
db.collection.aggregate([
  {
    $project: {
      size: { $bsonSize: "$$ROOT" }
    }
  },
  {
    $group: {
      _id: null,
      avgSize: { $avg: "$size" },
      maxSize: { $max: "$size" },
      minSize: { $min: "$size" },
      count: { $sum: 1 }
    }
  }
]);

// Find documents exceeding threshold
db.collection.find({
  $expr: { $gt: [ { $bsonSize: "$$ROOT" }, 10 * 1024 * 1024 ] }
});
```

Strategies to Reduce Document Size

1. Split into Separate Collections (Subset Pattern):

javascript

```javascript
// BEFORE: One large document
{
  _id: "user123",
  profile: { ... },
  settings: { ... },
  activityLog: [ /* 10,000 entries */ ],  // Huge array!
  notifications: [ /* 5,000 entries */ ]
}

// AFTER: Split collections
// users collection (small)
{
  _id: "user123",
  profile: { ... },
  settings: { ... }
}

// user_activity collection (referenced)
```

```
{
  _id: ObjectId(),
  userId: "user123",
  timestamp: ISODate(...),
  action: "login",
  details: { ... }
}

// user_notifications collection (referenced)
{
  _id: ObjectId(),
  userId: "user123",
  message: "...",
  read: false,
  createdAt: ISODate(...)
}
```

2. Use Bucket Pattern for Time-Series:

```javascript
// BEFORE: One document per reading
{ sensor: "temp1", timestamp: ISODate("2024-01-15T10:00:00Z"), value: 22.5 }
{ sensor: "temp1", timestamp: ISODate("2024-01-15T10:01:00Z"), value: 22.6 }
// 1440 documents per day!

// AFTER: Bucket pattern
{
  sensor: "temp1",
  date: ISODate("2024-01-15"),
  readings: [
    { time: "10:00", value: 22.5 },
    { time: "10:01", value: 22.6 },
    // ... 1440 readings in one document
  ],
  metadata: {
    unit: "celsius",
    location: "room-101"
  }
}
// 1 document per day!
```

3. Compress Repeated Data:

```javascript
// BEFORE: Repeated metadata
{
  orders: [
    {
      productId: "prod123",
      productName: "MongoDB Guide",
      productPrice: 49.99,
      quantity: 2
    },
    {
      productId: "prod123",
      productName: "MongoDB Guide",  // Repeated
```

```
      productPrice: 49.99,            // Repeated
      quantity: 1
    }
  ]
}

// AFTER: Reference product data
{
  orders: [
    { productId: "prod123", quantity: 2 },
    { productId: "prod123", quantity: 1 }
  ]
}
// Product details stored separately
```

4. Use GridFS for Large Binaries:

javascript

```
// DON'T: Store files in documents
{
  filename: "video.mp4",
  data: BinData(0, "very large binary data..."),  // Would exceed 16MB!
  metadata: { ... }
}

// DO: Use GridFS
// fs.files collection
{
  _id: ObjectId("file123"),
  filename: "video.mp4",
  length: 104857600,
  chunkSize: 261120,
  uploadDate: ISODate(...),
  metadata: { ... }
}

// fs.chunks collection
{
  _id: ObjectId("chunk1"),
  files_id: ObjectId("file123"),
  n: 0,
  data: BinData(0, "first chunk...")
}
```

Document Size Limits - Deep Theory

The 16MB Limit in Depth

The 16MB limit applies to **every BSON document**, including:

- Stored documents
- Aggregation pipeline results

- Messages sent over the wire
- Index entries (indirectly)

What Happens When You Hit the Limit:

javascript

```javascript
// Insert too large - fails
db.collection.insertOne(hugeDocument);
// WriteError: document too large

// Aggregation produces large results - fails
db.collection.aggregate([
  { $group: { _id: null, allData: { $push: "$$ROOT" } } }
]);
// Command failed: aggregation result exceeds maximum document size

// Update that would exceed limit - fails
db.collection.updateOne(
  { _id: 1 },
  { $push: { largeArray: { $each: manyElements } } }
);
// Resulting document after update would be too large
```

Working Around the 16MB Limit

1. Paginate Large Results:

javascript

```javascript
// Instead of returning all at once
const cursor = db.collection.find().batchSize(100);
while (cursor.hasNext()) {
  const batch = [];
  for (let i = 0; i < 100 && cursor.hasNext(); i++) {
    batch.push(cursor.next());
  }
  processBatch(batch);
}
```

2. Use $slice for Array Fields:

javascript

```javascript
// Only return part of large arrays
db.posts.find(
  { _id: postId },
  { comments: { $slice: [page * 20, 20] } }
);
```

3. Stream Large Results:

```javascript
// Use cursor streaming instead of loading all
```

```
const stream = db.largeCollection.find().stream();
stream.on('data', (doc) => process(doc));
stream.on('end', () => console.log('Done'));
```

Read/Write Patterns - Deep Theory

Understanding Workload Patterns

Every application has a unique read/write pattern. Optimizing for your specific pattern is crucial.

The Read/Write Spectrum:

```text
Write-Heavy                         Read-Heavy
    |                                   |
    ▼                                   ▼
Logging, IoT           Balanced           Caching, Catalogs
Analytics                                 Reporting
```

Write Patterns

Write-Heavy Workload Characteristics:

- High insert/update throughput
- Index updates on every write
- Journal and oplog pressure
- Lock contention on hot documents

Optimizing for Writes:

```javascript
// 1. Batch writes
db.collection.bulkWrite([
  { insertOne: { document: doc1 } },
  { insertOne: { document: doc2 } },
  // ... 1000 operations
], { ordered: false });

// 2. Use unacknowledged write concern when safe
db.collection.insertOne(doc, { writeConcern: { w: 0 } });

// 3. Minimize indexes on write-heavy collections
// Only index fields used for queries
```

```javascript
// 4. Use document pre-allocation
db.collection.insertOne({
  _id: userId,
  data: {},
  history: new Array(100).fill(null)  // Pre-allocate
});

// 5. Shard by write key to distribute load
sh.shardCollection("db.logs", { shardKey: "hashed" });
```

Read Patterns

Read-Heavy Workload Characteristics:

- Complex queries with filters/sorts
- Heavy index usage
- Potential for covered queries
- Cache sensitivity

Optimizing for Reads:

javascript

```javascript
// 1. Create covering indexes
db.products.createIndex(
  { category: 1, price: 1, name: 1 }
);
// Query only needs these fields

// 2. Use read preference to offload to secondaries
db.collection.find().readPref("secondary");

// 3. Project only needed fields
db.users.find(
  { status: "active" },
  { name: 1, email: 1, _id: 0 }
);

// 4. Use hint to force correct index
db.orders.find({ status: "shipped" })
  .hint({ status: 1, orderDate: -1 });

// 5. Cache frequent queries
// Application-level caching
```

Mixed Workload Patterns

Most applications have mixed workloads:

javascript

```javascript
// Example: E-commerce application
```

```
// - Product catalog: Read-heavy, cached
// - Orders: Write-medium, read-medium
// - Inventory: Write-heavy, consistency critical
// - Analytics: Write-heavy, eventual consistency

// Strategy per workload:
const strategies = {
  products: { caching: true, replicaReads: true },
  orders: { transactions: true, indexes: ['status', 'customerId'] },
  inventory: { sharding: true, writeConcern: 'majority' },
  analytics: { batchWrites: true, ttlIndexes: true }
};
```

Caching Strategies - Deep Theory

The Caching Hierarchy

```text

Level 1: Application Cache (Redis, Memcached)
    ↓
Level 2: MongoDB Cache (WiredTiger Cache)
    ↓
Level 3: Operating System Cache
    ↓
Level 4: Disk
```

WiredTiger Cache Theory

The WiredTiger cache is MongoDB's primary in-memory data store:

```text

┌──────────────────────────────┐
│   WiredTiger Cache           │
├──────────────────────────────┤
│ ┌──────────┐  ┌──────────┐   │
│ │ Page 1   │  │ Page 2   │   │  Hot data
│ └──────────┘  └──────────┘   │
│ ┌──────────┐  ┌──────────┐   │
│ │ Page 3   │  │ Page 4   │   │  Warm data
│ └──────────┘  └──────────┘   │
│   ... (eviction) ...         │
└──────────────────────────────┘
        ↓
┌──────────────────────────────┐
│         Disk                 │  Cold data
└──────────────────────────────┘
```

Cache Behavior:

1. **Reads**: Check cache first (cache hit = fast, cache miss = disk I/O)
2. **Writes**: Write to cache, journal, then lazily to disk
3. **Eviction**: LRU (Least Recently Used) when cache full
4. **Checkpoint**: Every 60 seconds, flush to disk

Cache Sizing Theory

javascript

```
// Cache size formula
Cache Size = Working Set × 1.2

Where:
- Working Set = Frequently accessed data + Indexes
- 1.2 = Safety margin

// Example:
Total Data: 100GB
Working Set (20%): 20GB
Indexes: 5GB
Total Working Set: 25GB
Recommended Cache: 30GB
```

Application-Level Caching

Cache-Aside Pattern:

javascript

```
async function getUser(userId) {
  // 1. Try cache first
  const cached = await redis.get(`user:${userId}`);
  if (cached) return JSON.parse(cached);

  // 2. Cache miss - get from MongoDB
  const user = await db.users.findOne({ _id: userId });

  // 3. Store in cache with TTL
  await redis.setex(`user:${userId}`, 3600, JSON.stringify(user));

  return user;
}

async function updateUser(userId, updates) {
  // 1. Update database
  await db.users.updateOne({ _id: userId }, { $set: updates });
```

```
  // 2. Invalidate cache
  await redis.del(`user:${userId}`);

  // 3. Or update cache (write-through)
  // const user = await db.users.findOne({ _id: userId });
  // await redis.setex(`user:${userId}`, 3600, JSON.stringify(user));
}
```

Cache Strategies Comparison:

Strategy	Read Performance	Write Performance	Consistency
Cache-Aside	High	Medium	Eventual
Write-Through	High	Low	Strong
Write-Behind	High	High	Eventual
Refresh-Ahead	Very High	Medium	Eventual

Query Result Caching

javascript

```javascript
// Cache expensive aggregation results
async function getDashboardData(date) {
  const cacheKey = `dashboard:${date}`;

  // Check cache
  const cached = await redis.get(cacheKey);
  if (cached) return JSON.parse(cached);

  // Expensive aggregation
  const result = await db.sales.aggregate([
    { $match: { date: { $gte: date, $lt: nextDay(date) } } },
    { $group: { _id: "$product", total: { $sum: "$amount" } } },
    { $sort: { total: -1 } },
    { $limit: 10 }
  ]).toArray();

  // Cache for 1 hour (data changes slowly)
  await redis.setex(cacheKey, 3600, JSON.stringify(result));

  return result;
}
```

11. Replication

The Fundamental Theory of Replication

What is Replication?

Replication is the process of synchronizing data across multiple servers. It's the foundation of high availability and disaster recovery in MongoDB.

The Core Idea:

```text

Primary (Master)              Secondaries (Slaves)
┌──────────────┐              ┌──────────────┐
│   Writes     │ ──oplog──→   │   Reads      │
│   Reads      │              │   Backup     │
└──────────────┘              └──────────────┘
        │                           │
        └──────────────┬────────────┘
               Automatic Failover
```

Why Replication Matters

1. **High Availability**: System stays up when servers fail
2. **Data Redundancy**: Multiple copies prevent data loss
3. **Read Scaling**: Distribute read load across servers
4. **Disaster Recovery**: Survive datacenter failures
5. **Maintenance**: Upgrade servers without downtime

The CAP Theorem and Replication

Replication involves the trade-off between Consistency and Availability:

```text

CP System (MongoDB default):
- Prefers consistency over availability
- Waits for majority on writes
- May reject writes during network partition
```

```
AP System (Eventual consistency):
- Prefers availability over consistency
- Accepts writes during partition
- May serve stale reads
```

Replica Sets - Deep Theory

What is a Replica Set?

A replica set is a group of MongoDB servers that maintain the same dataset through replication. It's the fundamental unit for high availability.

Replica Set Structure:

```text
                    ┌─────────────────┐
                    │   Arbiter       │
                    │ (Vote only)     │
                    └─────────┬───────┘
                              │
┌─────────────────────────────┼─────────────────────────────┐
│                    Primary│                              │
│                   ┌────────▼────────┐                    │
│                   │   Primary       │                    │
│                   │ (Read + Write)│                    │
│                   └────────┬────────┘                    │
│                            │                             │
│            ┌───────────────┼───────────────┐             │
│            │               │               │             │
│     ┌──────▼──────┐  ┌──────▼──────┐ ┌──────▼──────┐    │
│     │ Secondary 1 │  │ Secondary 2│ │ Secondary 3│    │
│     │ (Read only) │  │ (Read only)│ │ (Read only)│    │
│     └─────────────┘  └─────────────┘ └─────────────┘    │
│                                                          │
│              One Replica Set                             │
└──────────────────────────────────────────────────────────┘
```

Member Types:

Member Type	Can be Primary?	Votes	Data	Use Case
Primary	Yes	1	Full	Handles all writes
Secondary	Yes (if elected)	1	Full	Read scaling, backup
Arbiter	No	1	None	Tie-breaking votes
Hidden	No	1	Full	Reporting, backup
Delayed	No	1	Full (lagged)	Disaster recovery
Priority 0	No	1	Full	Never becomes primary

The Election Process

When a primary fails, secondaries hold an election:

```text
Step 1: Detect Primary Failure
Secondary 1: "I haven't heard from primary in 10 seconds"

Step 2: Request Election
Secondary 1: "I request to become primary"
Secondary 2: "I vote for Secondary 1"
Secondary 3: "I vote for Secondary 1"
Arbiter: "I vote for Secondary 1"

Step 3: Win Election
Secondary 1: "I have majority votes, I am now primary"

Step 4: Catch Up
Secondary 1: "Applying any missed operations from oplog"
```

```
Step 5: Resume Operations
New Primary: "Ready to accept writes"
```

Election Factors:

- **Priority**: Higher priority members more likely to win
- **Oplog freshness**: Must be within 5 seconds of latest op
- **Network connectivity**: Must reach majority
- **Votes**: Must receive majority of possible votes

Oplog (Operations Log) Theory

The oplog is a special capped collection that records all write operations:

javascript

```javascript
// Oplog document structure
{
  "ts": Timestamp(1672531200, 1),  // Operation time + ordinal
  "t": NumberLong(1),               // Term (election ID)
  "h": NumberLong("7280111994143649393"), // Unique hash
  "v": 2,                           // Version
  "op": "i",                         // Operation type: i=insert, u=update, d=dele
te
  "ns": "test.users",                // Namespace (database.collection)
  "o": {                             // The actual operation
    "_id": ObjectId("..."),
    "name": "John"
  },
  "o2": { _id: 1 }                   // Update criteria (for updates)
}

// View oplog
use local
db.oplog.rs.find().sort({$natural: -1}).limit(1)
```

Oplog Size Considerations:

Workload	Recommended Oplog Size	Reasoning
Light	1-2GB	Enough for maintenance window
Normal	5-10% of disk	24-48 hours of operations
Heavy writes	10-20% of disk	Accommodate peak loads

Workload	Recommended Oplog Size	Reasoning
Initial sync	Larger	Secondary needs time to sync

```javascript
// Check oplog size and window
db.getReplicationInfo();

// Resize oplog
db.adminCommand({ replSetResizeOplog: 1, size: 16000 }); // 16GB
```

Primary vs Secondary - Deep Theory

Primary Role

The primary is the **leader** of the replica set:

Responsibilities:

1. **Accept all writes** (by default)
2. **Maintain oplog** of all operations
3. **Coordinate elections** when needed
4. **Track secondaries' progress**
5. **Handle all operations requiring linearizable reads**

Primary Behavior:

```javascript
// Check if current node is primary
db.isMaster().ismaster;  // true for primary

// Primary can be demoted
rs.stepDown(60);  // Step down for 60 seconds

// Force re-election
rs.reconfig(config, { force: true });
```

Secondary Role

Secondaries are **followers** that replicate data from the primary:

Responsibilities:

1. **Apply operations** from primary's oplog
2. **Serve reads** (if configured)
3. **Vote in elections**
4. **Can become primary** if elected
5. **Maintain copy of data**

Secondary States:

```text
STARTUP     →  Starting up, not yet part of set
RECOVERING  →  Catching up with primary
SECONDARY   →  Fully synced, serving reads
ROLLBACK    →  Undoing operations after primary change
ARBITER     →  Arbiter node (no data)
```

Replication Lag

Replication lag is the delay between an operation on primary and its application on secondaries:

```javascript
// Check replication lag
rs.printSlaveReplicationInfo();

// Output:
// source: mongo1:27017
// syncedTo: Tue Jan 15 2024 10:30:45 GMT+0000
// lag: 5 secs (acceptable)

// Causes of lag:
// 1. Network latency
// 2. Secondary underpowered
// 3. Long-running operations
// 4. Index builds
// 5. Lock contention

// Monitor lag
db.adminCommand({ replSetGetStatus: 1 }).members.forEach(member => {
  print(member.name, "lag:", member.optimeDate - member.primaryOptimeDate);
});
```

Automatic Failover - Deep Theory

How Failover Works

Automatic failover is the process of electing a new primary when the current primary becomes unavailable:

Failure Detection:

```text
Time: T0 - Primary healthy
Time: T1 - Network issue, primary unreachable
Time: T2 - Secondaries detect missed heartbeat
Time: T3 - Election timeout (default 10 seconds)
Time: T4 - Election begins
Time: T5 - New primary elected
Time: T6 - New primary accepts writes
```

Heartbeat Mechanism:

- Members send heartbeats every 2 seconds
- After 10 seconds without response, member considered unavailable
- Election triggered if primary unreachable

Failover Scenarios

Scenario 1: Primary Crash

```text
1. Primary process dies
2. Secondaries detect missing heartbeats
3. Election held among reachable secondaries
4. New primary elected
5. Writes resume (typically 10-30 seconds downtime)
```

Scenario 2: Network Partition

```text
Situation: Primary isolated from rest of set
Result:
- Primary sees itself alone, steps down
```

```
- Remaining members elect new primary
- Old primary rejoins as secondary
```

Scenario 3: Data Center Failure

```text
Setup: Replica set across 3 data centers
Failure: One data center goes offline
Outcome: Remaining data centers continue if they have majority
```

Failover Configuration

```javascript
// Configure election timeout
rs.config().settings = {
  electionTimeoutMillis: 10000,  // Default 10 seconds
  heartbeatIntervalMillis: 2000,  // Default 2 seconds
  catchUpTimeoutMillis: 60000      // Time to catch up after election
};

// Set priorities (higher = more likely to become primary)
cfg = rs.conf();
cfg.members[0].priority = 10;   // Preferred primary
cfg.members[1].priority = 5;    // Backup
cfg.members[2].priority = 1;    // Least preferred
rs.reconfig(cfg);

// Prevent certain members from becoming primary
cfg.members[3].priority = 0;    // Never becomes primary
rs.reconfig(cfg);
```

Handling Failover in Applications

```javascript
// Driver handles failover automatically
const client = new MongoClient(uri, {
  replicaSet: 'rs0',
  retryWrites: true,      // Automatically retry writes after failover
  retryReads: true        // Automatically retry reads after failover
});

// But applications should handle:
try {
  await collection.insertOne(doc);
} catch (e) {
  if (e.code === 11600) {  // InterruptedDueToReplStateChange
    // Primary stepped down, retry
    await collection.insertOne(doc);
  }
}
```

Read Preferences - Deep Theory

What are Read Preferences?

Read preferences control how MongoDB clients route read operations to replica set members. They're crucial for scaling reads and ensuring consistency.

The Read Preference Spectrum:

```text
Strong Consistency                    Eventual Consistency
        │                                     │
        ▼                                     ▼
    primary                        secondary, nearest
        │                                     │
   Consistent                       Potentially stale
   Always current                    Better latency
```

Read Preference Modes

Mode	Description	Use Case
primary	All reads go to primary (default)	Strong consistency needed
primaryPreferred	Primary if available, else secondary	Read scaling with consistency fal
secondary	Always read from secondaries	Offload reads, analytics
secondaryPreferred	Secondaries if available, else primary	Reduce primary load
nearest	Lowest latency member	Geo-distributed apps

javascript

```javascript
// Read preference examples

// 1. Primary (default) - strong consistency
db.users.find().readPref("primary");
```

```
// 2. Secondary - offload reads
db.analytics.find().readPref("secondary");

// 3. Nearest - lowest latency
db.products.find().readPref("nearest");

// 4. With tags - target specific members
db.orders.find().readPref("secondary", [
  { region: "us-east", tier: "ssd" }
]);

// 5. In connection string
mongodb://server1,server2,server3/?replicaSet=rs0&readPreference=secondary
```

Read Preference + Tag Sets

Tag sets allow fine-grained control over which secondaries handle reads:

javascript

```
// Configure tags
cfg = rs.conf();
cfg.members[0].tags = { region: "us-east", tier: "high", disk: "ssd" };
cfg.members[1].tags = { region: "us-east", tier: "standard", disk: "hdd" };
cfg.members[2].tags = { region: "us-west", tier: "high", disk: "ssd" };
rs.reconfig(cfg);

// Read from high-tier SSDs in us-east
db.collection.find().readPref("secondary", [
  { region: "us-east", tier: "high" }
]);

// Fallback chain
db.collection.find().readPref("secondary", [
  { region: "us-east", tier: "high" },  // Try these first
  { region: "us-east" },                 // Then any in us-east
  {}                                     // Then any secondary
]);
```

Read Preference and Consistency

Causal Consistency with Read Preferences:

javascript

```
// Session ensures causal consistency across reads/writes
const session = client.startSession({
  defaultTransactionOptions: {
    readPreference: { mode: "secondary" },
    readConcern: { level: "majority" }
  }
});

session.withTransaction(async () => {
  const collection = session.client.db("test").users;
```

```
  // Write to primary
  await collection.insertOne({ name: "John" });

  // Read from secondary - guaranteed to see the write
  // because session maintains causal consistency
  const user = await collection.findOne({ name: "John" });
});
```

Consistency Levels with Read Preferences:

Read Preference	Read Concern	Consistency Guarantee
primary	local	Strong (current data)
primary	majority	Strong + durable
secondary	local	Possibly stale (replication lag)
secondary	majority	May be stale, but durable
nearest	local	Lowest latency, may be stale

Summary: The Complete Mental Model

Performance Optimization

Think of performance optimization as **tuning a race car**:

- **Indexes** are the tires - they provide traction (selectivity)
- **Queries** are the driving line - efficient path through data
- **Documents** are the weight - lighter is faster
- **Cache** is the fuel - must be sufficient for the race
- **Hardware** is the engine - limits ultimate speed

Replication

Think of replication as **airline fleet management**:

- **Primary** is the lead plane - coordinates all flights
- **Secondaries** are backup planes - ready to take over
- **Oplog** is the flight recorder - logs all actions
- **Failover** is emergency landing procedure - automatic but careful
- **Read preferences** are passenger routing - direct based on needs

The Golden Rules

For Performance:

1. **Index selectivity is everything** - high cardinality fields first
2. **Profile before optimizing** - know what's actually slow
3. **Document size matters** - keep them small (< 4MB ideal)
4. **Cache is king** - design for working set to fit in RAM
5. **Batch operations** - reduce round trips

For Replication:

1. **Plan for failure** - test failover scenarios
2. **Monitor replication lag** - it's your early warning system
3. **Right-size oplog** - enough for maintenance, not so large it wastes space
4. **Use read preferences wisely** - consistency vs. performance trade-offs
5. **Tag sets for complex routing** - when one size doesn't fit all

MongoDB Sharding & Architecture: Complete Theory Notes for Senior Developers

---

12. Sharding (Scalability)

The Fundamental Theory of Sharding

What is Sharding?

Sharding is MongoDB's approach to **horizontal scaling**—distributing data across multiple machines to handle datasets too large for a single server. It's not just about storage; it's about distributing workload.

The Core Problem Sharding Solves:

```text

Single Server Limits:
├── CPU: One server's processing power
├── RAM: One server's memory (caching limit)
├── Disk: One server's storage capacity
├── IOPS: One server's disk throughput
└── Network: One server's bandwidth

Sharding Solution:
├── CPU: Aggregate power of N servers
├── RAM: Combined cache of N servers
├── Disk: Total storage across cluster
├── IOPS: Parallel I/O across servers
└── Network: Distributed bandwidth
```

Vertical vs Horizontal Scaling

Vertical Scaling (Scale Up):

```text

Add more power to existing server
```

```
├── Bigger machine (more RAM, faster CPU)
├── Simpler architecture
├── Limited by hardware maximums
└── Expensive at high end
```

Horizontal Scaling (Scale Out):

```text

Add more servers to the cluster
├── Linear scalability
├── Commodity hardware
├── No theoretical limit
└── Complex architecture
```

The Decision Point: When to Shard

```text

Is single server sufficient?
├── YES → Don't shard (keep it simple)
└── NO → Can you optimize first?
    ├── YES → Optimize (indexes, schema)
    └── NO → Need to shard
        ├── Data > 2-3TB
        ├── Working set > RAM
        ├── Write throughput > single server
        └── Need geographical distribution
```

Sharding Architecture Components

The Three Pillars of Sharding

```text

                        ┌─────────────────┐
                        │   Application   │
                        └────────┬────────┘
                                 │
                    ┌────────────┼────────────┐
                    │            │            │
              ┌─────▼─────┐  ┌─────▼─────┐  ┌─────▼─────┐
              │  mongos   │  │  mongos   │  │  mongos   │  (Query Routers)
              └───────────┘  └───────────┘  └───────────┘
                          │            │            │
```

```
                  │             │             │
                  └─────────────┼─────────────┘
                                │
                  ┌─────────────┼─────────────┐
                  │             │             │
            ┌─────▼─────┐  ┌────▼──────┐  ┌────▼──────┐
            │ Config    │  │ Config    │  │ Config    │  (Config Servers)
            │ Server 1  │  │ Server 2  │  │ Server 3  │
            └───────────┘  └───────────┘  └───────────┘
                  │             │             │
                  └─────────────┼─────────────┘
                                │
            ┌───────────────────┼───────────────────┐
            │                   │                   │
      ┌─────▼─────┐       ┌─────▼─────┐       ┌─────▼─────┐
      │  Shard 1  │       │  Shard 2  │       │  Shard 3  │
      │(Replica   │       │(Replica   │       │(Replica   │
      │   Set)    │       │   Set)    │       │   Set)    │
      └───────────┘       └───────────┘       └───────────┘
```

1. Query Routers (mongos):

- Route queries to appropriate shards
- Aggregate results from multiple shards
- No persistent state (stateless)
- Can run many instances

2. Config Servers:

- Store cluster metadata
- Map chunks to shards
- Must be replica set (3 minimum)
- Critical for cluster operation

3. Shards:

- Store actual data
- Each shard is a replica set
- Provide data redundancy

- Handle read/write operations

Shard Key Selection - The Most Critical Decision

What is a Shard Key?

A shard key is a field or compound field that determines how MongoDB distributes documents across shards. It's the **single most important decision** in sharding.

The Shard Key's Role:

```text
Document → Shard Key Value → Hash/ Range → Target Shard
                  ↓
         Determines distribution
         Affects query performance
         Cannot be changed after sharding
```

Characteristics of a Good Shard Key

1. High Cardinality

```javascript
// GOOD: High cardinality (many unique values)
{ _id: ObjectId, email: "user@example.com" }
// email has many possible values

// BAD: Low cardinality
{ status: "active" }  // Only 2-3 possible values
// All "active" documents go to one shard!
```

2. Low Frequency

```javascript
// GOOD: Values evenly distributed
{ customerId: 12345 }  // Each customer has similar data volume

// BAD: High frequency values
{ region: "US-East" }  // One region has 80% of data
// Creates a "jumbo" shard
```

3. Non-Monotonically Changing

```javascript
// BAD: Monotonically increasing
```

```
{ timestamp: ISODate() }  // All new data goes to last shard

// GOOD: Hashed or distributed
{ hashed(timestamp): ... }  // Even distribution
{ customerId: ..., timestamp: ... }  // Compound with distribution
```

Shard Key Strategies

Strategy 1: Hashed Shard Key

javascript

```
// Create hashed index
sh.shardCollection("db.events", { userId: "hashed" });

// Pros:
// - Even distribution guaranteed
// - Simple to choose
// - Good for write scaling

// Cons:
// - Range queries hit all shards
// - No data locality for related documents

// Best for:
// - High write throughput
// - No range queries needed
// - IoT data, logs, events
```

Strategy 2: Ranged Shard Key

```
javascript

// Create ranged shard key
sh.shardCollection("db.orders", { customerId: 1, orderDate: -1 });


// Pros:
// - Efficient range queries
// - Data locality (customer's orders together)
// - Good for read-heavy workloads


// Cons:
// - Can create hotspots
// - Requires careful planning
// - Uneven distribution possible


// Best for:
// - Customer-centric data
// - Range queries common
// - Time-series with partitioning
```

Strategy 3: Compound Shard Key

javascript

```javascript
// Compound key for best of both worlds
sh.shardCollection("db.sales", {
  region: 1,           // First part for queries
  hashed(_id): 1       // Second for distribution
});

// Or
sh.shardCollection("db.orders", {
  customerId: 1,       // Equality queries
  orderDate: 1         // Range queries
});

// Pros:
// - Supports diverse query patterns
// - Better distribution control
// - Can optimize for common queries

// Cons:
// - Complex to design
// - Must include in all queries
```

Shard Key Decision Framework

```text
What are your query patterns?
├── Mostly point queries (equality) → Hashed key good
├── Mostly range queries → Ranged key needed
└── Mixed → Compound key

What's your write pattern?
├── High write volume → Need even distribution
└── Moderate writes → Can optimize for reads

Can you include shard key in all queries?
├── YES → Good (targeted queries)
└── NO → Will have scatter-gather queries

Is data growth predictable?
├── YES → Can plan ranges
└── NO → Use hashed for safety
```

Chunk Distribution - Deep Theory

What are Chunks?

Chunks are contiguous ranges of shard key values that MongoDB uses to manage data distribution. They're the **unit of data movement** in a sharded cluster.

text

```
Chunk Structure:
┌──────────────────────────────────────────┐
│ Chunk Range: [1000 - 2000)               │
│                                          │
│ Documents:                               │
│ { _id: 1001, ... }                       │
│ { _id: 1002, ... }                       │
│ ...                                      │
│ { _id: 1999, ... }                       │
│                                          │
│ Size: ~64MB (default)                    │
│ Shard: Shard A                           │
└──────────────────────────────────────────┘
```

Chunk Splitting

As data grows, chunks automatically split when they exceed the chunk size:

text

```
Before Split (Chunk size > 64MB):
┌──────────────────────────────┐
│ [1000 - 2000) (128MB)        │
│ Documents: 1000-1999         │
└──────────────────────────────┘


Split Point Detection:
┌──────────────────────────────┐
│ [1000 - 2000) (128MB)        │
│ Split at 1500 (median)       │
└──────────────┬───────────────┘
               ↓
After Split:
┌─────────────┐ ┌─────────────┐
```

```
│ [1000-1500) │ │ [1500-2000) │
│ (64MB)      │ │ (64MB)      │
└─────────────┘ └─────────────┘
```

javascript

```javascript
// Monitor chunks
use config
db.chunks.find({ ns: "db.collection" }).pretty()

// Manual split (if needed)
sh.splitAt("db.collection", { shardKey: 1500 })

// View chunk distribution
sh.status()
```

Chunk Migration

The balancer moves chunks between shards to maintain balance:

```text
Before Balancing:
Shard A: 10 chunks (100GB)
Shard B: 2 chunks (20GB)
Shard C: 3 chunks (30GB)

Balancer Decision:
- Shard A has 5 chunks more than average
- Move 2 chunks from A to B and C

Migration Process:
1. Source shard marks chunk as "moving"
2. Target shard builds indexes
3. Data copied in background
4. Metadata updated when complete
5. Source shard deletes original

After Balancing:
Shard A: 8 chunks (80GB)
Shard B: 3 chunks (30GB)
Shard C: 4 chunks (40GB)
```

Balancing Configuration

javascript

```javascript
// Check balancer state
sh.getBalancerState()
```

```
// Enable/disable balancer
sh.startBalancer()
sh.stopBalancer()

// Configure balancing window (maintenance window)
db.settings.update(
  { _id: "balancer" },
  { $set: {
    activeWindow: {
      start: "01:00",  // 1 AM
      stop: "06:00"    // 6 AM
    }
  }},
  { upsert: true }
)

// Set chunk size (default 64MB)
db.settings.update(
  { _id: "chunksize" },
  { $set: { value: 128 } },  // 128MB chunks
  { upsert: true }
)
```

Jumbo Chunks

Jumbo chunks are chunks that cannot be split or moved:

javascript

```
// Causes of jumbo chunks:
// 1. All documents have same shard key value
// 2. Single document > chunk size (impossible to split)
// 3. Migration failures

// Detect jumbo chunks
sh.status(true)  // Shows jumbo flag

// Clear jumbo flag (if resolved)
db.adminCommand({
  cleanupOrphaned: "db.collection",
  startingFromKey: { shardKey: value }
})
```

Balancing Data - Deep Theory

The Balancer Algorithm

The balancer runs continuously to keep data evenly distributed:

text

```
Every few seconds:
1. Check if balancing is enabled
```

```
2. Calculate chunks per shard
3. Find most and least loaded shards
4. Calculate migration threshold
5. Move chunks if imbalance > threshold

Threshold Formula:
   If maxChunks - minChunks > migrationThreshold
   Then move chunks until balanced

Migration Threshold:
   < 20 chunks: 2 chunks difference
   >= 20 chunks: 20% difference
```

Zone Sharding (Data Locality)

Zones allow you to control where data lives:

javascript

```javascript
// Define zones
sh.addShardTag("shard01", "US")
sh.addShardTag("shard02", "EU")
sh.addShardTag("shard03", "ASIA")

// Assign ranges to zones
sh.addTagRange(
  "db.users",
  { region: "US" },    // Start of range
  { region: "US" },    // End of range
  "US"                 // Zone name
)

sh.addTagRange(
  "db.users",
  { region: "EU" },
  { region: "EU" },
  "EU"
)

// Now:
// - US users stay on US shards
// - EU users stay on EU shards
// - Queries from EU only hit EU shards
```

Manual vs Automatic Balancing

javascript

```javascript
// Automatic (default)
sh.setBalancerState(true)
// Let MongoDB manage distribution

// Manual intervention
```

```
// Move specific chunk
sh.moveChunk(
  "db.collection",
  { shardKey: 1500 },  // Find chunk containing this key
  "shard02"            // Target shard
)

// Split hot chunk manually
sh.splitAt(
  "db.collection",
  { shardKey: 5000 }
)

// Merge chunks (reduce fragmentation)
sh.mergeChunks(
  "db.collection",
  { shardKey: 1000 },  // Lower bound
  { shardKey: 2000 }   // Upper bound
)
```

13. MongoDB Architecture

Storage Engine (WiredTiger) - Deep Theory

What is a Storage Engine?

The storage engine is the component responsible for managing how data is stored, both in memory and on disk. It's the **foundation** of MongoDB's performance and reliability.

WiredTiger Architecture:

```text
                    ┌──────────────────────────────────────┐
                    │            MongoDB Layer             │
                    └──────────────┬───────────────────────┘
                                   │
                    ┌──────────────▼───────────────────────┐
                    │         WiredTiger Storage Engine    │
                    ├──────────────────────────────────────┤
                    │  ┌──────────────┐  ┌──────────────┐  │
                    │  │    Cache     │  │   Journal    │  │
                    │  │  (Memory)    │  │   (Disk)     │  │
                    │  └──────┬───────┘  └──────┬───────┘  │
                    │         │                 │          │
```

```
                    │  ┌──────▼──────────────────▼──────┐ │
                    │  │        Checkpoint (Disk)        │ │
                    │  └────────────────────────────────┘ │
                    └──────────────────────────────────────┘
```

B-Trees and LSM Trees

WiredTiger uses a **B-Tree** structure with MVCC:

text

```
B-Tree Structure:
                    [Root]
                      │
           ┌──────────┼──────────┐
         [Int]      [Int]      [Int]
           │          │          │
      ┌────┼────┐  ┌─┼─┐  ┌────┼────┐
    [Leaf] [Leaf] [Leaf] [Leaf] [Leaf] [Leaf]
     (data) (data) (data) (data) (data) (data)

MVCC (Multi-Version Concurrency Control):
Document v1: { _id: 1, name: "John", balance: 100 }
Document v2: { _id: 1, name: "John", balance: 150 }  (updated)
Document v3: { _id: 1, name: "John", balance: 200 }  (updated)

Readers see consistent snapshot based on timestamp
```

Compression

WiredTiger provides three levels of compression:

javascript

```javascript
// Configuration
storage:
  wiredTiger:
    engineConfig:
      cacheSizeGB: 4
      journalCompressor: snappy
    collectionConfig:
      blockCompressor: snappy  # Options: none, snappy, zlib, zstd
    indexConfig:
      prefixCompression: true
```

```
// Compression ratios:
// Snappy: 2-4x compression, fast (default)
// Zlib: 3-5x compression, slower
// Zstd: 3-5x compression, balanced
// None: No compression, fastest for SSDs
```

Cache Management

```javascript
// Cache behavior
db.serverStatus().wiredTiger.cache

// Key metrics:
// - bytes currently in the cache
// - maximum bytes configured
// - pages read into cache
// - pages written from cache

// Cache eviction:
// 1. Clean pages (unmodified) evicted first
// 2. Dirty pages (modified) written then evicted
// 3. LRU (Least Recently Used) algorithm
// 4. Eviction when cache reaches 80% (configurable)
```

Journaling - Deep Theory

What is Journaling?

Journaling is MongoDB's mechanism for ensuring **durability**—guaranteeing that committed writes survive a crash or restart.

The Journaling Flow:

```text
Write Operation → Memory (Cache) → Journal (Disk) → Acknowledged
                                          ↓
                                     Checkpoint
                                          ↓
                                     Data Files
```

Journal Files

```javascript
// Journal location
/data/db/journal/
```

```
├── WiredTigerLog.0000000001
├── WiredTigerLog.0000000002
├── WiredTigerLog.0000000003
└── WiredTigerPreplog.0

// Journal entry structure
{
  "timestamp": ISODate(...),
  "operation": "update",
  "document": { ... },
  "checksum": "..."
}
```

Journaling Configuration

javascript

```javascript
// Journal settings
storage:
  journal:
    enabled: true
    commitIntervalMs: 100  // How often to write to journal

// Write concern with journal
db.collection.insertOne(
  { name: "John" },
  { writeConcern: { w: 1, j: true } }  // Wait for journal
)

// Journal vs No Journal:
// j: true  - Data safe after crash, slower
// j: false - Risk of losing last 100ms of writes, faster
```

Recovery Process

When MongoDB restarts after a crash:

text

```
1. Find last checkpoint
2. Replay journal entries after checkpoint
3. Apply committed transactions
4. Uncommitted transactions discarded
5. Data files now consistent
```

Memory Usage - Deep Theory

The Memory Hierarchy

```text

L1: WiredTiger Cache (Configurable)
    ├── Working set (hot data)
    ├── Index pages
```

```
    └── Document pages

L2: Filesystem Cache (OS managed)
    ├── Recently accessed files
    └── Read-ahead buffers

L3: Disk (Slowest)
    └── All data
```

Working Set Theory

The working set is the subset of data and indexes that is **frequently accessed**:

```text
Total Data: 500GB
Working Set (20%): 100GB
Indexes: 30GB
Total Working Set: 130GB

If RAM = 128GB:  → Page faults, slower
If RAM = 256GB:  → All in memory, fast
```

Memory Calculation

javascript

```javascript
// Calculate working set size
// 1. Find frequently accessed documents
db.collection.aggregate([
  { $match: { lastAccess: { $gt: new Date(Date.now() - 24*60*60*1000) } } },
  { $group: { _id: null, totalSize: { $sum: { $bsonSize: "$$ROOT" } } } }
])

// 2. Add index sizes
db.collection.stats().totalIndexSize

// 3. Add overhead (20%)
Recommended Cache = (HotData + Indexes) * 1.2
```

Memory Pressure Indicators

javascript

```javascript
// Monitor memory pressure
db.serverStatus().metrics.cursor.timedOut  // Cursor timeouts
db.serverStatus().wiredTiger.cache.pages_evicted  // Eviction rate
db.serverStatus().wiredTiger.cache.pages_read_into_cache  // Page faults

// Check page fault rate
db.serverStatus().extra_info.page_faults
```

```
// If page faults increasing → Working set too large for RAM
```

Write Concern - Deep Theory

What is Write Concern?

Write concern specifies the level of acknowledgment requested from MongoDB for write operations. It's the primary control for the **durability vs. performance trade-off**.

The Write Concern Spectrum:

```text
w: 0                  → Fastest, least durable
w: 1                  → Acknowledged by primary
w: majority           → Acknowledged by majority of replica set
w: <number>           → Acknowledged by N members
j: true               → Written to journal
wtimeout: <ms>        → Fail if not acknowledged in time
```

Write Concern Levels

```javascript
// Level 0: Unacknowledged (fire and forget)
db.collection.insertOne(
  { name: "John" },
  { writeConcern: { w: 0 } }
)
// Fastest, but no confirmation
// Use for: logging, metrics, non-critical data

// Level 1: Acknowledged (default)
db.collection.insertOne(
  { name: "John" },
  { writeConcern: { w: 1 } }
)
// Confirms primary received write
// Use for: most applications

// Level majority: Replica acknowledged
db.collection.insertOne(
  { name: "John" },
  { writeConcern: { w: "majority", wtimeout: 5000 } }
)
// Confirms most replicas have write
// Use for: critical data, financial transactions

// Level all: Full acknowledgment
```

```javascript
db.collection.insertOne(
  { name: "John" },
  { writeConcern: { w: "all", j: true } }
)
// Confirms all members have journaled write
// Slowest, most durable
// Use for: absolute must-not-lose data
```

Write Concern Internals

```text
Client writes with w: majority

1. Primary receives write
2. Primary writes to its oplog
3. Primary sends to secondaries
4. Secondaries apply and acknowledge
5. Primary counts acknowledgments
6. When majority reached, primary acknowledges client
7. Client receives confirmation
```

Write Concern in Replica Sets

```javascript
// In a 3-node replica set:
// w: 1      → Wait for primary only
// w: 2      → Wait for primary + 1 secondary
// w: majority → Wait for 2 nodes (primary + 1 secondary)
// w: 3      → Wait for all nodes (primary + 2 secondaries)

// Custom write concern names
cfg = rs.conf()
cfg.settings.getLastErrorDefaults = { w: "majority", wtimeout: 5000 }
cfg.settings.getLastErrorModes = {
  "multipleDC": { "dc": 2 }  // Wait for 2 data centers
}
rs.reconfig(cfg)
```

Read Concern - Deep Theory

What is Read Concern?

Read concern controls the consistency and isolation properties of data read from replica sets and sharded clusters. It determines **how fresh the data** you read must be.

The Read Concern Spectrum:

```text
local        → Fastest, may read uncommitted data (default)
available    → Similar to local, for sharded clusters
majority     → Only read data committed to majority
linearizable → Strongest, read most recent acknowledged write
snapshot     → Read from specific point in time
```

Read Concern Levels

```javascript
// local: Read from primary's current view
db.collection.find().readConcern("local")
// May read uncommitted data in transactions
// Fastest, default
// Use for: most reads

// majority: Only read committed data
db.collection.find().readConcern("majority")
// Guarantees data won't roll back
// Slightly slower
// Use for: critical reads, after writes

// linearizable: Most recent acknowledged write
db.collection.find().readConcern("linearizable")
// Must read from primary
// Slowest, most consistent
// Use for: read-your-writes guarantees across sessions

// snapshot: Point-in-time read
db.collection.find().readConcern("snapshot")
// Used in transactions
// Consistent view throughout transaction
```

Read Concern + Write Concern Matrix

Write Concern	Read Concern	Guarantee
w: 1	local	May read uncommitted

Write Concern	Read Concern	Guarantee
w: majority	majority	Read committed
w: majority	linearizable	Strong consistency
w: 1	majority	May read stale but committed
w: 1	snapshot	Point-in-time consistency

Causal Consistency

javascript

```javascript
// Session ensures causal consistency across operations
const session = client.startSession({
  causalConsistency: true
});

session.withTransaction(async () => {
  // Write with w: majority
  await collection.insertOne({ name: "John" }, { session });

  // Read with readConcern: majority
  // Guaranteed to see the write
  const doc = await collection.findOne(
    { name: "John" },
    { session, readConcern: { level: "majority" } }
  );
});
```

Summary: The Complete Mental Model

Sharding

Think of sharding as **city planning**:

- **Shard key** is the zoning law - determines where things go
- **Chunks** are city blocks - manageable units of data
- **Balancer** is the city planner - redistributes growth
- **mongos** are the roads - route traffic to neighborhoods

- **Config servers** are the city records - know where everything is

Architecture

Think of MongoDB architecture as a **modern office building**:

- **WiredTiger** is the building infrastructure (plumbing, electrical)
- **Cache** is the reception area - where active work happens
- **Journal** is the security camera footage - records everything
- **Write concern** is the receipt - proof of transaction
- **Read concern** is the freshness of information - today's news vs archives

The Golden Rules

For Sharding:

1. **Choose shard key wisely** - it's permanent
2. **High cardinality + low frequency** = good distribution
3. **Include shard key in queries** for efficiency
4. **Monitor chunk distribution** - prevent jumbo chunks
5. **Plan for growth** - today's data is tomorrow's small data

For Architecture:

1. **Working set must fit in cache** for good performance
2. **Journal is for durability**, cache is for speed
3. **Higher write concern = slower writes**
4. **Higher read concern = more consistent reads**
5. **Test with production load** - theory meets reality

MongoDB Security & Backup/Restore: Complete Theory Notes for Senior Developers

---

14. Security

The Fundamental Theory of Security

What is Database Security?

Database security is the practice of protecting database systems from unauthorized access, misuse, and data breaches. In MongoDB, security operates on multiple layers, forming a **defense-in-depth** strategy.

The Security Layers:

```text
Layer 1: Network Security
    ├── Firewalls
    ├── Virtual Private Networks (VPN)
    └── IP Whitelisting

Layer 2: Authentication
    ├── Who can connect?
    ├── Username/Password
    ├── X.509 Certificates
    ├── LDAP/Active Directory
    └── Kerberos

Layer 3: Authorization
    ├── What can they do?
    ├── Role-Based Access Control (RBAC)
    ├── Privileges and Actions
    └── Resource Restrictions

Layer 4: Encryption
    ├── Data in transit (TLS/SSL)
```

```
    ├── Data at rest (Encryption)
    ├── Data in use (Field-level encryption)
    └── Key management

Layer 5: Auditing
    ├── Who did what?
    ├── When did they do it?
    ├── What data was affected?
    └── Audit logs
```

The Principle of Least Privilege

The fundamental principle of security: **Give users only the permissions they absolutely need, and nothing more.**

```text
Wrong: Grant all-powerful root access to application
    └── If compromised, entire database is exposed

Right: Create specific roles with minimal permissions
    └── Application can only read/write its own collections
    └── Even if compromised, damage is contained
```

Security in Depth

No single security measure is perfect. Multiple layers ensure that if one fails, others still protect:

```text
Firewall (network layer)
    ↓
Authentication (identity layer)
    ↓
Authorization (permission layer)
    ↓
Encryption (data layer)
    ↓
Audit (monitoring layer)
```

Authentication - Deep Theory

What is Authentication?

Authentication is the process of verifying the identity of a user or application connecting to MongoDB. It answers the question: **"Who are you?"**

Authentication Mechanisms

1. SCRAM (Salted Challenge Response Authentication Mechanism)

javascript

```javascript
// Default authentication mechanism
// How it works:
// 1. Client sends username
// 2. Server sends random salt + iteration count
// 3. Client hashes password with salt
// 4. Server verifies hash matches stored credentials

// Enable SCRAM
use admin
db.createUser({
  user: "appUser",
  pwd: "securePassword123",
  roles: [ { role: "readWrite", db: "myApp" } ]
})

// Connection string
mongodb://appUser:securePassword123@localhost:27017/myApp
```

2. X.509 Certificate Authentication

javascript

```javascript
// Mutual TLS authentication
// Client presents certificate, server validates

// Generate client certificate
openssl req -newkey rsa:2048 -nodes -keyout client.key -out client.csr
openssl x509 -req -in client.csr -CA ca.pem -CAkey ca.key -out client.crt

// Create user from certificate subject
db.getSiblingDB("$external").runCommand({
  createUser: "CN=client,OU=MyOrg,O=MyCompany",
  roles: [ { role: "readWrite", db: "myApp" } ]
})
```

```
// Connection with certificate
mongodb://localhost:27017/?ssl=true&sslCertificate=client.pem
```

3. LDAP Authentication

```javascript
// Integrate with corporate directory

// mongod.conf
security:
  authorization: enabled
  ldap:
    servers: "ldap.example.com"
    bind:
      method: simple
      username: "cn=admin,dc=example,dc=com"
      password: "secret"

// Authenticate with LDAP credentials
db.auth({
  mechanism: "PLAIN",
  user: "john.doe@example.com",
  pwd: "password123"
})
```

4. Kerberos Authentication

```javascript
// Enterprise environments
security:
  authenticationMechanisms: GSSAPI
  kerberos:
    keytab: /etc/mongodb.keytab
    servicePrincipal: mongodb/mongodb-host@EXAMPLE.COM
```

Authentication Flow

```text
Client Request                MongoDB
    |                            |
    |---- Connect --------------->|
    |                            |
    |<--- Ask for auth -----------|
    |                            |
    |---- Send credentials ------>|
    |                            |
    |    Validate credentials     |
    |    Check against storage    |
    |                            |
```

```
    |<--- Auth success/failure ---|
    |                             |
```

Authorization (Roles) - Deep Theory

What is Authorization?

Authorization determines what an authenticated user is allowed to do. It answers the question: **"What can you do?"**

Role-Based Access Control (RBAC)

MongoDB uses RBAC where permissions are assigned to roles, and roles are assigned to users.

```text
Users                    Roles                    Privileges
┌──────────┐            ┌──────────┐            ┌──────────────┐
│ User A   │──────────▶│ read     │──────────▶│ find on db1  │
└──────────┘            └──────────┘            └──────────────┘
        ┌──────────────▶│ write    │──────────▶│ insert on db1│
        │               └──────────┘            │ update on db1│
┌──────────┐            ┌──────────┐            └──────────────┘
│ User B   │──────────▶│ admin    │──────────▶│ all on all   │
└──────────┘            └──────────┘            └──────────────┘
```

Built-in Roles

Database-Level Roles:

Role	Privileges	Use Case
read	Read all non-system collections	Reporting apps
readWrite	Read and write all non-system collections	Application CRUD
dbAdmin	Manage database (indexes, schema)	DBA tasks

Role	Privileges	Use Case
dbOwner	All dbAdmin + readWrite + userAdmin	Database owner
userAdmin	Create/modify users in database	User management

Cluster-Level Roles:

Role	Privileges	Use Case
clusterAdmin	All cluster management	Cluster DBA
clusterManager	Monitoring, sharding, replication	Operations
clusterMonitor	Read-only cluster status	Monitoring tools
hostManager	Manage servers	System admins

Global Roles:

Role	Privileges	Use Case
root	All privileges on all resources	Emergency only
backup	Backup data	Backup tools
restore	Restore data	Recovery tools
readAnyDatabase	Read all databases	Analytics platform
readWriteAnyDatabase	Read/write all databases	Migration tools

Role	Privileges	Use Case
userAdminAnyDatabase	Manage all users	Identity management

Custom Roles

```javascript
// Create application-specific role
use myApp
db.createRole({
  role: "orderProcessor",
  privileges: [
    {
      resource: { db: "myApp", collection: "orders" },
      actions: ["insert", "update", "find"]
    },
    {
      resource: { db: "myApp", collection: "inventory" },
      actions: ["find", "update"]
    },
    {
      resource: { db: "myApp", collection: "customers" },
      actions: ["find"]
    }
  ],
  roles: []  // Can inherit from other roles
})

// Role with inheritance
db.createRole({
  role: "customerService",
  privileges: [],
  roles: [
    { role: "read", db: "myApp" },
    { role: "read", db: "support" }
  ]
})

// Create user with custom role
db.createUser({
  user: "orderApp",
  pwd: "password123",
  roles: [ "orderProcessor" ]
})
```

Privilege Model

A privilege consists of a **resource** and a set of **actions**:

```javascript
// Resource patterns
```

```
{ db: "myApp", collection: "orders" }        // Specific collection
{ db: "myApp", collection: "" }               // All collections in database
{ db: "", collection: "" }                     // All databases
{ cluster: true }                              // Cluster-wide operations

// Actions
// Read: find, count, distinct
// Write: insert, update, delete, upsert
// Management: createCollection, dropCollection, createIndex, dropIndex
// Administration: killCursors, killOp, reIndex
// Database: dropDatabase, createUser, dropUser
```

Data Encryption - Deep Theory

Encryption in Transit (TLS/SSL)

Protects data as it moves between client and server, or between replica set members.

text

```
Without TLS (Plain Text):
Client              Server
  │                    │
  │──password123──────▶│  (Anyone can see!)
  │──INSERT user data─▶│
  │──credit card: 4111│
  │                    │

With TLS (Encrypted):
Client              Server
  │──TLS Handshake───▶│
  │   Encrypted        │
  │   Channel          │
  │   Established      │
  │                    │
  │──Encrypted data──▶│  (Only server can decrypt)
  │──Encrypted data──▶│
  │                    │
```

Configuration:

javascript

```
// Generate certificates
openssl req -newkey rsa:2048 -nodes -keyout mongodb.key -x509 -out mongodb.crt
cat mongodb.key mongodb.crt > mongodb.pem

// mongod.conf
```

```
net:
  ssl:
    mode: requireSSL
    PEMKeyFile: /etc/ssl/mongodb.pem
    CAFile: /etc/ssl/ca.pem

// Connection string
mongodb://server:27017/?ssl=true&sslCAFile=ca.pem
```

Encryption at Rest

Protects data files on disk. If someone steals the physical server, they cannot read the data.

```text
Without Encryption:
Disk: [Data Files] ← Anyone with disk access can read


With Encryption:
Disk: [Encrypted Data] ← Needs key to decrypt
           │
           │
    [Key Management System]
           │
           │
    Application ← Has key in memory
```

Configuration:

```javascript
// Generate master key
openssl rand -base64 32 > mongodb-keyfile
chmod 600 mongodb-keyfile

// mongod.conf
security:
  enableEncryption: true
  encryptionKeyFile: /etc/mongodb-keyfile
  encryptionCipherMode: AES256-CBC

// For KMIP integration (enterprise)
security:
  enableEncryption: true
  kmip:
    serverName: kmip.example.com
    port: 5696
    clientCertificateFile: /etc/ssl/kmip.pem
    serverCAFile: /etc/ssl/ca.pem
```

Client-Side Field Level Encryption (CSFLE)

Encrypts specific fields before sending to database. Even MongoDB cannot read these fields.

```text
Application                     MongoDB
    |                              |
    | Encrypt sensitive fields     |
    |   - SSN → "encrypted123"     |
    |   - CreditCard → "encrypt456"|
    |                              |
    |──Send document with encrypted|
    |  fields──────────────────────▶|
    |                              |
    |                              | Stores encrypted data
    |                              | Cannot decrypt without key
    |                              |
    |──Query───────────────────────▶|
    |                              |
    |<──Encrypted results──────────|
    |                              |
    | Decrypt fields with local key|
    |                              |
```

Implementation:

```javascript
// 1. Create data encryption key
const kms = {
  local: { key: localMasterKey }  // Or use AWS KMS, Azure Key Vault
};

const client = new MongoClient(uri, {
  autoEncryption: {
    kmsProviders: kms,
    keyVaultNamespace: "encryption.__keyVault",
    schemaMap: {
      "medical.patients": {
        bsonType: "object",
        properties: {
          ssn: {
            encrypt: {
              bsonType: "string",
              algorithm: "AEAD_AES_256_CBC_HMAC_SHA_512-Random"
            }
          },
```

```
          insuranceId: {
            encrypt: {
              bsonType: "string",
              algorithm: "AEAD_AES_256_CBC_HMAC_SHA_512-Deterministic"
            }
          }
        }
      }
    }
  }
});

// 2. Insert document (automatically encrypted)
await db.patients.insertOne({
  name: "John Doe",
  ssn: "123-45-6789",  // Automatically encrypted
  insuranceId: "INS789" // Automatically encrypted (deterministic)
});

// 3. Query (deterministic fields can be queried)
await db.patients.find({ insuranceId: "INS789" }); // Works!
await db.patients.find({ ssn: "123-45-6789" });    // Won't work (random encryptio
n)
```

Key Management

The biggest challenge in encryption is key management:

```text
Poor Key Management:
- Store key with data → Useless encryption
- Hard-code in application → Key exposure
- No key rotation → Long-term vulnerability

Good Key Management:
- Hardware Security Modules (HSM)
- Cloud KMS (AWS KMS, Azure Key Vault)
- Key rotation policies
- Separate key from encrypted data
- Access logging for key usage
```

Network Security - Deep Theory

Network Segmentation

```javascript
```

```
// Bind to specific interfaces
net:
  bindIp: 192.168.1.100,192.168.2.100  // Only internal networks
  port: 27017
  // Not bound to 0.0.0.0 (all interfaces)
```

Firewall Rules

text

```text
Allow only necessary traffic:

Production Network:
├── Application Servers (10.0.1.0/24) → MongoDB (10.0.2.0/24) port 27017
├── Admin Workstations (10.0.3.0/24) → MongoDB port 27017
└── Monitoring (10.0.4.0/24) → MongoDB port 27017

Block everything else:
- No internet access to MongoDB
- No SSH from outside
- No database ports exposed
```

IP Whitelisting

javascript

```javascript
// MongoDB Atlas IP whitelist
// Only allow specific IPs to connect
[
  { "ipAddress": "10.0.1.10", "comment": "App Server 1" },
  { "ipAddress": "10.0.1.11", "comment": "App Server 2" },
  { "ipAddress": "10.0.3.0/24", "comment": "Admin Network" }
]
```

Authentication Restriction

javascript

```javascript
// Restrict users to specific IPs
db.createUser({
  user: "appUser",
  pwd: "password123",
  roles: [ "readWrite" ],
  authenticationRestrictions: [
    { clientSource: ["10.0.1.0/24"] }  // Only from app network
  ]
});
```

Preventing Injection Attacks - Deep Theory

What is NoSQL Injection?

Unlike SQL injection, NoSQL injection exploits the query structure itself:

```javascript
// VULNERABLE CODE
const username = req.body.username;  // User input: { $ne: null }
const password = req.body.password;

// Direct interpolation into query
const user = db.users.findOne({
  username: username,
  password: password
});

// If username = { $ne: null }, password = { $ne: null }
// Query becomes: db.users.findOne({ username: { $ne: null }, password: { $ne: nul
l } })
// Returns FIRST user in database! Authentication bypassed!
```

Safe Practices

1. Use Parameterized Queries

```javascript
// GOOD: Use variables, not string building
const user = db.users.findOne({
  username: req.body.username,  // MongoDB driver handles safely
  password: req.body.password
});

// BAD: String concatenation
const query = "{ username: '" + username + "', password: '" + password + "' }";
// Never do this!
```

2. Validate Input Types

```javascript
// GOOD: Validate and sanitize
function validateLogin(username, password) {
  if (typeof username !== 'string' || typeof password !== 'string') {
    throw new Error('Invalid input types');
  }

  // Additional validation
  if (username.length > 100 || password.length > 100) {
    throw new Error('Input too long');
  }

  return db.users.findOne({ username, password });
}
```

3. Use Schema Validation

```javascript
// Database-level protection
db.createCollection("users", {
```

```
  validator: {
    $jsonSchema: {
      bsonType: "object",
      properties: {
        username: { bsonType: "string", maxLength: 100 },
        password: { bsonType: "string", maxLength: 100 }
      }
    }
  }
});
// Prevents injection objects from being inserted
```

4. Escape User Input

javascript

```
// For regex searches, escape special characters
function escapeRegex(text) {
  return text.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
}

db.products.find({
  name: { $regex: escapeRegex(req.body.search), $options: 'i' }
});
```

15. Backup & Restore

The Fundamental Theory of Backup

Why Backups Matter

Backups are your last line of defense against data loss. The **3-2-1 Rule** of backup:

text

```
3 Copies of your data
2 Different media types (disk, tape, cloud)
1 Off-site copy
```

Types of Backups

Backup Type	What it Captures	Recovery Time	Use Case
Physical	Raw data files	Fast	Full system recovery
Logical	Data as BSON/JSON	Slower	Selective restore

Backup Type	What it Captures	Recovery Time	Use Case
Snapshot	Point-in-time image	Fastest	Quick recovery
Incremental	Changes since last	Medium	Frequent backups
Continuous	Every operation	Point-in-time	Minimal data loss

Backup Strategy Decision Tree

text

```
How much data can you lose?
├── None → Continuous backup + point-in-time recovery
├── Minutes → Frequent incremental backups
└── Hours → Daily full backups

How fast must you recover?
├── Minutes → Snapshots + replica set
├── Hours → mongodump/mongorestore
└── Days → Occasional backups acceptable

What's your budget?
├── High → MongoDB Atlas with continuous backup
├── Medium → Ops Manager with snapshots
└── Low → mongodump to S3/cloud storage
```

mongodump - Deep Theory

What is mongodump?

mongodump is a utility that creates a binary export of your database's data. It's the **logical backup** tool for MongoDB.

How mongodump Works:

text

```
MongoDB Instance
    │
```

```
    ├── Collection 1
    │   └── Documents → Read → BSON → Write → file1.bson
    ├── Collection 2
    │   └── Documents → Read → BSON → Write → file2.bson
    └── Metadata
        └── Indexes → JSON → Write → collection.metadata.json

Output Directory:
/backup/
├── admin/
│   ├── system.users.bson
│   └── system.users.metadata.json
├── mydb/
│   ├── orders.bson
│   ├── orders.metadata.json
│   ├── users.bson
│   └── users.metadata.json
└── oplog.bson (if --oplog used)
```

Basic Usage

javascript

```javascript
// Full database backup
mongodump --uri="mongodb://user:pass@localhost:27017" --out=/backups/full/

// Single database
mongodump --db=myApp --out=/backups/myApp/

// Single collection
mongodump --db=myApp --collection=users --out=/backups/users/

// With authentication
mongodump --host=localhost --port=27017 \
  --username=backupUser --password=secret \
  --authenticationDatabase=admin \
  --out=/backups/daily/
```

Advanced Options

```javascript
// Backup with oplog (for point-in-time recovery)
mongodump --oplog --out=/backups/with-oplog/
```

```
// Compress backup (save space)
mongodump --archive=/backups/archive.gz --gzip

// Exclude specific collections
mongodump --db=myApp --excludeCollection=logs --excludeCollection=temp

// Query-based backup (only specific documents)
mongodump --db=myApp --collection=orders \
  --query='{ "status": "completed", "date": { "$gte": ISODate("2024-01-01") } }' \
  --out=/backups/orders-2024/

// Backup from secondary (reduce primary load)
mongodump --uri="mongodb://secondary:27017/?readPreference=secondary" \
  --out=/backups/from-secondary/
```

mongodump Performance Considerations

```javascript
// Throttle backup speed (don't overwhelm primary)
mongodump --forceTableScan --numParallelCollections=2

// Backup to remote location
mongodump --archive | ssh user@backup-server "cat > /backups/db.archive"

// Monitor progress (verbose)
mongodump --verbose --out=/backups/
```

---

mongorestore - Deep Theory

What is mongorestore?

mongorestore is the counterpart to mongodump, loading data from BSON files back into MongoDB.

How mongorestore Works:

```text
Backup Files
    │
    ├── mydb/users.bson → Read BSON → Parse → Insert documents
    ├── mydb/users.metadata.json → Read JSON → Create indexes
    └── mydb/orders.bson → Read BSON → Parse → Insert documents

MongoDB Instance
    ├── mydb.users (restored)
```

```
    ├── mydb.orders (restored)
    └── Indexes recreated
```

Basic Usage

```javascript
// Full restore
mongorestore --uri="mongodb://localhost:27017" /backups/full/

// Restore single database
mongorestore --db=myApp --drop /backups/myApp/

// Restore single collection
mongorestore --db=myApp --collection=users /backups/myApp/users.bson

// With authentication
mongorestore --host=localhost --port=27017 \
  --username=restoreUser --password=secret \
  --authenticationDatabase=admin \
  /backups/daily/
```

Advanced Options

```javascript
// Drop collections before restore (clean restore)
mongorestore --drop /backups/myApp/

// Restore with oplog replay
mongorestore --oplogReplay /backups/with-oplog/

// Restore from compressed archive
mongorestore --archive=/backups/archive.gz --gzip

// Restore without indexes (faster)
mongorestore --noIndexRestore /backups/myApp/

// Convert BSON to JSON during restore (debugging)
mongorestore --verbose --objcheck /backups/myApp/

// Restore to a different database
mongorestore --nsFrom="myApp.*" --nsTo="myApp_restore.*" /backups/myApp/
```

Restore Strategies

```javascript
// Scenario 1: Disaster recovery - full restore
mongorestore --drop /backups/latest/

// Scenario 2: Restore single user's data
mongodump --db=myApp --collection=users --query='{ "userId": "123" }' --out=/tmp/u
ser123/
mongorestore --drop /tmp/user123/

// Scenario 3: Partial restore with aggregation
```

```
// 1. Restore to temporary database
mongorestore --db=temp_restore /backups/myApp/

// 2. Copy needed data
db.temp_restore.orders.aggregate([
  { $match: { date: { $gt: new Date("2024-01-01") } } },
  { $out: "myApp.orders_2024" }
]);

// 3. Drop temporary database
db.getSiblingDB("temp_restore").dropDatabase();
```

Cloud Backups - Deep Theory

MongoDB Atlas Backups

Atlas provides automated, continuous backups with point-in-time recovery:

text

```
Continuous Backup Architecture:


Primary Node
    │
    ├── Oplog stream → Cloud storage
    │                    │
    │                    ├── Snapshots (every 6-24 hours)
    │                    ├── Oplog (continuous)
    │                    └── Indexes (for fast restore)
    │
    └── Checkpoint every 100 writes
```

Backup Tiers:

Tier	Retention	Recovery Point	Use Case
Local	Up to 7 days	Last 24 hours continuous	Dev/test
Cloud Provider	Up to 35 days	Point-in-time	Production
Continuous	Up to 30 days	Any point in last 30 days	Critical systems

Atlas Backup Configuration:

```javascript
// Via Atlas UI or API
{
  "clusterName": "myCluster",
  "backupEnabled": true,
  "pitEnabled": true,           // Point-in-time recovery
  "retentionDays": 7,            // Snapshot retention
  "frequencyHours": 6,           // Snapshot frequency
  "copySettings": [
    {
      "regionName": "us-east-1",
      "shouldCopyIndexes": true
    }
  ]
}
```

Ops Manager Backups

For self-managed deployments, Ops Manager provides enterprise-grade backup:

```javascript
// Ops Manager Backup Configuration
{
  "backupAgent": {
    "enabled": true,
    "oplogStore": "/data/oplog",
    "snapshotStore": "/data/snapshots"
  },
  "schedules": [
    {
      "frequency": "HOURLY",
      "retention": 24  // Keep 24 hourly snapshots
    },
    {
      "frequency": "DAILY",
      "retention": 30  // Keep 30 daily snapshots
    },
    {
      "frequency": "WEEKLY",
      "retention": 52  // Keep 52 weekly snapshots
    }
  ]
}
```

Cloud Storage Integration

```javascript
// Backup directly to S3
mongodump --archive | aws s3 cp - s3://my-bucket/backups/$(date +%Y%m%d).archive

// Scheduled backup with lifecycle
0 2 * * * mongodump --archive --gzip | \
```

```
  aws s3 cp - s3://my-bucket/backups/$(date +\%Y\%m\%d).archive.gz

// S3 lifecycle policy (auto-delete old backups)
{
  "Rules": [
    {
      "Id": "Delete old backups",
      "Status": "Enabled",
      "Prefix": "backups/",
      "Expiration": { "Days": 30 }
    }
  ]
}
```

Cross-Region Replication

```javascript
// Backup to multiple regions for disaster recovery
mongodump --archive | \
  tee >(aws s3 cp - s3://us-east-1-bucket/backup.gz) \
      >(aws s3 cp - s3://us-west-2-bucket/backup.gz)

// Or use Atlas cross-region snapshot copy
{
  "copySettings": [
    { "regionName": "us-east-1", "cloudProvider": "AWS" },
    { "regionName": "us-west-2", "cloudProvider": "AWS" },
    { "regionName": "eu-west-1", "cloudProvider": "AWS" }
  ]
}
```

Point-in-Time Recovery - Deep Theory

What is Point-in-Time Recovery?

Point-in-Time Recovery (PITR) allows you to restore a database to any moment in time, not just when backups were taken.

The Problem PITR Solves:

```text
Timeline:
2 AM: Full backup
3:15 PM: Accidental DROP DATABASE
3:16 PM: You notice

Without PITR:
- Restore 2 AM backup
```

```
- Lose 3:15 PM to 2 AM (13 hours of data!)

With PITR:
- Restore to 3:14:59
- Lose 1 second of data
```

How PITR Works

PITR combines **full backups** with **oplog replay**:

text

```text
Full Backup (2 AM)    Oplog (2 AM - 3:14 PM)    Restore Point (3:14:59 PM)
     │                        │                            │
     └────────────────────────┘                            │
               │                                            │
               ↓                                            │
     Restore full backup                                    │
               │                                            │
               ├──────────────┬────────────────┬────────────┘
               ↓              ↓                ↓
        Apply oplog     Apply oplog     Apply oplog
        up to 10 AM     up to 2 PM      up to 3:14:59
```

Implementing PITR

1. With Ops Manager/Atlas:

javascript

```javascript
// Atlas UI: Click "Restore" → Select point in time
// Or via API
curl -u "username:apiKey" --digest \
  --header "Accept: application/json" \
  --header "Content-Type: application/json" \
  --request POST "https://cloud.mongodb.com/api/atlas/v1.0/groups/GROUP_ID/cluster
s/CLUSTER_NAME/restoreJobs" \
  --data '{
    "snapshotId": "none",
    "pointInTimeUTCSeconds": 1705320899,
    "deliveryType": "download"
  }'
```

2. Self-Managed with Oplog:

javascript

```javascript
// Step 1: Restore last full backup
mongorestore --drop /backups/latest/

// Step 2: Find oplog position for target time
use local
db.oplog.rs.find({
```

```
  "ts": { $lt: Timestamp(targetTime, 0) }
}).sort({$natural: -1}).limit(1)

// Step 3: Replay oplog to that point
mongorestore --oplogReplay \
  --oplogFile /backups/oplog.bson \
  --oplogLimit "timestamp:increment"
```

3. Manual PITR Script:

```javascript
#!/bin/bash
# pitr-restore.sh

TARGET_TIME=$1  # e.g., "2024-01-15 15:14:59"
BACKUP_DIR="/backups/latest"
OPLOG_DIR="/backups/oplog"

# Restore full backup
mongorestore --drop $BACKUP_DIR

# Find oplog timestamp
TIMESTAMP=$(mongo --quiet --eval "
  var target = ISODate('$TARGET_TIME');
  var ts = db.getSiblingDB('local').oplog.rs.find(
    { ts: { \$lt: Timestamp(target.getTime()/1000, 0) } }
  ).sort({ts: -1}).limit(1).next().ts;
  print(ts.t + ',' + ts.i);
")

# Replay oplog
mongorestore --oplogReplay --oplogFile $OPLOG_DIR/oplog.bson \
  --oplogLimit "${TIMESTAMP%%,*}:${TIMESTAMP#*,}"
```

PITR Requirements and Limitations

```javascript
// Requirements for PITR:
// 1. Must have oplog covering the period (oplog size matters!)
// 2. Full backups must be taken regularly
// 3. Oplog must be continuous (no gaps)

// Calculate required oplog size for PITR:
oplogSizeGB = (peak writes per second * seconds in retention period) / (1000 * 100
0 * 1000) * 1.5

// Example:
// Peak writes: 10,000/second
// Retention: 24 hours (86,400 seconds)
// Writes total: 864M operations
// Each operation ~500 bytes
// Total: ~432GB
// Recommended oplog: 650GB
```

Summary: The Complete Mental Model

Security

Think of security as **protecting a fortress**:

- **Authentication** is the gatekeeper - verifying identity before entry
- **Authorization** is the key system - determining which doors you can open
- **Encryption** is the vault - protecting valuables even if walls are breached
- **Network security** is the moat - controlling who can approach
- **Injection prevention** is training guards - recognizing trickery

Backup & Restore

Think of backup as **insurance policies**:

- **mongodump** is taking photos of your valuables (logical backup)
- **Physical backup** is having a duplicate vault (file copy)
- **PITR** is time travel insurance - can go back to any moment
- **Cloud backups** are off-site storage - safe from local disasters
- **Restore testing** is reading the fine print - know what's covered

The Golden Rules

For Security:

1. **Defense in depth** - multiple layers of protection
2. **Least privilege** - give minimum necessary access
3. **Encrypt everything** - in transit, at rest, sensitive fields
4. **Never trust input** - validate, sanitize, escape
5. **Audit everything** - you can't protect what you can't see

For Backup & Restore:

1. **Test your restores** - a backup you can't restore is useless
2. **Follow 3-2-1 rule** - 3 copies, 2 media, 1 off-site
3. **Automate everything** - manual backups are forgotten backups
4. **Monitor backup success** - failed backups are invisible data loss
5. **Plan for point-in-time** - yesterday's backup may not be enough

MongoDB Tools & Backend Integration: Complete Theory Notes for Senior Developers

---

16. MongoDB Tools & Ecosystem

MongoDB Compass (GUI) - Deep Theory

What is MongoDB Compass?

MongoDB Compass is the official graphical user interface for MongoDB. It transforms database interaction from command-line operations to visual, intuitive exploration.

The Philosophy Behind Compass:

```text
Traditional Approach:                    Compass Approach:
mongosh > use mydb                       Visual database explorer
mongosh > db.users.find({age: {$gt: 25}})  Point, click, filter
mongosh > db.users.createIndex({email:1})  GUI index management
mongosh > db.users.aggregate([...])       Visual aggregation builder
```

Compass isn't just a GUI—it's a **visualization and optimization tool** that helps you understand your data and queries.

Compass Architecture

```text
┌──────────────────────────────────────────────────────┐
│                  Compass Application                  │
├──────────────────────────────────────────────────────┤
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  │
│  │  Schema      │  │  Explain     │  │  Index       │  │
│  │  Analyzer    │  │  Planner     │  │  Manager     │  │
│  └──────────────┘  └──────────────┘  └──────────────┘  │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  │
│  │  Aggregation│  │  Document    │  │  Performance│  │
```

```
│  │  Builder    │  │  Viewer     │  │  Advisor    │  │
│  └─────────────┘  └─────────────┘  └─────────────┘  │
├──────────────────────────────────────────────────────┤
│                MongoDB Driver Connection              │
├──────────────────────────────────────────────────────┤
│                 MongoDB Server/Cluster                │
└──────────────────────────────────────────────────────┘
```

Key Features and Their Uses

1. Schema Visualization

```javascript
// What Compass Shows:
// - Field distributions
// - Data types
// - Missing fields percentage
// - Common values

// Use case: Understanding data quality
// - Identify inconsistent fields
// - Plan schema migrations
// - Discover hidden patterns
```

2. Explain Plan Visualizer

```javascript
// Visual representation of query execution:
// - Which indexes used
// - Document examined count
// - Execution time
// - Query stages

// Use case: Query optimization
// - Spot full collection scans
// - Verify index usage
// - Compare query performance
```

3. Aggregation Pipeline Builder

```javascript
// Drag-and-drop pipeline stages:
```

```
// $match → $group → $sort → $project

// Real-time preview of results
// Stage-by-stage document transformation
// Export pipeline to code (Node.js, Python, Java)

// Use case: Building complex aggregations
// - Experiment visually
// - Understand data flow
// - Generate production code
```

4. Performance Advisor (Atlas)

```javascript
// Automatic recommendations:
// - Missing indexes
// - Slow queries
// - Schema improvements

// Use case: Proactive optimization
// - Fix issues before users notice
// - Learn best practices
// - Automate index creation
```

5. Real-Time Performance Metrics

```javascript
// Visual dashboard showing:
// - Operations per second
// - Latency percentiles
// - Connection counts
// - Memory usage
// - Disk I/O

// Use case: Production monitoring
// - Spot anomalies quickly
// - Understand workload patterns
// - Capacity planning
```

When to Use Compass vs Command Line

Task	Compass	mongosh	Reason
Data Exploration	✅ Best	⚠️ OK	Visual browsing faster
Query Optimization	✅ Best	⚠️ OK	Visual explain plans
Schema Analysis	✅ Best	❌ Hard	Automatic field analysis
Ad-hoc Queries	⚠️ OK	✅ Best	Type faster than click
Scripting	❌ No	✅ Best	Automation needs code
Bulk Operations	❌ No	✅ Best	Scripting essential
Production Debug	⚠️ OK	✅ Best	SSH access often limited

Mongo Shell (mongosh) - Deep Theory

What is mongosh?

mongosh is the modern command-line interface for MongoDB. It's a fully-featured JavaScript REPL (Read-Eval-Print Loop) environment for interacting with MongoDB.

The Evolution:

```text
mongo (legacy) → mongosh (modern)
- Basic JavaScript      - Full ES6+ support
- Limited helpers       - Advanced helpers
- No syntax highlighting - Syntax highlighting
- Basic authentication  - Modern auth methods
```

mongosh Architecture

```text
┌─────────────────────────────────────────┐
│              mongosh Application         │
├─────────────────────────────────────────┤
│  ┌──────────────────────────────────┐   │
│  │      JavaScript Runtime (Node.js)     │   │
│  └──────────────────────────────────┘   │
│  ┌──────────────────────────────────┐   │
│  │      MongoDB Node.js Driver           │   │
│  └──────────────────────────────────┘   │
├─────────────────────────────────────────┤
│              MongoDB Connection          │
└─────────────────────────────────────────┘
```

Key Features and Techniques

1. Advanced Scripting

```javascript
// mongosh supports full JavaScript
const databaseName = 'myApp';
const backupDate = new Date().toISOString().split('T')[0];

// Conditional logic
if (db.getMongo().isMaster().ismaster) {
  print('Connected to primary');
}

// Functions
function analyzeCollection(collectionName) {
  const stats = db[collectionName].stats();
  const count = db[collectionName].countDocuments();

  print(`Collection: ${collectionName}`);
  print(`  Documents: ${count}`);
  print(`  Size: ${stats.size} bytes`);
  print(`  Indexes: ${stats.nindexes}`);
}

// Loops
['users', 'orders', 'products'].forEach(analyzeCollection);
```

2. Built-in Helpers

```javascript
// Show databases
show dbs
show databases

// Switch database
```

```
use myApp

// Show collections
show collections
show tables

// Show users
show users

// Show roles
show roles

// Profile queries
db.setProfilingLevel(1, 50)  // Log queries >50ms
show profile  // Show slow queries
```

3. Working with Cursors

```javascript
// Iterate with JavaScript
const cursor = db.users.find({ active: true });

// Convert to array (careful with large results)
const users = cursor.toArray();

// Process in batches
let batch = [];
while (cursor.hasNext()) {
  batch.push(cursor.next());
  if (batch.length === 100) {
    processBatch(batch);
    batch = [];
  }
}
if (batch.length) processBatch(batch);

// Pretty print
db.users.find().limit(5).pretty()
```

4. Configuration and Customization

```javascript
// .mongoshrc.js file (in home directory)
config.set('displayBatchSize', 20);
config.set('banner', false);
config.set('inspectDepth', 10);

// Custom prompt
prompt = () => {
  const isMaster = db.isMaster().ismaster ? 'PRIMARY' : 'SECONDARY';
  return `mongodb [${isMaster}] > `;
};

// Custom commands
function findSlowQueries() {
  return db.system.profile.find({
```

```
    millis: { $gt: 100 }
  }).sort({ ts: -1 }).limit(20);
}
```

5. Working with Files

```javascript
// Load external scripts
load('/scripts/migration.js');

// Run JavaScript file
mongosh myApp /scripts/backup.js

// Output to file
db.users.find().toArray() > /tmp/users.json

// Import from file
var users = cat('/tmp/users.json');
eval(users);
```

6. Connection Management

```javascript
// Multiple connections
const conn1 = connect('mongodb://server1:27017');
const conn2 = connect('mongodb://server2:27017');

conn1.getDB('myApp').users.find();
conn2.getDB('myApp').orders.find();

// Connection options
const conn = connect('mongodb://user:pass@localhost:27017/?replicaSet=rs0');

// Check connection status
db.getMongo().isMaster();
db.getMongo().getReplicationInfo();
```

mongosh Best Practices

```javascript
// 1. Always use variables for reuse
const dbName = 'production';
const coll = db.getSiblingDB(dbName).users;

// 2. Validate before destructive operations
const count = db.orders.countDocuments({ status: 'temp' });
print(`About to delete ${count} temporary orders`);
if (count < 10000) {
  db.orders.deleteMany({ status: 'temp' });
} else {
  print('Too many to delete, investigate first');
}

// 3. Use transactions for multi-document operations
const session = db.getMongo().startSession();
```

```
session.startTransaction();
try {
  session.getDatabase('bank').accounts.updateOne(
    { _id: 1 },
    { $inc: { balance: -100 } }
  );
  session.getDatabase('bank').accounts.updateOne(
    { _id: 2 },
    { $inc: { balance: 100 } }
  );
  session.commitTransaction();
} catch (e) {
  session.abortTransaction();
} finally {
  session.endSession();
}
```

Atlas (Cloud Platform) - Deep Theory

What is MongoDB Atlas?

Atlas is MongoDB's Database as a Service (DBaaS) platform. It's not just "MongoDB in the cloud"—it's a fully managed platform with automation, monitoring, and advanced features.

The Atlas Value Proposition:

```text
Self-Managed:                          Atlas:
- Provision servers                    - Click to deploy
- Configure replication                - Automatic replication
- Manage backups                       - Continuous backups
- Monitor performance                   - Built-in monitoring
- Apply security patches               - Automatic updates
- Scale when needed                    - Auto-scaling options
- Handle failover                      - Automatic failover
```

Atlas Architecture

```text
┌──────────────────────────────────────────────────────────────┐
│                      Atlas Control Plane                      │
│  (UI, API, Automation, Monitoring, Billing, Alerts)           │
```

```
├──────────────────────────────────────────────────────────────┤
│  ┌──────────────────────────────────────────────────────┐   │
│  │                  Cloud Provider Layer                  │  │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────┐  │  │
│  │  │   AWS    │  │   Azure  │  │   GCP    │  │Others│  │  │
│  │  └──────────┘  └──────────┘  └──────────┘  └──────┘  │  │
│  └──────────────────────────────────────────────────────┘   │
├──────────────────────────────────────────────────────────────┤
│  ┌──────────────────────────────────────────────────────┐   │
│  │                  MongoDB Deployment                    │  │
│  │  ┌────────────┐  ┌────────────┐  ┌────────────┐    │  │
│  │  │   Replica  │  │   Sharded  │  │   Serverless│   │  │
│  │  │     Set    │  │   Cluster  │  │            │    │  │
│  │  └────────────┘  └────────────┘  └────────────┘    │  │
│  └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
```

Atlas Features Deep Dive

1. Deployment Options

```javascript
// Replica Set (3 nodes)
{
  "clusterType": "REPLICASET",
  "providerSettings": {
    "providerName": "AWS",
    "regionName": "us-east-1",
    "instanceSizeName": "M10"
  }
}

// Sharded Cluster
{
  "clusterType": "SHARDED",
  "shards": [
    { "providerSettings": { "instanceSizeName": "M30" } },
    { "providerSettings": { "instanceSizeName": "M30" } }
  ],
  "mongos": { "instanceSizeName": "M30" }
}

// Serverless
{
  "serverlessSpec": {
    "continuousBackupEnabled": true,
    "providerSettings": {
      "providerName": "SERVERLESS",
      "regionName": "us-east-1"
    }
```

```
  }
}
```

2. Global Clusters

```javascript
// Distribute data worldwide
{
  "globalCluster": true,
  "customZoneMappings": [
    {
      "zone": "US",
      "location": "NORTH_AMERICA"
    },
    {
      "zone": "EU",
      "location": "EUROPE"
    }
  ]
}

// Zone-specific sharding
sh.addShardTag("shard01", "US")
sh.addTagRange(
  "db.users",
  { region: "US" },
  { region: "US" },
  "US"
)
```

3. Performance Optimization

```javascript
// Auto-scaling configuration
{
  "autoScaling": {
    "diskGBEnabled": true,  // Auto-expand storage
    "compute": {
      "enabled": true,      // Auto-scale CPU/RAM
      "scaleDownEnabled": true
    }
  }
}

// Performance Advisor
// - Identifies slow queries
// - Recommends indexes
// - One-click index creation

// Query Profiler
// - Visual query analysis
// - Execution stats
// - Index usage patterns
```

4. Security Features

```javascript
// Network security
{
  "ipAccessList": [
    { "ipAddress": "10.0.1.0/24", "comment": "App Servers" },
    { "ipAddress": "192.168.1.100", "comment": "Admin" }
  ]
}

// Encryption at rest (always enabled)
// Encryption in transit (TLS/SSL always enabled)

// Private networking
{
  "privateLink": {
    "enabled": true,
    "endpointServiceName": "com.amazonaws.vpce.us-east-1.vpce-svc-12345"
  }
}

// Database auditing
{
  "auditLog": {
    "enabled": true,
    "filter": "{ atype: { $in: ['authCheck', 'createUser'] } }"
  }
}
```

5. Backup and Recovery

```javascript
// Continuous backup (point-in-time recovery)
{
  "backupEnabled": true,
  "pitEnabled": true,
  "retentionDays": 7
}

// Cloud provider snapshots
{
  "snapshotBackupPolicies": [
    {
      "frequencyType": "hourly",
      "retentionValue": 24
    },
    {
      "frequencyType": "daily",
      "retentionValue": 30
    }
  ]
}

// Cross-region snapshot copy
{
  "copySettings": [
    {
      "regionName": "us-west-2",
```

```
      "cloudProvider": "AWS"
    }
  ]
}
```

6. Monitoring and Alerts

javascript

```javascript
// Alert configuration
{
  "alerts": [
    {
      "type": "REPLICATION_LAG",
      "threshold": 60,  // Alert if lag > 60 seconds
      "notification": "email"
    },
    {
      "type": "CPU_UTILIZATION",
      "threshold": 80,  // Alert if CPU > 80%
      "notification": "slack"
    }
  ]
}

// Metrics available:
// - Operations per second
// - Connection count
// - Memory usage
// - Disk latency
// - Replication lag
// - Query execution time
```

17. Backend Integration (Full Stack MUST)

Node.js Integration - Deep Theory

The MongoDB Node.js Driver

The MongoDB Node.js driver is the official library that allows Node.js applications to interact with MongoDB. It's a **native JavaScript driver** that implements the MongoDB wire protocol.

Driver Architecture:

text

```text
┌─────────────────────────────────────────────┐
│            Node.js Application              │
├─────────────────────────────────────────────┤
```

```
│  ┌──────────────────────────────────────┐  │
│  │         MongoDB Node.js Driver          │  │
│  ├──────────────────────────────────────┤  │
│  │  ┌─────────┐  ┌─────────┐  ┌────────┐ │  │
│  │  │ Topology│  │ Connection│  │ Cursor │ │  │
│  │  │ Manager │  │   Pool   │  │ Manager│ │  │
│  │  └─────────┘  └─────────┘  └────────┘ │  │
│  │  ┌─────────┐  ┌─────────┐  ┌────────┐ │  │
│  │  │ BSON    │  │  GridFS │  │ Sessions│ │  │
│  │  │ Parser  │  │  Stream │  │ Manager│ │  │
│  │  └─────────┘  └─────────┘  └────────┘ │  │
│  └──────────────────────────────────────┘  │
├──────────────────────────────────────────┤
│           TCP/IP Network Stack              │
└──────────────────────────────────────────┘
```

Connecting to MongoDB

Basic Connection:

```javascript
const { MongoClient } = require('mongodb');

// Connection URI
const uri = 'mongodb://localhost:27017';
const client = new MongoClient(uri);

async function connect() {
  try {
    await client.connect();
    console.log('Connected to MongoDB');

    const db = client.db('myApp');
    const collection = db.collection('users');

    // Perform operations
    const user = await collection.findOne({ email: 'user@example.com' });

    return user;
  } catch (error) {
    console.error('Connection failed:', error);
    throw error;
  }
}
```

Connection Options:

```javascript
const client = new MongoClient(uri, {
```

```
  // Connection pool settings
  maxPoolSize: 100,           // Maximum connections in pool
  minPoolSize: 10,            // Minimum connections to maintain
  maxIdleTimeMS: 30000,       // Close idle connections after 30s
  waitQueueTimeoutMS: 5000,   // Fail if no connection available

  // Write concern
  w: 'majority',              // Wait for majority acknowledgment
  wtimeoutMS: 5000,           // Write timeout
  journal: true,              // Wait for journal

  // Read preference
  readPreference: 'secondaryPreferred',  // Read from secondaries if available

  // Retry logic
  retryWrites: true,          // Automatically retry failed writes
  retryReads: true,           // Automatically retry failed reads

  // SSL/TLS
  tls: true,
  tlsCAFile: '/path/to/ca.pem',

  // Authentication
  authSource: 'admin',
  authMechanism: 'SCRAM-SHA-256'
});
```

Connection Pooling - Deep Theory

What is Connection Pooling?

Connection pooling is the practice of maintaining a cache of database connections that can be reused across requests. It's **critical for performance** in web applications.

The Problem Connection Pooling Solves:

```text
Without Pooling:
Request 1: Open connection → Use → Close connection
Request 2: Open connection → Use → Close connection
Request 3: Open connection → Use → Close connection

Each request: TCP handshake + authentication (10-100ms overhead)

With Pooling:
[Pool: 10 connections ready]
Request 1: Take from pool → Use → Return to pool
Request 2: Take from pool → Use → Return to pool
```

```
Request 3: Take from pool → Use → Return to pool

No connection overhead per request!
```

Pool Architecture

```text
┌──────────────────────────────────────────┐
│            Connection Pool                  │
│  ┌─────┐ ┌─────┐ ┌─────┐ ┌─────┐          │
│  │Conn1│ │Conn2│ │Conn3│ │Conn4│ ... (max)  │
│  └─────┘ └─────┘ └─────┘ └─────┘          │
│      ↑       ↑       ↑       ↑              │
└──────┼───────┼───────┼───────┼──────────────┘
       │       │       │       │
    ┌──┴──┐ ┌──┴──┐ ┌──┴──┐ ┌──┴──┐
    │Req 1│ │Req 2│ │Req 3│ │Req 4│
    └─────┘ └─────┘ └─────┘ └─────┘
```

Pool Lifecycle

```javascript
// 1. Pool Initialization
// When client connects, minPoolSize connections created

// 2. Connection Acquisition
async function handleRequest(req, res) {
  // Gets connection from pool (waits if none available)
  const result = await db.collection('users').findOne({ _id: req.userId });
  // Connection automatically returned to pool
  res.json(result);
}

// 3. Connection Maintenance
// - Pool checks connection health
// - Replaces dead connections
// - Closes idle connections (maxIdleTimeMS)
// - Maintains minPoolSize

// 4. Pool Exhaustion
// If all connections busy and pool at maxPoolSize:
// - New requests wait in queue
```

```
// - If waitQueueTimeoutMS exceeded, throws error
```

Pool Sizing Theory

```javascript
// Calculate optimal pool size

const CONCURRENT_REQUESTS = 100;        // Peak concurrent users
const AVG_QUERY_TIME_MS = 50;           // Average query duration
const REQUEST_RATE = 1000;               // Requests per second

// Little's Law: Connections needed = Request rate × Average response time
const connectionsNeeded = (REQUEST_RATE * AVG_QUERY_TIME_MS) / 1000;

// Rule of thumb formulas:
// - CPU-bound app: pool size = number of CPU cores × 2
// - I/O-bound app: pool size = higher (queries wait on network)

// MongoDB recommendations:
// - Start with 10-50 connections
// - Monitor connection usage
// - Increase if you see waitQueueTimeouts
```

Pool Monitoring

```javascript
// Monitor pool status
const status = client.topology?.connections;
console.log({
  active: status?.active,
  available: status?.available,
  total: status?.total,
  exhausted: status?.exhausted
});

// Track pool events
client.on('connectionPoolCreated', (event) => {
  console.log('Pool created:', event);
});

client.on('connectionCheckOutStarted', (event) => {
  console.log('Getting connection from pool');
});

client.on('connectionCheckedOut', (event) => {
  console.log('Connection acquired');
});

client.on('connectionCheckedIn', (event) => {
  console.log('Connection returned to pool');
});
```

Error Handling - Deep Theory

Error Types in MongoDB

```javascript
// 1. Connection Errors
try {
  await client.connect();
} catch (error) {
  if (error.name === 'MongoNetworkError') {
    // Network issue, server unreachable
    logger.error('Database network error:', error);
    // Implement retry with backoff
  }
}

// 2. Write Errors
try {
  await collection.insertOne(doc);
} catch (error) {
  if (error.code === 11000) {
    // Duplicate key error
    logger.warn('Duplicate key:', error.keyValue);
    return { error: 'Record already exists' };
  } else if (error.code === 121) {
    // Document validation failed
    logger.warn('Validation failed:', error.errInfo);
    return { error: 'Invalid document structure' };
  }
}

// 3. Read Errors
try {
  await collection.findOne({ _id: id });
} catch (error) {
  if (error.code === 2) {
    // Bad value (invalid ObjectId)
    logger.warn('Invalid ID format:', id);
    return { error: 'Invalid ID format' };
  }
}

// 4. Transaction Errors
const session = client.startSession();
try {
  session.startTransaction();
  // ... operations
  await session.commitTransaction();
} catch (error) {
  await session.abortTransaction();

  if (error.errorLabels?.includes('TransientTransactionError')) {
    // Transient error, can retry
    logger.info('Transient transaction error, retrying');
  } else if (error.errorLabels?.includes('UnknownTransactionCommitResult')) {
    // Commit result unknown, check status
    logger.warn('Unknown commit result');
```

```
  }
}
```

Retry Strategies

javascript

```javascript
// Exponential backoff retry
async function withRetry(operation, maxRetries = 3) {
  let lastError;

  for (let i = 0; i < maxRetries; i++) {
    try {
      return await operation();
    } catch (error) {
      lastError = error;

      // Don't retry certain errors
      if (error.code === 11000) {  // Duplicate key
        throw error;
      }

      // Wait with exponential backoff
      const waitTime = Math.pow(2, i) * 100;  // 100ms, 200ms, 400ms
      logger.warn(`Operation failed, retry ${i + 1}/${maxRetries} in ${waitTime}ms
`);
      await new Promise(resolve => setTimeout(resolve, waitTime));
    }
  }

  throw lastError;
}

// Usage
const user = await withRetry(() =>
  collection.findOne({ email: 'user@example.com' })
);
```

Graceful Error Handling Pattern

javascript

```javascript
class DatabaseService {
  constructor() {
    this.client = null;
    this.isConnected = false;
    this.reconnecting = false;
  }

  async connect() {
    try {
      this.client = new MongoClient(uri);
      await this.client.connect();
      this.isConnected = true;
      this.reconnecting = false;

      // Monitor connection
      this.client.on('connectionClosed', () => {
```

```
        this.isConnected = false;
        this.handleDisconnect();
      });

    } catch (error) {
      logger.error('Initial connection failed:', error);
      this.isConnected = false;
      this.scheduleReconnect();
    }
  }

  async handleDisconnect() {
    if (this.reconnecting) return;
    this.reconnecting = true;
    logger.warn('Database disconnected, attempting reconnect');

    await this.scheduleReconnect();
  }

  async scheduleReconnect() {
    const maxAttempts = 10;
    const baseDelay = 1000;

    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
      try {
        await this.client.connect();
        this.isConnected = true;
        this.reconnecting = false;
        logger.info('Reconnected to database');
        return;
      } catch (error) {
        const delay = baseDelay * Math.pow(2, attempt - 1);
        logger.warn(`Reconnect attempt ${attempt} failed, retrying in ${delay}ms`)
;
        await new Promise(resolve => setTimeout(resolve, delay));
      }
    }

    logger.error('Failed to reconnect after maximum attempts');
    // Notify monitoring system
  }

  async findOne(collection, query) {
    if (!this.isConnected) {
      throw new Error('Database not connected');
    }

    try {
      return await this.client.db().collection(collection).findOne(query);
    } catch (error) {
      if (error.name === 'MongoNetworkError') {
        this.handleDisconnect();
      }
      throw error;
    }
  }
}
```

Transactions in Code - Deep Theory

Transaction Patterns

```javascript
// Pattern 1: Session with Callback
async function transferFunds(fromId, toId, amount) {
  const session = client.startSession();

  try {
    const result = await session.withTransaction(async () => {
      const accounts = client.db('bank').collection('accounts');

      // Check balance
      const fromAccount = await accounts.findOne(
        { _id: fromId, balance: { $gte: amount } },
        { session }
      );

      if (!fromAccount) {
        throw new Error('Insufficient funds');
      }

      // Perform transfer
      await accounts.updateOne(
        { _id: fromId },
        { $inc: { balance: -amount } },
        { session }
      );

      await accounts.updateOne(
        { _id: toId },
        { $inc: { balance: amount } },
        { session }
      );

      // Record transaction
      await client.db('bank').collection('transactions').insertOne({
        fromId,
        toId,
        amount,
        timestamp: new Date()
      }, { session });
    });

    return { success: true, result };
  } catch (error) {
    logger.error('Transaction failed:', error);
    return { success: false, error: error.message };
  } finally {
    await session.endSession();
  }
}
```

Pattern 2: Manual Transaction Control

```javascript
async function processOrder(orderData) {
  const session = client.startSession();

  try {
    session.startTransaction({
      readConcern: { level: 'snapshot' },
      writeConcern: { w: 'majority' },
      readPreference: 'primary'
    });

    // Reserve inventory
    const inventory = client.db('store').collection('inventory');
    for (const item of orderData.items) {
      const result = await inventory.updateOne(
        {
          productId: item.productId,
          quantity: { $gte: item.quantity }
        },
        { $inc: { quantity: -item.quantity } },
        { session }
      );

      if (result.modifiedCount === 0) {
        throw new Error(`Insufficient inventory for ${item.productId}`);
      }
    }

    // Create order
    const orders = client.db('store').collection('orders');
    await orders.insertOne({
      ...orderData,
      status: 'confirmed',
      createdAt: new Date()
    }, { session });

    // Update customer stats
    const customers = client.db('store').collection('customers');
    await customers.updateOne(
      { _id: orderData.customerId },
      {
        $inc: { orderCount: 1, totalSpent: orderData.total },
        $push: { recentOrders: orderData.orderId }
      },
      { session }
    );

    await session.commitTransaction();
    return { success: true };

  } catch (error) {
    await session.abortTransaction();
    logger.error('Order processing failed:', error);
    return { success: false, error: error.message };
  } finally {
    await session.endSession();
```

```
  }
}
```

Pattern 3: Transaction Retry Logic

```javascript
async function withTransactionRetry(fn, maxRetries = 3) {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    const session = client.startSession();

    try {
      session.startTransaction();
      const result = await fn(session);
      await session.commitTransaction();
      return result;
    } catch (error) {
      await session.abortTransaction();

      // Check if error is transient and retryable
      const isTransient = error.errorLabels?.includes('TransientTransactionError')
;
      const isCommitUnknown = error.errorLabels?.includes('UnknownTransactionCommi
tResult');

      if ((isTransient || isCommitUnknown) && attempt < maxRetries) {
        logger.info(`Transient error, retrying (${attempt}/${maxRetries})`);
        // Exponential backoff
        await new Promise(resolve =>
          setTimeout(resolve, Math.pow(2, attempt) * 100)
        );
        continue;
      }

      throw error;
    } finally {
      await session.endSession();
    }
  }
}

// Usage
await withTransactionRetry(async (session) => {
  await collection1.updateOne({ _id: 1 }, { $inc: { count: 1 } }, { session });
  await collection2.updateOne({ _id: 2 }, { $inc: { count: 1 } }, { session });
});
```

Popular ODM: Mongoose

What is Mongoose?

Mongoose is an Object Document Mapper (ODM) for MongoDB and Node.js. It provides a **schema-based solution** to model your application data with built-in type casting, validation, query building, and business logic hooks.

The Problem Mongoose Solves:

```text
Raw Driver:                              Mongoose:
- No structure enforcement               - Schema validation
- Manual validation                       - Automatic validation
- No relationship mapping                 - Population (references)
- No middleware                            - Pre/post hooks
- Raw JavaScript objects                  - Model instances
- Manual type conversion                   - Automatic casting
```

Mongoose Architecture:

```text
┌──────────────────────────────────────────────┐
│           Node.js Application                 │
├──────────────────────────────────────────────┤
│  ┌──────────────────────────────────────┐   │
│  │              Mongoose ODM              │  │
│  ├──────────────────────────────────────┤   │
│  │  ┌─────────┐  ┌─────────┐  ┌────────┐ │  │
│  │  │ Schema  │  │  Model  │  │Document│ │  │
│  │  └─────────┘  └─────────┘  └────────┘ │  │
│  │  ┌─────────┐  ┌─────────┐  ┌────────┐ │  │
│  │  │Middleware│  │Validation│  │Population│ │  │
│  │  └─────────┘  └─────────┘  └────────┘ │  │
│  └──────────────────────────────────────┘   │
├──────────────────────────────────────────────┤
│         MongoDB Node.js Driver                │
└──────────────────────────────────────────────┘
```

Schema Definition - Deep Theory

What is a Schema?

A schema defines the structure of documents within a collection, including field types, validators, defaults, and indexes. It's the **blueprint** for your data.

javascript

```javascript
const mongoose = require('mongoose');
const { Schema } = mongoose;

// Basic schema definition
const userSchema = new Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true,
    minlength: [2, 'Name must be at least 2 characters'],
    maxlength: [50, 'Name cannot exceed 50 characters']
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    match: [/^\S+@\S+\.\S+$/, 'Please use a valid email address']
  },
  age: {
    type: Number,
    min: [13, 'Must be at least 13 years old'],
    max: [120, 'Age cannot exceed 120']
  },
  status: {
    type: String,
    enum: ['active', 'inactive', 'suspended'],
    default: 'active'
  },
  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true  // Cannot be modified after creation
  }
}, {
  timestamps: true,  // Adds createdAt and updatedAt automatically
  versionKey: '__v',  // Version key for document versioning
  collection: 'users' // Explicit collection name
});
```

Schema Types

javascript

```javascript
// All available schema types
const productSchema = new Schema({
  // String types
```

```
  sku: String,
  name: { type: String, required: true },
  description: { type: String, maxlength: 1000 },

  // Number types
  price: { type: Number, min: 0, required: true },
  quantity: { type: Number, min: 0, default: 0 },
  rating: { type: Number, min: 0, max: 5 },

  // Date types
  releaseDate: Date,
  lastUpdated: { type: Date, default: Date.now },

  // Boolean
  inStock: { type: Boolean, default: true },

  // Array types
  tags: [String],
  sizes: [{
    size: String,
    quantity: Number
  }],

  // Mixed (any type)
  metadata: Schema.Types.Mixed,

  // ObjectId (for references)
  category: { type: Schema.Types.ObjectId, ref: 'Category' },
  createdBy: { type: Schema.Types.ObjectId, ref: 'User' },

  // Decimal (for precise calculations)
  cost: { type: Schema.Types.Decimal128, required: true },

  // Map (key-value pairs)
  attributes: {
    type: Map,
    of: String
  }
});
```

Advanced Schema Options

```javascript
// Schema with indexes
const orderSchema = new Schema({
  orderId: { type: String, index: true },
  customerId: { type: Schema.Types.ObjectId, ref: 'User' },
  status: String,
  createdAt: Date,
  total: Number
});

// Compound index
orderSchema.index({ customerId: 1, createdAt: -1 });

// Unique index
orderSchema.index({ orderId: 1 }, { unique: true });
```

```
// Text index
orderSchema.index({ description: 'text' });

// Schema with virtuals
userSchema.virtual('fullName').get(function() {
  return `${this.firstName} ${this.lastName}`;
});

userSchema.virtual('profileUrl').get(function() {
  return `/users/${this._id}`;
});

// Virtual populate
orderSchema.virtual('items', {
  ref: 'OrderItem',
  localField: '_id',
  foreignField: 'orderId'
});

// Schema with custom methods
userSchema.methods.generateAuthToken = function() {
  return jwt.sign({ id: this._id }, process.env.JWT_SECRET);
};

userSchema.methods.comparePassword = async function(password) {
  return await bcrypt.compare(password, this.password);
};

// Schema with statics
userSchema.statics.findByEmail = function(email) {
  return this.findOne({ email: email.toLowerCase() });
};

userSchema.statics.getActiveUsers = function() {
  return this.find({ status: 'active' });
};

// Schema with query helpers
userSchema.query.byName = function(name) {
  return this.where({ name: new RegExp(name, 'i') });
};

userSchema.query.active = function() {
  return this.where({ status: 'active' });
};
```

Models - Deep Theory

What is a Model?

A model is a constructor compiled from a schema that represents a MongoDB collection. It provides an interface for **querying, creating, updating, and deleting** documents.

javascript

```javascript
// Create model from schema
const User = mongoose.model('User', userSchema);
const Product = mongoose.model('Product', productSchema);

// Models are singletons
const sameUserModel = mongoose.model('User'); // Returns existing model
```

Model Operations

javascript

```javascript
// Create
const user = new User({
  name: 'John Doe',
  email: 'john@example.com',
  age: 30
});

await user.save();  // Save to database

// Or using create
const user = await User.create({
  name: 'Jane Doe',
  email: 'jane@example.com'
});

// Bulk create
const users = await User.insertMany([
  { name: 'User1', email: 'user1@example.com' },
  { name: 'User2', email: 'user2@example.com' }
]);

// Read
const user = await User.findById('507f1f77bcf86cd799439011');
const user = await User.findOne({ email: 'john@example.com' });
const users = await User.find({ age: { $gt: 25 } }).sort('-createdAt');

// With query chaining
const activeUsers = await User
  .find({ status: 'active' })
  .select('name email')
  .limit(10)
  .skip(20)
  .sort('-createdAt')
  .populate('orders');

// Update
const user = await User.findByIdAndUpdate(
  id,
  { $set: { status: 'inactive' } },
  { new: true, runValidators: true }
```

```
);

// Update many
await User.updateMany(
  { status: 'inactive' },
  { $set: { lastInactiveAt: new Date() } }
);

// Delete
const user = await User.findByIdAndDelete(id);
await User.deleteMany({ status: 'deleted' });
```

Document Instances

```javascript
// Documents have methods
const user = await User.findById(id);

// Document properties
console.log(user._id);  // Document ID
console.log(user.name); // Field value

// Document methods (custom)
const token = user.generateAuthToken();
const isValid = await user.comparePassword(password);

// Document instance methods
await user.save();  // Save changes
await user.deleteOne(); // Delete document

// Check document state
console.log(user.isNew);  // True if not saved
console.log(user.isModified('name'));  // True if name modified
console.log(user.isSelected('email')); // True if email selected

// Transform document
const obj = user.toObject();  // Plain JavaScript object
const json = user.toJSON();    // JSON representation
const data = user.toObject({ virtuals: true });  // Include virtuals
```

Middleware (Hooks) - Deep Theory

What are Middleware?

Middleware (also called pre and post hooks) are functions that execute during the lifecycle of a document operation. They allow you to **inject logic** at specific points.

Middleware Types:

```text
Document Middleware:
- init (when document loaded)
- validate
- save
- remove
- updateOne
- deleteOne

Query Middleware:
- count
- find
- findOne
- updateMany
- deleteMany

Aggregate Middleware:
- aggregate
```

Pre Middleware

```javascript
// Pre-save hook
userSchema.pre('save', async function(next) {
  // Hash password before saving
  if (this.isModified('password')) {
    this.password = await bcrypt.hash(this.password, 10);
  }

  // Set timestamps
  if (this.isNew) {
    this.createdAt = new Date();
  }
  this.updatedAt = new Date();

  next();  // Continue with save
});

// Pre-validate hook
userSchema.pre('validate', function(next) {
  // Ensure email is lowercase
  if (this.email) {
    this.email = this.email.toLowerCase();
  }

  next();
});

// Pre-remove hook
userSchema.pre('remove', async function(next) {
```

```
  // Delete related documents
  await Order.deleteMany({ userId: this._id });
  await Comment.deleteMany({ userId: this._id });

  next();
});

// Pre-find hook
userSchema.pre(/^find/, function(next) {
  // Automatically populate references
  this.populate('department');

  // Exclude deleted documents
  this.where({ deletedAt: null });

  next();
});
```

Post Middleware

```javascript
// Post-save hook
userSchema.post('save', function(doc, next) {
  // Log activity
  logger.info(`User ${doc._id} saved`);

  // Emit event for other services
  eventEmitter.emit('user.created', doc);

  next();
});

// Post-find hook
userSchema.post(/^find/, function(docs, next) {
  if (Array.isArray(docs)) {
    docs.forEach(doc => {
      doc.viewCount = (doc.viewCount || 0) + 1;
    });
  }
  next();
});

// Post-remove hook
userSchema.post('remove', async function(doc) {
  // Cleanup completed
  await redis.del(`user:${doc._id}`);
});
```

Error Handling Middleware

```javascript
// Post-save error handling
userSchema.post('save', function(error, doc, next) {
  if (error.name === 'MongoServerError' && error.code === 11000) {
    next(new Error('Email already exists'));
  } else {
```

```
    next(error);
  }
});

// Multiple error handlers
orderSchema.post('save', errorHandler1, errorHandler2);
```

Validation - Deep Theory

Built-in Validation

```javascript
const userSchema = new Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    minlength: [2, 'Too short'],
    maxlength: [50, 'Too long'],
    match: [/^[a-zA-Z\s]+$/, 'Invalid characters'],
    enum: ['user', 'admin']  // Only for strings
  },
  age: {
    type: Number,
    required: true,
    min: [0, 'Age cannot be negative'],
    max: [150, 'Age cannot exceed 150']
  },
  email: {
    type: String,
    required: true,
    unique: true,  // Not a validator, but creates index
    lowercase: true,
    validate: {
      validator: function(v) {
        return /^\S+@\S+\.\S+$/.test(v);
      },
      message: props => `${props.value} is not a valid email!`
    }
  }
});
```

Custom Validation

```javascript
// Synchronous validator
userSchema.path('phone').validate({
  validator: function(phone) {
    return phone && phone.length === 10;
  },
  message: 'Phone number must be 10 digits'
```

```javascript
});

// Asynchronous validator
userSchema.path('email').validate({
  isAsync: true,
  validator: function(email, callback) {
    User.findOne({ email: email }, (err, user) => {
      if (err) return callback(false);
      callback(!user);  // true if email is unique
    });
  },
  message: 'Email already exists'
});

// Multiple validators
userSchema.path('password').validate({
  validator: v => v.length >= 8,
  message: 'Password must be at least 8 characters'
}).validate({
  validator: v => /[A-Z]/.test(v),
  message: 'Password must contain an uppercase letter'
});
```

Validation on Update

javascript

```javascript
// By default, update operations don't run validators
await User.findByIdAndUpdate(id, { age: -5 });  // Invalid age allowed!

// Enable validators
await User.findByIdAndUpdate(
  id,
  { age: -5 },
  {
    runValidators: true,      // Run schema validators
    context: 'query'           // Important for unique validator
  }
);

// Update with validation options
await User.updateOne(
  { _id: id },
  { $set: { age: 25 } },
  {
    runValidators: true,
    setDefaultsOnInsert: true  // Apply defaults for new fields
  }
);
```

Custom Validation Error Messages

```javascript
const userSchema = new Schema({
  name: {
    type: String,
    validate: {
      validator: v => /^[a-zA-Z\s]+$/.test(v),
      message: '{VALUE} is not a valid name. Only letters and spaces allowed.'
    }
  },
  age: {
    type: Number,
    min: [13, 'You must be at least 13. You entered {VALUE}'],
    max: [120, 'Age cannot exceed 120. You entered {VALUE}']
  }
});

// Handling validation errors
try {
  await user.save();
} catch (error) {
  if (error.name === 'ValidationError') {
    Object.values(error.errors).forEach(err => {
      console.log(err.path, err.message);
    });
  }
}
```

Summary: The Complete Mental Model

Tools & Ecosystem

Think of MongoDB tools as your **workshop**:

- **Compass** is your microscope and workbench - examine, analyze, build
- **mongosh** is your precision toolkit - exact control for expert work
- **Atlas** is your automated factory - push-button deployment and management

Backend Integration

Think of backend integration as **plumbing**:

- **Connection pooling** is water pressure - enough for demand, not too much
- **Error handling** is leak detection - find and fix before damage

- **Transactions** are precision work - everything must flow correctly
- **Mongoose** is your blueprint system - design before building

The Golden Rules

For Tools:

1. **Use the right tool** - Compass for exploration, mongosh for automation
2. **Learn the shortcuts** - Tools are faster when you know them well
3. **Atlas for production** - Let experts manage infrastructure

For Backend Integration:

1. **Pool connections** - Never open/close per request
2. **Handle errors gracefully** - Every error has a response plan
3. **Use transactions sparingly** - Only when atomicity required
4. **Mongoose for structure** - Schemas prevent data rot
5. **Validate at every layer** - Schema, application, and user input

MongoDB Advanced Features & DevOps: Complete Theory Notes for Senior Developers

---

18. Advanced Features

Change Streams (Real-time Updates) - Deep Theory

What are Change Streams?

Change streams allow applications to **listen for real-time data changes** directly from MongoDB. They provide a way to react to insert, update, replace, and delete operations as they happen.

The Problem Change Streams Solve:

```text
Traditional Approach:                    Change Streams Approach:
while(true) {                            const cursor = db.watch();
    const newData = db.find({             cursor.on('change', (change) => {
        timestamp: { $gt: lastCheck }         processChange(change);
    });                                     });
    process(newData);                       // Real-time, event-driven
    sleep(1000);                         // No polling, no delay
}
// Polling overhead, latency
// Missed changes between polls
```

Change Streams Architecture

```text
┌──────────────────────────────────────────────────────────────┐
│                       MongoDB Cluster                        │
├──────────────────────────────────────────────────────────────┤
│  Primary                                                     │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Oplog (capped collection)                             │    │
```

```
│   │ [op1][op2][op3][op4][op5][op6][op7][op8]...           │   │
│   └──────────────────────────────────────────────────────┘   │
│           ↑                                          ↑       │
│           │ Watch                                    │ Watch │
│   ┌───────┴─────────┐                       ┌───────┴─────────┐
│   │ Application 1   │                       │ Application 2   │
│   │ Change Stream   │                       │ Change Stream   │
│   └─────────────────┘                       └─────────────────┘
```

How Change Streams Work

javascript

```javascript
// 1. Open a change stream
const changeStream = db.collection('orders').watch();

// 2. Listen for changes
changeStream.on('change', (change) => {
  console.log('Change detected:', change);

  switch(change.operationType) {
    case 'insert':
      handleNewOrder(change.fullDocument);
      break;
    case 'update':
      handleOrderUpdate(change.documentKey._id, change.updateDescription);
      break;
    case 'delete':
      handleOrderDeletion(change.documentKey._id);
      break;
    case 'replace':
      handleOrderReplace(change.fullDocument);
      break;
  }
});

// 3. Error handling
changeStream.on('error', (error) => {
  console.error('Change stream error:', error);
  // Reconnect logic
});
```

Change Stream Events

javascript

```javascript
// Insert event
{
  "_id": { "_data": "8265E4B3F0000000012B022C010029..." },
  "operationType": "insert",
  "clusterTime": Timestamp(1643125744, 1),
  "ns": { "db": "myDB", "coll": "orders" },
  "documentKey": { "_id": ObjectId("65e4b3f0a1b2c3d4e5f67890") },
  "fullDocument": {
    "_id": ObjectId("65e4b3f0a1b2c3d4e5f67890"),
```

```
    "customer": "John Doe",
    "total": 299.99,
    "status": "pending"
  }
}

// Update event
{
  "operationType": "update",
  "ns": { "db": "myDB", "coll": "orders" },
  "documentKey": { "_id": ObjectId("65e4b3f0a1b2c3d4e5f67890") },
  "updateDescription": {
    "updatedFields": { "status": "shipped", "tracking": "1Z999AA" },
    "removedFields": ["tempFlag"]
  }
}

// Delete event
{
  "operationType": "delete",
  "ns": { "db": "myDB", "coll": "orders" },
  "documentKey": { "_id": ObjectId("65e4b3f0a1b2c3d4e5f67890") }
}

// Invalidate event (when collection dropped)
{
  "operationType": "invalidate"
}
```

Change Stream Options

javascript

```javascript
// Pipeline for filtering
const changeStream = db.orders.watch([
  {
    $match: {
      "fullDocument.status": "pending",  // Only pending orders
      "operationType": "insert"           // Only inserts
    }
  },
  {
    $project: {
      "fullDocument.customer": 1,
      "fullDocument.total": 1
    }
  }
]);

// Resume after disconnection
const resumeToken = lastKnownToken;
const changeStream = db.orders.watch([], {
  resumeAfter: resumeToken,
  maxAwaitTimeMS: 5000  // Wait up to 5 seconds for changes
});

// Start from now (default)
const changeStream = db.orders.watch([], { startAtOperationTime: Timestamp() });
```

```javascript
// Include both pre and post images
const changeStream = db.orders.watch([], {
  fullDocument: 'updateLookup',  // Fetch full document on updates
  fullDocumentBeforeChange: 'whenAvailable'  // Include pre-image
});
```

Use Cases

```javascript
// 1. Real-time notifications
const changeStream = db.orders.watch([
  { $match: { operationType: 'insert' } }
]);

changeStream.on('change', (change) => {
  // Send email/SMS/push notification
  notifyCustomer(change.fullDocument.customerId, 'Order placed');
});

// 2. Cache invalidation
const changeStream = db.products.watch();

changeStream.on('change', (change) => {
  // Invalidate cache for this product
  redis.del(`product:${change.documentKey._id}`);
});

// 3. Search index updates
const changeStream = db.articles.watch();

changeStream.on('change', (change) => {
  // Update Elasticsearch
  elasticsearch.index({
    index: 'articles',
    id: change.documentKey._id,
    body: change.fullDocument
  });
});

// 4. Analytics pipeline
const changeStream = db.userActions.watch();

changeStream.on('change', (change) => {
  // Send to Kafka for stream processing
  kafka.send('user-events', change);
});

// 5. Cross-service synchronization
const changeStream = db.customers.watch();

changeStream.on('change', (change) => {
  // Update CRM system
  crmApi.syncCustomer(change.fullDocument);
});
```

Change Streams vs Triggers

Aspect	Change Streams	Database Triggers
Location	Application-side	Database-side
Latency	Real-time	Same transaction
Complexity	Simple to implement	Complex to maintain
Scalability	Multiple listeners	Single execution
Use Case	Event-driven apps	Data integrity

TTL Collections - Deep Theory

What are TTL Collections?

TTL (Time-To-Live) collections automatically remove documents after a specified period. They're MongoDB's built-in mechanism for **data expiration and cleanup**.

The Problem TTL Collections Solve:

```text
Manual Cleanup:                          TTL Collections:
- Write cleanup scripts                   - Automatic deletion
- Schedule cron jobs                       - Built-in mechanism
- Monitor disk space                       - Self-maintaining
- Risk of forgetting                        - Set and forget
- Inefficient queries                       - Efficient background process
```

How TTL Works

```text
┌──────────────────────────────────────────────────────────────┐
│                          TTL Index                           │
```

```
│  Collection: events                                          │
│  Index: { createdAt: 1 } with expireAfterSeconds: 3600       │
├──────────────────────────────────────────────────────────────┤
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Document 1: createdAt: 10:00                          │    │
│  │ Document 2: createdAt: 10:30                          │    │
│  │ Document 3: createdAt: 11:00 ← Current time: 11:30    │    │
│  │ Document 4: createdAt: 11:15                          │    │
│  └──────────────────────────────────────────────────────┘    │
│                              ↓                               │
│  TTL Monitor runs every 60 seconds                           │
│  Finds: Documents where createdAt < (current - 3600)         │
│  Result: Documents 1 and 2 are deleted                       │
└──────────────────────────────────────────────────────────────┘
```

Implementing TTL

```javascript
// 1. Create TTL index
db.events.createIndex(
  { createdAt: 1 },
  { expireAfterSeconds: 3600 }  // Delete after 1 hour
);

// 2. Insert documents
db.events.insertOne({
  eventType: 'user_login',
  userId: 12345,
  createdAt: new Date()  // TTL uses this field
});

// 3. TTL with different expiration times
db.sessions.createIndex(
  { lastActivity: 1 },
  { expireAfterSeconds: 86400 }  // 24 hours
);

// 4. Compound TTL (not allowed - must be single field)
// This WON'T work:
db.events.createIndex(
  { createdAt: 1, eventType: 1 },
  { expireAfterSeconds: 3600 }
);  // Error!

// 5. TTL on a date array (uses earliest date)
db.sessions.createIndex(
  { activityTimestamps: 1 },
  { expireAfterSeconds: 86400 }
);
// Document expires when earliest timestamp in array is old
```

TTL with Different Timeframes

javascript

```javascript
// Session data - expire after 30 minutes
db.sessions.createIndex(
  { lastAccess: 1 },
  { expireAfterSeconds: 1800 }
);

// Logs - expire after 7 days
db.logs.createIndex(
  { timestamp: 1 },
  { expireAfterSeconds: 604800 }
);

// Notifications - expire after 30 days
db.notifications.createIndex(
  { createdAt: 1 },
  { expireAfterSeconds: 2592000 }
);

// Temporary data - expire after 1 hour
db.tempData.createIndex(
  { createdAt: 1 },
  { expireAfterSeconds: 3600 }
);
```

TTL Limitations and Considerations

javascript

```javascript
// 1. TTL doesn't work on capped collections
db.createCollection("logs", { capped: true, size: 1000000 });
db.logs.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 });  // Won't wor
k!

// 2. TTL runs every 60 seconds, not real-time
// Documents may persist up to 60 seconds after expiration

// 3. TTL index can't be compound
// Must be single field index

// 4. Date field must be BSON date type
db.collection.createIndex({ date: 1 }, { expireAfterSeconds: 3600 });
// Works with: ISODate("2024-01-15")
// Doesn't work with: "2024-01-15" (string)

// 5. TTL doesn't guarantee exact deletion time
// Factors affecting deletion:
// - TTL monitor frequency (60 seconds)
// - Replica set lag
// - Load on primary
```

Monitoring TTL

```javascript
```

```
// Check TTL indexes
db.collection.getIndexes();

// View TTL deletion stats
db.serverStatus().metrics.ttl;
{
  "deletedDocuments": 15678,
  "passes": 1250
}

// Check if TTL is working
db.collection.find({
  createdAt: { $lt: new Date(Date.now() - 3600 * 1000) }
}).count();  // Should be 0 if TTL working
```

Capped Collections - Deep Theory

What are Capped Collections?

Capped collections are fixed-size collections that maintain insertion order and automatically remove the oldest documents when the size limit is reached. They're designed for **high-throughput, circular buffer** use cases.

The Problem Capped Collections Solve:

```text
Regular Collections:                    Capped Collections:
- Unlimited growth                      - Fixed size
- Manual cleanup needed                  - Automatic overwrite
- Variable performance                    - Consistent performance
- Can't guarantee order                   - Guaranteed insertion order
- Good for permanent data                  - Perfect for rolling logs
```

Capped Collection Architecture

```text
┌──────────────────────────────────────────────────────────────┐
│                    Capped Collection                          │
│  Fixed size: 1GB                                              │
├──────────────────────────────────────────────────────────────┤
│  ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐ ┌───┐          │
│  │D1 │ │D2 │ │D3 │ │D4 │ │D5 │ │D6 │ │D7 │ │D8 │ ...       │
│  └───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘          │
│    ↑                            ↑                             │
│  Oldest                       Newest                          │
```

```
│                                                                  │
│  When collection is full:                                        │
│  ┌────┐ ┌────┐ ┌────┐ ┌────┐ ┌────┐ ┌────┐ ┌────┐ ┌────┐          │
│  │D2  │ │D3  │ │D4  │ │D5  │ │D6  │ │D7  │ │D8  │ │D9  │ ...       │
│  └────┘ └────┘ └────┘ └────┘ └────┘ └────┘ └────┘ └────┘          │
│    ↑                            ↑                                 │
│  Oldest (D1 removed)          Newest (D9 added)                  │
└──────────────────────────────────────────────────────────────────┘
```

Creating Capped Collections

javascript

```javascript
// Create capped collection
db.createCollection("logs", {
  capped: true,
  size: 1000000,  // 1MB max size
  max: 5000       // Optional: max number of documents
});

// Convert existing collection to capped
db.runCommand({
  convertToCapped: "logs",
  size: 1000000
});

// Check if collection is capped
db.logs.isCapped();  // Returns true/false

// Get capped collection stats
db.logs.stats().capped;  // true
db.logs.stats().maxSize;  // Size in bytes
db.logs.stats().max;       // Max documents
```

Use Cases

javascript

```javascript
// 1. Application logs
db.createCollection("app_logs", {
  capped: true,
  size: 100 * 1024 * 1024,  // 100MB
  max: 10000
});

// Insert logs (oldest automatically removed)
db.app_logs.insertOne({
  timestamp: new Date(),
  level: "ERROR",
  message: "Database connection failed",
  stack: "..."
});

// 2. Event sourcing / audit trails
db.createCollection("audit_events", {
```

```
  capped: true,
  size: 500 * 1024 * 1024,  // 500MB
  max: 50000
});

// 3. Real-time metrics
db.createCollection("metrics", {
  capped: true,
  size: 50 * 1024 * 1024,  // 50MB
  max: 10000
});

// Insert metrics every second
setInterval(() => {
  db.metrics.insertOne({
    timestamp: new Date(),
    cpu: os.loadavg()[0],
    memory: process.memoryUsage().heapUsed,
    connections: db.serverStatus().connections.current
  });
}, 1000);
```

Tailable Cursors

javascript

```
// Tailable cursor - like "tail -f" for capped collections
const cursor = db.logs.find(
  {},  // No filter, get all
  {
    tailable: true,
    awaitData: true,  // Wait for new data
    noCursorTimeout: true
  }
);

// Process logs in real-time
function processLogs(cursor) {
  while (cursor.hasNext()) {
    const log = cursor.next();
    processLog(log);
  }

  // Wait a bit, then check again
  setTimeout(() => processLogs(cursor), 100);
}

// ES6 async iterator
const cursor = db.logs.find().tailable().awaitData();
for await (const doc of cursor) {
  console.log('New log:', doc);
}
```

Capped Collection Limitations

```
javascript

// 1. No document deletion
```

```javascript
db.logs.deleteOne({ _id: id });  // Error! Cannot delete from capped

// 2. No updates that increase size
db.logs.updateOne(
  { _id: id },
  { $set: { message: "This is a much longer message that won't fit" } }
);  // Error! Document size changed

// 3. No sharding
sh.shardCollection("db.logs", { _id: 1 });  // Error! Can't shard capped

// 4. Document order is insertion order
// Can't change order, can't reorder

// 5. _id index automatically created, but others must be created manually
db.logs.createIndex({ timestamp: -1 });  // Allowed
```

Geospatial Queries - Deep Theory

What are Geospatial Queries?

Geospatial queries allow you to query documents based on geographic location. MongoDB supports various geospatial data types and query operators for **location-based applications**.

The Problem Geospatial Queries Solve:

```text
Traditional Approach:                    Geospatial Queries:
- Store lat/lng as fields                 - Native geo data types
- Manual distance calculation              - Built-in distance operators
- No spatial indexing                       - Geospatial indexes
- Complex polygon logic                      - Simple polygon queries
- Slow for large datasets                     - Optimized for geo
```

GeoJSON Format

```javascript
// Point (single location)
{
  location: {
    type: "Point",
    coordinates: [-73.97, 40.77]  // [longitude, latitude]
  }
}

// LineString (path)
{
  path: {
```

```
    type: "LineString",
    coordinates: [
      [-73.97, 40.77],
      [-73.98, 40.78],
      [-73.99, 40.79]
    ]
  }
}

// Polygon (area)
{
  area: {
    type: "Polygon",
    coordinates: [[
      [-73.97, 40.77],
      [-73.98, 40.78],
      [-73.99, 40.79],
      [-73.97, 40.77]  // Back to start
    ]]
  }
}

// MultiPoint, MultiLineString, MultiPolygon
{
  locations: {
    type: "MultiPoint",
    coordinates: [
      [-73.97, 40.77],
      [-73.98, 40.78]
    ]
  }
}
```

Geospatial Indexes

javascript

```
// 2dsphere index (for GeoJSON)
db.places.createIndex({ location: "2dsphere" });

// 2d index (for legacy coordinate pairs)
db.places.createIndex({ location: "2d" });

// Compound with other fields
db.restaurants.createIndex({
  location: "2dsphere",
  cuisine: 1,
  price: 1
});
```

Geospatial Query Operators

javascript

```
// 1. $near (find near a point)
db.restaurants.find({
  location: {
    $near: {
```

```
      $geometry: {
        type: "Point",
        coordinates: [-73.97, 40.77]
      },
      $maxDistance: 1000,  // meters
      $minDistance: 100
    }
  }
});

// 2. $geoWithin (find within area)
db.places.find({
  location: {
    $geoWithin: {
      $geometry: {
        type: "Polygon",
        coordinates: [[
          [-73.97, 40.77],
          [-73.98, 40.78],
          [-73.99, 40.79],
          [-73.97, 40.77]
        ]]
      }
    }
  }
});

// 3. $geoIntersects (find intersecting)
db.routes.find({
  path: {
    $geoIntersects: {
      $geometry: {
        type: "Point",
        coordinates: [-73.97, 40.77]
      }
    }
  }
});

// 4. $geoWithin with polygon (legacy)
db.places.find({
  location: {
    $geoWithin: {
      $box: [[-73.97, 40.77], [-73.98, 40.78]]  // Rectangle
    }
  }
});

// 5. $geoWithin with circle
db.places.find({
  location: {
    $geoWithin: {
      $center: [[-73.97, 40.77], 10]  // 10 km radius (legacy)
    }
  }
});
```

Geospatial Aggregation

javascript

```javascript
// Calculate distances
db.places.aggregate([
  {
    $geoNear: {
      near: { type: "Point", coordinates: [-73.97, 40.77] },
      distanceField: "distance",  // Field to store calculated distance
      spherical: true,
      maxDistance: 5000,
      query: { type: "restaurant" },
      limit: 10
    }
  },
  {
    $project: {
      name: 1,
      distance: 1,
      distanceKm: { $divide: ["$distance", 1000] }
    }
  }
]);

// Group by region
db.places.aggregate([
  {
    $geoNear: {
      near: { type: "Point", coordinates: [-73.97, 40.77] },
      distanceField: "distance",
      includeLocs: "location"
    }
  },
  {
    $bucket: {
      groupBy: "$distance",
      boundaries: [0, 1000, 5000, 10000],
      default: "Other",
      output: {
        count: { $sum: 1 },
        places: { $push: "$name" }
      }
    }
  }
]);
```

Real-World Examples

javascript

```javascript
// 1. Find nearby restaurants
app.get('/api/restaurants/nearby', async (req, res) => {
  const { lat, lng, radius = 5000 } = req.query;

  const restaurants = await db.restaurants.find({
    location: {
      $near: {
        $geometry: {
          type: "Point",
          coordinates: [parseFloat(lng), parseFloat(lat)]
```

```
        },
        $maxDistance: parseInt(radius)
      }
    },
    cuisine: req.query.cuisine
  }).limit(20).toArray();

  res.json(restaurants);
});

// 2. Geofencing (user entered area)
const geofence = {
  type: "Polygon",
  coordinates: [/* fence coordinates */]
};

const userLocation = {
  type: "Point",
  coordinates: [user.lng, user.lat]
};

const inGeofence = await db.geofences.findOne({
  area: {
    $geoIntersects: {
      $geometry: userLocation
    }
  }
});

if (inGeofence) {
  sendNotification(user, "Welcome to the area!");
}

// 3. Delivery zone check
const deliveryZones = await db.zones.find({
  area: {
    $geoIntersects: {
      $geometry: customerLocation
    }
  }
}).toArray();

if (deliveryZones.length > 0) {
  // Customer is in delivery area
  calculateDeliveryFee(deliveryZones[0].fee);
}
```

Full-Text Search - Deep Theory

What is Full-Text Search?

Full-text search allows you to search for text within string fields, with features like stemming, weighting, and relevance scoring. MongoDB provides both built-in text search and Atlas Search (Lucene-based).

The Problem Full-Text Search Solves:

```text
Regex Search:                              Text Search:
db.articles.find({                            db.articles.find({
  content: /mongodb performance/                $text: { $search: "mongodb performa
nce" }
})                                              })
// No relevance scoring                      // Relevance scores
// No stemming                                // Understands "running" = "run"
// Can't handle typos                          // Fuzzy matching options
// Slow on large collections                    // Optimized with text index
```

Text Index

```javascript
// Create text index on single field
db.articles.createIndex({ content: "text" });

// Create text index on multiple fields
db.articles.createIndex({
  title: "text",
  content: "text",
  tags: "text"
});

// With weights (importance)
db.articles.createIndex(
  {
    title: "text",
    content: "text",
    tags: "text"
  },
  {
    weights: {
      title: 10,    // Matches in title are 10x more important
      tags: 5,      // Tags are 5x more important
      content: 1    // Content is baseline
    },
    name: "articles_text_index"
  }
);
```

Text Search Queries

```javascript
// Basic text search
db.articles.find(
  { $text: { $search: "mongodb performance" } },
  { score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } });
```

```
// Exact phrase
db.articles.find({
  $text: { $search: "\"mongodb indexing\"" }
});

// Exclude words
db.articles.find({
  $text: { $search: "mongodb -performance" }  // Exclude performance
});

// Language-specific search
db.articles.find({
  $text: {
    $search: "chien",
    $language: "fr"  // French stemming
  }
});

// Case and diacritic sensitivity
db.articles.find({
  $text: {
    $search: "café",
    $caseSensitive: false,
    $diacriticSensitive: true
  }
});
```

Text Score and Relevance

```javascript
// Get text score
db.articles.find(
  { $text: { $search: "mongodb" } },
  {
    title: 1,
    score: { $meta: "textScore" }
  }
).sort({ score: { $meta: "textScore" } });

// Score factors:
// - Term frequency (how often term appears)
// - Inverse document frequency (rarer terms weigh more)
// - Field weights (configured in index)
// - Document length (shorter docs weigh more)

// Use score in aggregation
db.articles.aggregate([
  {
    $match: {
      $text: { $search: "mongodb performance" }
    }
  },
  {
    $addFields: {
      relevance: { $meta: "textScore" }
    }
  },
```

```
  {
    $match: {
      relevance: { $gt: 1.0 }  // Only highly relevant
    }
  },
  {
    $project: {
      title: 1,
      relevance: 1,
      excerpt: { $substr: ["$content", 0, 200] }
    }
  },
  { $sort: { relevance: -1 } },
  { $limit: 10 }
]);
```

Atlas Search (Advanced)

```javascript
// Atlas Search index definition (in Atlas UI)
{
  "mappings": {
    "dynamic": false,
    "fields": {
      "title": {
        "type": "string",
        "analyzer": "lucene.standard"
      },
      "content": {
        "type": "string",
        "analyzer": "lucene.english"
      },
      "price": {
        "type": "number"
      },
      "publishDate": {
        "type": "date"
      }
    }
  }
}

// Atlas Search query
db.articles.aggregate([
  {
    $search: {
      "compound": {
        "must": [
          {
            "text": {
              "query": "mongodb",
              "path": "title",
              "score": { "boost": { "value": 5 } }
            }
          },
          {
            "text": {
              "query": ["database", "nosql"],
```

```
              "path": "content"
            }
          }
        ],
        "filter": [
          {
            "range": {
              "path": "price",
              "lte": 100
            }
          }
        ]
      }
    }
  },
  {
    $addFields: {
      score: { $meta: "searchScore" }
    }
  },
  { $sort: { score: -1 } },
  { $limit: 20 }
]);
```

Search Features

```javascript
// Fuzzy matching (typo tolerance)
db.articles.find({
  $text: { $search: "mongdodb", $fuzzy: { maxEdits: 2 } }
});

// Autocomplete
db.products.aggregate([
  {
    $search: {
      "autocomplete": {
        "query": "lapt",
        "path": "name"
      }
    }
  },
  { $limit: 5 },
  { $project: { name: 1 } }
]);

// Faceted search
db.products.aggregate([
  {
    $searchMeta: {
      "facet": {
        "operator": {
          "text": {
            "query": "laptop",
            "path": "description"
          }
        },
        "facets": {
```

```
          "brandFacet": {
            "type": "string",
            "path": "brand"
          },
          "priceRanges": {
            "type": "range",
            "path": "price",
            "boundaries": [0, 500, 1000, 2000],
            "default": "other"
          }
        }
      }
    }
  }
]);

// Highlighting
db.articles.aggregate([
  {
    $search: {
      "text": {
        "query": "mongodb",
        "path": "content"
      },
      "highlight": {
        "path": "content"
      }
    }
  },
  {
    $project: {
      title: 1,
      highlights: { $meta: "searchHighlights" }
    }
  }
]);
```

19. DevOps & Production Usage

Monitoring - Deep Theory

What to Monitor?

Monitoring is the practice of collecting, analyzing, and acting on system metrics. In production MongoDB, you need to monitor **everything that can fail**.

The Four Golden Signals:

```text

1. Latency: Time to serve requests
```

```
2. Traffic: How many requests
3. Errors: Rate of failed requests
4. Saturation: How full the system is
```

Key Metrics to Monitor

```javascript
// 1. Operation Counters
db.serverStatus().opcounters
{
  "insert": 123456,
  "query": 789012,
  "update": 345678,
  "delete": 90123,
  "getmore": 45678,
  "command": 567890
}

// 2. Connections
db.serverStatus().connections
{
  "current": 42,
  "available": 958,
  "totalCreated": 1234
}
// Alert if current > 80% of available

// 3. Memory
db.serverStatus().mem
{
  "resident": 4096,  // MB
  "virtual": 8192,
  "mapped": 6144
}
// Resident memory should be stable

// 4. Cache Activity
db.serverStatus().wiredTiger.cache
{
  "bytes currently in the cache": 10737418240,  // 10GB
  "maximum bytes configured": 21474836480,      // 20GB
  "pages read into cache": 123456,
  "pages written from cache": 78901
}
// Cache hit ratio = (pages read - pages written) / pages read

// 5. Replication Lag
rs.printSlaveReplicationInfo()
// Alert if lag > 10 seconds

// 6. Query Performance
db.currentOp({
  "secs_running": { $gt: 5 },
  "op": { $in: ["query", "command"] }
})
// Long-running queries need optimization
```

```
// 7. Disk Usage
db.stats().dataSize
db.stats().indexSize
// Alert if disk usage > 80%

// 8. Errors and Warnings
db.serverStatus().asserts
{
  "regular": 0,
  "warning": 10,
  "msg": 0,
  "user": 5,
  "rollovers": 0
}
```

Monitoring Tools

```javascript
// 1. mongostat (command-line)
mongostat --host localhost:27017
// Output:
// insert query update delete getmore command dirty used flushes vsize   res qrw a
rw net_in net_out conn time
//    0     1      0      0       0       2|0  0.1% 80.0%       0 1.98G 1.8G 0|0 1
|0   62b   56.5k    1 10:00:01

// 2. mongotop
mongotop --host localhost:27017
// Shows read/write activity per collection

// 3. MongoDB Cloud Manager / Ops Manager
// - Full monitoring suite
// - Alerting
// - Performance advisor
// - Backup management

// 4. Prometheus + Grafana
// - Export metrics via mongodb_exporter
// - Custom dashboards
// - Alert rules

// 5. Datadog integration
// - Pre-built dashboards
// - Machine learning for anomaly detection
// - APM integration
```

Setting Up Alerts

```javascript
// Critical alerts (page on-call)
- Replica set has no primary
- Node down for >5 minutes
- Disk usage >90%
- Replication lag >1 hour

// Warning alerts (ticket during work hours)
- CPU >80% for 15 minutes
- Memory pressure high
- Connection count >80% of max
- Slow queries (>1s) increasing
- Cache eviction rate high

// Informational alerts (email)
- Daily backup completed/failed
- Index build completed
- Version upgrade available
- New slow query detected
```

Logging - Deep Theory

MongoDB Log Levels

```javascript
// Log levels (0-5):
// 0: Off
// 1: Severe
// 2: Error
// 3: Warning
// 4: Info (default)
// 5: Debug

// Set log level
db.setLogLevel(3, "command")  // Set command component to warning
db.setLogLevel(2)              // Set all components to error

// Components:
// - access
// - command
// - control
```

```
// - geo
// - index
// - network
// - query
// - replication
// - sharding
// - storage
// - transaction
// - write
```

Log Analysis

```javascript
// Sample log line
{
  "t": {"$date": "2024-01-15T10:30:45.123Z"},
  "s": "I",  // Severity: I=Info, W=Warning, E=Error, F=Fatal
  "c": "COMMAND",  // Component
  "id": 12345,  // Log ID
  "ctx": "conn123",  // Context (connection)
  "msg": "command",  // Message
  "attr": {  // Attributes
    "command": "find",
    "ns": "test.users",
    "planSummary": "IXSCAN { age: 1 }",
    "keysExamined": 100,
    "docsExamined": 100,
    "cursorExhausted": true,
    "numYields": 0,
    "nreturned": 50,
    "reslen": 1024,
    "locks": {
      "ReplicationStateTransition": "w:1",
      "Global": "r:1",
      "Database": "r:1",
      "Collection": "r:1"
    },
    "protocol": "op_msg",
    "durationMillis": 5
  }
}
```

Log Management Strategies

```javascript
// 1. Log rotation (mongod.conf)
systemLog:
  destination: file
  path: /var/log/mongodb/mongod.log
  logAppend: true
  logRotate: reopen
  timeStampFormat: iso8601-utc

// 2. Centralized logging
// Use rsyslog to forward to central server
// Or use ELK stack (Elasticsearch, Logstash, Kibana)
```

```
// 3. Log parsing scripts
#!/bin/bash
# parse_slow_queries.sh
grep "durationMillis" /var/log/mongodb/mongod.log | \
  awk -F'durationMillis' '{print $2}' | \
  awk -F'}' '{print $1}' | \
  sort -n | \
  tail -20

// 4. Log-based alerts
// Alert on patterns:
// - "Failed to authenticate"
// - "Connection refused"
// - "Too many open files"
// - "No primary detected"
```

Structured Logging

```javascript
// Application logging (Node.js)
const logger = winston.createLogger({
  level: 'info',
  format: winston.format.json(),
  defaultMeta: { service: 'user-service' },
  transports: [
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});

logger.info('Database query executed', {
  operation: 'find',
  collection: 'users',
  filter: { status: 'active' },
  duration: 45,
  userId: '12345'
});
```

Scaling Strategies - Deep Theory

When to Scale

The Scaling Decision Tree:

```text
Is performance degrading?
├── YES → Identify bottleneck
│   ├── CPU high → Scale vertically or shard
│   ├── RAM high → Optimize indexes or add RAM
│   ├── Disk I/O high → Add indexes or faster disks
```

```
│   └── Network high → Compress or add nodes
└── NO → Don't scale yet
```

Vertical Scaling (Scale Up)

```javascript
// Add more power to existing server
// - More CPU cores
// - More RAM
// - Faster disks (SSD)
// - Better network

// Pros:
// - Simple, no application changes
// - No sharding complexity
// - Consistent performance

// Cons:
// - Hardware limits
// - Expensive at high end
// - Single point of failure
```

Horizontal Scaling (Scale Out)

```javascript
// 1. Replica Sets for read scaling
// Add more secondaries
rs.add("mongodb3:27017")
rs.add("mongodb4:27017")

// Configure read preference
db.collection.find().readPref("secondary")

// 2. Sharding for write scaling
sh.shardCollection("db.events", { userId: "hashed" })
sh.addShard("shard03/rs3:27017")

// 3. Application-level sharding
// Shard by tenant/region
const db = connection.db(`tenant_${tenantId}`);

// 4. Caching layer
// Add Redis/Memcached in front
```

Scaling Strategies by Component

```javascript
// CPU-bound: Need more processing power
```

```
// Solutions:
// - Optimize queries (indexes, covered queries)
// - Add indexes to reduce document scans
// - Shard to distribute load
// - Upgrade CPU

// Memory-bound: Need more RAM
// Solutions:
// - Increase WiredTiger cache
// - Optimize indexes (smaller, fewer)
// - Shard to distribute working set
// - Add RAM

// I/O-bound: Disk too slow
// Solutions:
// - Use SSDs
// - Add indexes to reduce scans
// - Increase cache to reduce reads
// - Shard to distribute I/O

// Network-bound: Too much data transfer
// Solutions:
// - Project only needed fields
// - Compress data
// - Add read replicas locally
// - Use $out for aggregations
```

Auto-scaling Considerations

javascript

```
// Cloud auto-scaling (AWS/Azure/GCP)
// MongoDB-specific considerations:

// 1. Stateful nature
// Can't just add nodes like web servers
// Need time for data sync

// 2. Connection handling
// Scaling up = new connections
// Scaling down = disconnect clients

// 3. Data rebalancing
// Adding shards triggers balancing
// Takes time and resources

// 4. Monitoring triggers
// Scale when:
// - CPU >70% for 15 minutes
// - Disk >80% full
// - Connections >80% of max
```

Deployment Best Practices - Deep Theory

Production Checklist

```javascript
// 1. Hardware and OS
☐ Use SSDs for data storage
☐ Separate disk for logs
☐ Disable NUMA (numactl --interleave=all)
☐ Disable Transparent Huge Pages
☐ Set appropriate file limits (ulimit -n 64000)
☐ Sync clock with NTP
☐ Configure RAID 10 for data drives

// 2. MongoDB Configuration
☐ Enable authentication
☐ Enable TLS/SSL
☐ Set bindIp to specific interfaces
☐ Configure appropriate cache size
☐ Set up replication (minimum 3 nodes)
☐ Configure backup strategy
☐ Set up monitoring and alerts

// 3. Security
☐ Firewall rules (only necessary ports)
☐ VPC/Private networking
☐ Encrypted storage
☐ Regular security updates
☐ Audit logging enabled
☐ Strong passwords/keyfiles

// 4. Performance
☐ Indexes on all query patterns
☐ Slow query logging enabled
☐ Connection pooling configured
☐ Working set fits in RAM
☐ Regular explain plan reviews
```

Configuration Best Practices

javascript

```
// mongod.conf production example
storage:
  dbPath: /data/db
  journal:
    enabled: true
    commitIntervalMs: 100
  wiredTiger:
    engineConfig:
      cacheSizeGB: 16  # 50-80% of RAM
      journalCompressor: snappy
    collectionConfig:
```

```yaml
      blockCompressor: snappy
    indexConfig:
      prefixCompression: true

systemLog:
  destination: file
  path: /var/log/mongodb/mongod.log
  logAppend: true
  logRotate: reopen
  verbosity: 0
  quiet: false

net:
  port: 27017
  bindIp: 10.0.1.100,10.0.2.100  # Internal IPs only
  maxIncomingConnections: 5000
  compression:
    compressors: snappy
  ssl:
    mode: requireSSL
    PEMKeyFile: /etc/ssl/mongodb.pem
    CAFile: /etc/ssl/ca.pem

processManagement:
  fork: true
  pidFilePath: /var/run/mongodb/mongod.pid

security:
  authorization: enabled
  keyFile: /etc/mongodb-keyfile
  clusterAuthMode: keyFile
  encryption:
    enableEncryption: true
    encryptionKeyFile: /etc/mongodb-encryption-key

replication:
  replSetName: rs0
  oplogSizeMB: 10240
  secondaryIndexPrefetch: all

operationProfiling:
  mode: slowOp
  slowOpThresholdMs: 100
```

Deployment Patterns

```javascript
// 1. Single region, multiple AZ
// Region: us-east-1
// AZ1: Primary
// AZ2: Secondary
// AZ3: Secondary
// Advantage: Survives AZ failure

// 2. Multi-region
// Region1 (primary): us-east-1 (3 nodes)
// Region2 (secondary): us-west-2 (3 nodes)
// Region3 (secondary): eu-west-1 (3 nodes)
```

```
// Advantage: Disaster recovery, local reads

// 3. Geo-distributed with tagging
// Configure zones
sh.addShardTag("shard01", "US")
sh.addShardTag("shard02", "EU")
sh.addShardTag("shard03", "ASIA")

// Route data
sh.addTagRange("db.users", { region: "US" }, { region: "US" }, "US")
sh.addTagRange("db.users", { region: "EU" }, { region: "EU" }, "EU")

// 4. Hybrid cloud (on-prem + cloud)
// On-prem: Primary + 1 Secondary
// Cloud: 1 Secondary (backup/DR)
// Advantage: Disaster recovery, burst capacity
```

Deployment Automation

```javascript
// Infrastructure as Code (Terraform)
resource "mongodbatlas_cluster" "production" {
  project_id   = var.project_id
  name         = "production-cluster"
  cluster_type = "REPLICASET"

  provider_name               = "AWS"
  provider_instance_size_name = "M40"
  provider_region_name        = "us-east-1"

  replication_specs {
    num_shards = 1
    regions_config {
      region_name     = "us-east-1"
      electable_nodes = 3
      priority        = 7
    }
  }

  backup_enabled               = true
  pit_enabled                  = true
  cloud_backup                 = true
}

// Ansible playbook
- name: Configure MongoDB
  hosts: mongodb
  tasks:
    - name: Copy mongod.conf
      template:
        src: mongod.conf.j2
        dest: /etc/mongod.conf
      notify: restart mongod

    - name: Start MongoDB
      systemd:
        name: mongod
        state: started
```

```yaml
        enabled: yes

// Kubernetes Operator
apiVersion: mongodb.com/v1
kind: MongoDB
metadata:
  name: my-replica-set
spec:
  members: 3
  version: "6.0.5"
  type: ReplicaSet
  persistent: true
  podSpec:
    cpu: "2"
    memory: "4Gi"
    storage: 100Gi
```

Deployment Validation

javascript

```javascript
// Pre-deployment checklist
function validateDeployment() {
  // Check connectivity
  assert(db.runCommand({ ping: 1 }).ok);

  // Check replication
  const status = rs.status();
  assert(status.ok);
  assert(status.members.every(m => m.stateStr === 'PRIMARY' || m.stateStr === 'SEC
ONDARY'));

  // Check indexes
  const indexes = db.collection.getIndexes();
  assert(indexes.length >= requiredIndexCount);

  // Check backup
  assert(backupCompletedRecently());

  // Check monitoring
  assert(monitoringConfigured());

  console.log('Deployment validation passed');
}
```

Summary: The Complete Mental Model

Advanced Features

Think of advanced features as **specialized tools** in your toolbox:

- **Change Streams** are like fiber optics - instant, real-time communication

- **TTL Collections** are self-cleaning ovens - maintain themselves
- **Capped Collections** are circular buffers - always fresh, never full
- **Geospatial Queries** are GPS - location-aware intelligence
- **Full-Text Search** is a librarian - finds what you need, even with typos

DevOps & Production

Think of production operations as **flying a modern aircraft**:

- **Monitoring** is your instrument panel - know your altitude, speed, fuel
- **Logging** is the black box - record everything for post-incident analysis
- **Scaling** is adding more engines or bigger wings - more power to fly
- **Deployment** is the pre-flight checklist - verify everything before takeoff

The Golden Rules

For Advanced Features:

1. **Change streams for events** - don't poll when you can listen
2. **TTL for automatic cleanup** - set it and forget it
3. **Capped for rolling data** - perfect for logs and metrics
4. **Geo for location apps** - let the database do the math
5. **Text search for content** - but consider Atlas Search for complex needs

For Production:

1. **Monitor everything** - if you can't measure it, you can't manage it
2. **Log intelligently** - too much is noise, too little is silence
3. **Scale predictively** - don't wait until it hurts
4. **Automate deployments** - humans make mistakes, code is consistent
5. **Test failure scenarios** - your disaster recovery is only as good as your last test

MongoDB Real-World Design & Decision Making: Complete Theory Notes for Senior Developers

20. Real-World Design Skills

The Senior Developer's Mindset

What Separates Senior from Junior

A junior developer knows how to use MongoDB features. A senior developer knows **when and why** to use them, and can **design systems** that scale, perform, and remain maintainable.

The Senior Developer's Framework:

text

```text
Every design decision is a trade-off:
┌──────────────────────────────────────────────────────────┐
│                     Trade-off Triangle                   │
│                                                           │
│                    ┌──────────────┐                       │
│                    │ Performance │                        │
│                    └──────┬───────┘                       │
│                           │                               │
│            ┌──────────────┼──────────────┐                │
│            │              │              │                │
│     ┌──────▼──────┐  ┌──────▼──────┐  ┌──────▼──────┐     │
│     │ Consistency │  │ Scalability │  │ Development │     │
│     │             │  │             │  │   Speed     │     │
│     └─────────────┘  └─────────────┘  └─────────────┘     │
│                                                           │
│        You can only optimize for two at a time            │
└──────────────────────────────────────────────────────────┘
```

The Design Process

```text
Step 1: Understand Requirements
    ├── What data do we store?
    ├── How is data accessed?
    ├── What are the relationships?
    ├── What are the performance goals?
    └── What are the constraints?

Step 2: Identify Access Patterns
    ├── Read-heavy or write-heavy?
    ├── Which queries run most often?
    ├── What data is accessed together?
    └── What are the response time SLAs?

Step 3: Choose Data Model
    ├── Embed or reference?
    ├── What indexes are needed?
    ├── How will data grow?
    └── What patterns apply?

Step 4: Validate Design
    ├── Test with realistic data volume
    ├── Measure query performance
    ├── Check document size growth
    └── Simulate failure scenarios

Step 5: Iterate and Optimize
    ├── Monitor in production
    ├── Identify slow queries
    ├── Refine indexes
    └── Evolve schema as needed
```

Design Scalable MongoDB Schemas - Deep Theory

The Scalability Mindset

Scalable schema design is about **planning for growth**—not just today's data volume, but tomorrow's, next year's, and beyond.

Scalability Dimensions:

```text
Data Volume Growth:                    Access Pattern Growth:
Today: 100GB                          Today: 100 reads/sec
Next year: 1TB                         Next year: 1000 reads/sec
In 3 years: 10TB                       In 3 years: 10,000 reads/sec

Design must handle:                    Design must handle:
- Efficient queries at 10TB             - Read replicas
- Indexes that scale                     - Caching layers
- Sharding readiness                      - Query optimization
- Archive strategies                       - Connection pooling
```

The Working Set Principle

```javascript
// The most critical scalability concept:
// WORKING SET MUST FIT IN RAM

// Working Set = Frequently accessed data + Indexes
// If working set exceeds RAM → Performance collapses

// Example:
Total Data: 500GB
Active Users (last 30 days): 20% of data = 100GB
Indexes: 30GB
Working Set: 130GB

// If RAM = 64GB → 66GB of working set on disk → Page faults → Slow
// If RAM = 256GB → Entire working set in RAM → Fast

// Design implication: Structure data to minimize working set
// - Archive old data
// - Use subset pattern
// - Partition by time/tenant
```

Schema Design for Scale - Principles

```javascript
// Principle 1: Pre-join at write time, not read time
// BAD: Normalized at read time
db.orders.aggregate([
  { $lookup: { from: "customers", localField: "customerId", foreignField: "_id" }
},
  { $lookup: { from: "products", localField: "items.productId", foreignField: "_id
" } }
]);  // Expensive joins on every read

// GOOD: Denormalize at write time
db.orders.insertOne({
  customer: { id: custId, name: "John", email: "john@example.com" },
  items: [
    { productId: prodId, name: "Laptop", price: 999 }
```

```
  ]
});  // Single document read, no joins

// Principle 2: Design for your most frequent query
// If 90% of queries are "get user profile with recent orders"
// Design: User document with recent orders array (limited)

// Principle 3: Separate hot and cold data
// Hot data: Frequently accessed (keep in main collection)
// Cold data: Rarely accessed (move to archive collection)

// Principle 4: Plan for sharding from day 1
// Choose shard key that:
// - Has high cardinality
// - Supports your common queries
// - Distributes writes evenly
```

Real-World Example: E-commerce Platform

```javascript
// SCENARIO: E-commerce platform scaling to millions of users

// Phase 1: Startup (0-10K users)
// Simple embedded model works
const orderSchema = {
  customer: {  // Embedded
    id: "123",
    name: "John",
    email: "john@example.com"
  },
  items: [  // Embedded
    { productId: "p1", name: "Laptop", price: 999, qty: 1 },
    { productId: "p2", name: "Mouse", price: 29, qty: 2 }
  ],
  total: 1057,
  status: "pending",
  createdAt: new Date()
};

// Phase 2: Growth (10K-100K users)
// Orders per customer growing, need optimization
const userSchema = {
  _id: "user123",
  name: "John",
  email: "john@example.com",
  recentOrders: [  // Subset pattern
    { orderId: "ord1", date: "2024-01-15", total: 1057 },
    { orderId: "ord2", date: "2024-01-10", total: 299 }
  ],
  orderStats: {
    totalOrders: 47,
    totalSpent: 15789,
    averageOrder: 336
  }
};

// Orders collection (full details)
const orderSchema = {
```

```
  _id: "ord1",
  userId: "user123",  // Reference
  items: [ /* full items */ ],
  total: 1057,
  status: "delivered",
  createdAt: "2024-01-15"
};

// Phase 3: Scale (100K-1M users)
// Need sharding and archiving
sh.shardCollection("ecommerce.orders", { userId: "hashed" });

// Archive older orders
if (order.createdAt < new Date(Date.now() - 365*24*60*60*1000)) {
  db.orders_archive.insertOne(order);
  db.orders.deleteOne({ _id: order._id });
}

// Phase 4: Enterprise (1M+ users)
// Multi-region, geo-distributed
sh.addShardTag("shardUS", "US")
sh.addShardTag("shardEU", "EU")
sh.addTagRange("ecommerce.orders", { region: "US" }, { region: "US" }, "US")
sh.addTagRange("ecommerce.orders", { region: "EU" }, { region: "EU" }, "EU")

// Read from local region
db.orders.find().readPref("nearest")
```

Decide Embedding vs Referencing - Decision Framework

The Decision Matrix

Factor	Embed	Reference	Reasoning
Relationship cardinality	One-to-few	One-to-many/squillions	Array size limits
Data access pattern	Always together	Sometimes together	Minimize queries
Data dependency	Child needs parent	Independent lifecycle	Prevent orphans
Write frequency	Low on child	High on child	Avoid document rewrites

Factor	Embed	Reference	Reasoning
Data size	Small (< 16MB)	Can be large	Document limits
Consistency needs	Atomic updates	Eventual OK	Transaction cost

Decision Tree

text

```
Start: How are the entities related?
│
├── One-to-One → ALWAYS embed
│   Example: User has one profile
│   Reason: No downside, better performance
│
├── One-to-Many → Ask: How many?
│   ├── Few (< 100) → Embed
│   │   Example: Blog post has tags
│   │   Reason: Small array, always accessed together
│   │
│   ├── Many (100-1000) → Reference with array of IDs
│   │   Example: User has orders
│   │   Reason: Document stays small, pagination possible
│   │
│   └── Squillions (unbounded) → Parent reference in child
│       Example: Product has reviews
│       Reason: Never exceed limits, efficient queries
│
└── Many-to-Many → Ask: With attributes?
    ├── No attributes → Array of IDs in both
    │   Example: Students and courses (simple enrollment)
    │   Reason: Simple, efficient
    │
    └── With attributes → Join collection
        Example: Students and courses with grade, date
        Reason: Relationship has its own data
```

Real-World Examples

```javascript
// EXAMPLE 1: Blog platform - Embedding decision

// POST with COMMENTS (One-to-Many, but comments unbounded)
// WRONG: Embed comments (will exceed 16MB)
{
  _id: "post123",
  title: "MongoDB Design",
  content: "...",
  comments: [  // Could be thousands!
    { user: "john", text: "Great post!", date: new Date() },
    // ... thousands more
  ]
}

// RIGHT: Reference comments
{
  _id: "post123",
  title: "MongoDB Design",
  content: "...",
  commentCount: 1543,
  recentComments: [  // Subset pattern
    { user: "john", text: "Great post!", date: new Date() },
    { user: "jane", text: "Thanks!", date: new Date() }
  ]
}

// Comments collection
{
  _id: "comment1",
  postId: "post123",
  userId: "user123",
  text: "Great post!",
  date: new Date()
}

// EXAMPLE 2: E-commerce - Mixed strategy

// PRODUCT with CATEGORY (One-to-One, embed)
{
  _id: "prod123",
  name: "Laptop",
  price: 999,
  category: {  // Embedded (always accessed together)
    id: "cat456",
    name: "Electronics"
  }
}

// ORDER with ITEMS (One-to-Many, embed items)
{
  _id: "ord123",
  customerId: "cust789",
  items: [  // Embed (few items, always needed)
    { productId: "prod123", name: "Laptop", price: 999, qty: 1 },
    { productId: "prod456", name: "Mouse", price: 29, qty: 2 }
```

```
  ],
  total: 1057
}

// CUSTOMER with ORDERS (One-to-Many, reference)
{
  _id: "cust789",
  name: "John Doe",
  email: "john@example.com",
  recentOrderIds: ["ord123", "ord124"]  // Reference (many orders)
}
```

Optimize Slow Queries - Systematic Approach

The Query Optimization Process

```text
Step 1: Identify slow queries
    ├── Enable profiling
    ├── Check logs
    └── Monitor in real-time

Step 2: Analyze with explain()
    ├── Which indexes used?
    ├── Documents examined vs returned
    ├── Execution time breakdown
    └── Sort/memory usage

Step 3: Diagnose the problem
    ├── No index → Create index
    ├── Wrong index → Redesign index (ESR)
    ├── Too many docs scanned → More selective query
    ├── In-memory sort → Add index for sort
    └── Large documents → Project only needed fields

Step 4: Implement fix
    ├── Create/modify index
    ├── Rewrite query
    ├── Restructure data
    └── Add hints if needed

Step 5: Verify improvement
```

```
    ├── Compare before/after explain
    ├── Test with production data volume
    └── Monitor in production
```

Real-World Optimization Examples

```javascript
// SCENARIO 1: E-commerce product search is slow

// Original query
db.products.find({
  category: "electronics",
  price: { $gt: 100, $lt: 1000 },
  inStock: true
}).sort({ rating: -1 });

// Step 1: Explain
db.products.find(...).explain("executionStats");
// Shows: COLLSCAN, 1M documents examined, 10K returned, in-memory sort

// Step 2: Create proper index (ESR rule)
db.products.createIndex({
  category: 1,     // Equality first
  rating: -1,      // Sort next
  price: 1,        // Range last
  inStock: 1       // Additional equality
});

// Step 3: Verify improvement
// Now: IXSCAN, 10K examined, indexed sort, 50ms vs 5s

// SCENARIO 2: User dashboard loading slowly

// Original query
db.orders.find({ userId: "user123" }).sort({ createdAt: -1 });

// Problem: User has 10,000 orders, but dashboard only shows 20

// Solution: Pagination with projection
db.orders.find(
  { userId: "user123" },
  { _id: 1, date: 1, total: 1, status: 1 }  // Only needed fields
).sort({ createdAt: -1 }).limit(20);

// Create compound index
db.orders.createIndex({ userId: 1, createdAt: -1 });

// SCENARIO 3: Aggregation pipeline timing out

// Original pipeline
db.sales.aggregate([
  { $match: { date: { $gte: new Date("2024-01-01") } } },
  { $group: { _id: "$productId", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } },
  { $limit: 10 }
]);
```

```
// Problem: $match not selective enough (6 months of data)

// Solution: Pre-aggregate or use bucket pattern
db.sales.aggregate([
  { $match: { date: { $gte: new Date("2024-01-01") } } },
  { $group: {
      _id: {
        productId: "$productId",
        month: { $month: "$date" }
      },
      total: { $sum: "$amount" }
  }},
  { $group: {
      _id: "$_id.productId",
      total: { $sum: "$total" }
  }},
  { $sort: { total: -1 } },
  { $limit: 10 }
]);
```

Common Anti-Patterns and Fixes

javascript

```
// ANTI-PATTERN 1: No index on query field
db.users.find({ email: "user@example.com" });  // COLLSCAN
// FIX: db.users.createIndex({ email: 1 });

// ANTI-PATTERN 2: Not using ESR order
db.orders.find({ status: "active", amount: { $gt: 100 } })
  .sort({ date: -1 });
// Index { status:1, amount:1, date:-1 } is better than { amount:1, status:1 }

// ANTI-PATTERN 3: Returning too much data
db.products.find({ category: "electronics" });  // Returns all fields
// FIX: Add projection { name:1, price:1, _id:0 }

// ANTI-PATTERN 4: Inefficient $or
db.orders.find({
  $or: [
    { status: "pending" },
    { priority: "high" }
  ]
});
// FIX: Use separate queries or compound index with both fields

// ANTI-PATTERN 5: Slow regex with leading wildcard
db.articles.find({ title: /.*mongodb.*/ });  // Can't use index
// FIX: Use text index or avoid leading wildcard

// ANTI-PATTERN 6: Not using covered queries
db.users.find({ email: "user@example.com" }, { name:1, email:1 });
// Can be covered if index includes both fields
db.users.createIndex({ email:1, name:1 });
```

Use Aggregation Pipelines - Design Patterns

Aggregation Pipeline Design Principles

```text
Principle 1: Filter early, filter often
    └── $match as early as possible

Principle 2: Project before group
    └── Reduce document size before grouping

Principle 3: Use indexes where possible
    └── $match and $sort at start can use indexes

Principle 4: Watch memory limits
    └── $group, $sort may need disk

Principle 5: Test with realistic data
    └── Performance at scale may differ
```

Common Aggregation Patterns

javascript

```javascript
// PATTERN 1: Reporting dashboard
const dashboardReport = db.sales.aggregate([
  // Filter recent data
  { $match: {
    date: { $gte: new Date("2024-01-01") },
    status: "completed"
  }},

  // Enrich with product details
  {
    $lookup: {
      from: "products",
      localField: "productId",
      foreignField: "_id",
      as: "product"
    }
  },
  { $unwind: "$product" },

  // Group by multiple dimensions
  {
    $group: {
      _id: {
        category: "$product.category",
        month: { $month: "$date" }
      },
      totalSales: { $sum: "$amount" },
```

```
      orderCount: { $sum: 1 },
      avgOrderValue: { $avg: "$amount" },
      uniqueCustomers: { $addToSet: "$customerId" }
    }
  },

  // Calculate derived fields
  {
    $addFields: {
      uniqueCustomerCount: { $size: "$uniqueCustomers" },
      salesPerCustomer: {
        $divide: ["$totalSales", { $size: "$uniqueCustomers" }]
      }
    }
  },

  // Sort and format
  { $sort: { "_id.month": -1, totalSales: -1 } },
  {
    $project: {
      _id: 0,
      category: "$_id.category",
      month: "$_id.month",
      totalSales: 1,
      orderCount: 1,
      avgOrderValue: { $round: ["$avgOrderValue", 2] },
      uniqueCustomers: "$uniqueCustomerCount"
    }
  }
]);

// PATTERN 2: Paginated search with facets
const searchResults = db.products.aggregate([
  // Text search
  {
    $search: {
      text: {
        query: "laptop",
        path: ["name", "description"]
      }
    }
  },

  // Facet counts
  {
    $facet: {
      results: [
        { $skip: 20 },
        { $limit: 10 },
        {
          $project: {
            name: 1,
            price: 1,
            brand: 1,
            score: { $meta: "searchScore" }
          }
        }
      ],
      brandFacets: [
```

```
        { $group: { _id: "$brand", count: { $sum: 1 } } },
        { $sort: { count: -1 } }
      ],
      priceRanges: [
        {
          $bucket: {
            groupBy: "$price",
            boundaries: [0, 500, 1000, 2000],
            default: "2000+",
            output: { count: { $sum: 1 } }
          }
        }
      ],
      totalCount: [
        { $count: "count" }
      ]
    }
  }
]);

// PATTERN 3: Time-series analysis
const timeSeriesAnalysis = db.sensorData.aggregate([
  // Filter time range
  { $match: {
    sensorId: "temp-001",
    timestamp: {
      $gte: new Date("2024-01-01"),
      $lt: new Date("2024-02-01")
    }
  }},

  // Bucket by hour
  {
    $bucket: {
      groupBy: "$timestamp",
      boundaries: generateHourBoundaries("2024-01-01", "2024-02-01"),
      output: {
        readings: { $push: "$$ROOT" },
        avg: { $avg: "$value" },
        min: { $min: "$value" },
        max: { $max: "$value" },
        count: { $sum: 1 }
      }
    }
  },

  // Calculate trends
  {
    $setWindowFields: {
      sortBy: { _id: 1 },
      output: {
        movingAvg: {
          $avg: "$avg",
          window: { documents: [-2, 2] }
        }
      }
    }
  }
]);
```

Handle Production-Level Data - Best Practices

Data Lifecycle Management

javascript

```javascript
// 1. Data Tiering Strategy
const dataTiering = {
  hot: {
    criteria: { lastAccess: { $gt: new Date(Date.now() - 30*24*60*60*1000) } },
    storage: "SSD",
    indexes: "all",
    location: "primary cluster"
  },
  warm: {
    criteria: { lastAccess: { $gt: new Date(Date.now() - 90*24*60*60*1000) } },
    storage: "HDD",
    indexes: "essential only",
    location: "secondary cluster"
  },
  cold: {
    criteria: { lastAccess: { $lt: new Date(Date.now() - 90*24*60*60*1000) } },
    storage: "S3/Archive",
    indexes: "none",
    location: "object storage"
  }
};

// Implement data tiering
async function tierData() {
  const session = db.getMongo().startSession();

  try {
    // Move old data to warm storage
    const warmData = await db.orders.find({
      createdAt: {
        $lt: new Date(Date.now() - 30*24*60*60*1000),
        $gt: new Date(Date.now() - 90*24*60*60*1000)
      }
    }).toArray();

    // Archive to cold storage
    const coldData = await db.orders.find({
      createdAt: { $lt: new Date(Date.now() - 90*24*60*60*1000) }
    }).toArray();

    await archiveToS3(coldData);
    await db.orders.deleteMany({
      _id: { $in: coldData.map(d => d._id) }
    });

  } finally {
    session.endSession();
  }
}

// 2. Archival Strategy
async function archiveCollection(collection, archiveDb, olderThan) {
```

```
  const archiveCollection = archiveDb.collection(collection.collectionName);

  // Use bulk operations for efficiency
  const batchSize = 1000;
  let lastId = null;

  while (true) {
    const query = lastId ?
      { _id: { $gt: lastId }, createdAt: { $lt: olderThan } } :
      { createdAt: { $lt: olderThan } };

    const docs = await collection.find(query)
      .limit(batchSize)
      .toArray();

    if (docs.length === 0) break;

    // Insert to archive
    await archiveCollection.insertMany(docs, { ordered: false });

    // Delete from main
    await collection.deleteMany({
      _id: { $in: docs.map(d => d._id) }
    });

    lastId = docs[docs.length - 1]._id;
  }
}

// 3. Purging Strategy (GDPR compliance)
async function purgeUserData(userId) {
  const session = db.getMongo().startSession();
  session.startTransaction();

  try {
    // Anonymize personal data
    await db.users.updateOne(
      { _id: userId },
      {
        $set: {
          name: "[REDACTED]",
          email: `deleted-${userId}@example.com`,
          deletedAt: new Date()
        },
        $unset: {
          address: "",
          phone: "",
          paymentMethods: ""
        }
      },
      { session }
    );

    // Keep orders but anonymize
    await db.orders.updateMany(
      { userId: userId },
      {
        $set: {
          "customer.name": "[REDACTED]",
```

```
          anonymized: true
        }
      },
      { session }
    );

    await session.commitTransaction();
  } catch (error) {
    await session.abortTransaction();
    throw error;
  } finally {
    session.endSession();
  }
}
```

Production Readiness Checklist

```javascript
const productionReadiness = {
  // Data modeling
  schemaValidation: true,
  indexesForAllQueries: true,
  documentSizeCheck: true,
  shardingStrategy: "planned",

  // Performance
  queryProfiling: true,
  slowQueryLogging: true,
  connectionPooling: true,
  cacheSizing: "working set fits in RAM",

  // High availability
  replicaSet: "3 nodes",
  automaticFailover: true,
  readPreference: "configured",
  writeConcern: "majority",

  // Data durability
  backups: "continuous with PITR",
  backupTesting: "monthly restore tests",
  journaling: true,

  // Security
  authentication: true,
  authorization: "RBAC",
  encryption: "TLS + at rest",
  networkIsolation: "VPC",

  // Monitoring
  metricsCollection: true,
  alertsConfigured: true,
  dashboardsCreated: true,
  logAggregation: true,

  // Operations
  deploymentAutomated: true,
  rollbackPlan: true,
  runbooks: true,
```

```
  onCallRotation: true
};
```

---

21. When to Use MongoDB vs SQL - Deep Decision Framework

The Fundamental Question

The choice between MongoDB and SQL isn't about which is "better"—it's about which is **more appropriate** for your specific use case.

The Decision Framework:

text

```
Project Requirements
        ↓
┌───────┴───────┐
↓               ↓
MongoDB Path    SQL Path
    ↓               ↓
Ask: Does my data look like? → Ask: Does my data look like?
    ↓                            ↓
JSON documents                Tables with rows
Nested structures             Normalized relations
Varying schema                Fixed schema
    ↓                            ↓
Ask: How do I query?         Ask: How do I query?
    ↓                            ↓
By document structure        By joining tables
Embedded data patterns        Complex SQL joins
    ↓                            ↓
Ask: How does it scale?       Ask: How does it scale?
    ↓                            ↓
Horizontal (sharding)        Vertical (bigger server)
Distributed by nature         ACID across nodes complex
```

Use MongoDB When - Deep Analysis

1. Flexible Schema Needed

```javascript
// Scenario: Content management system
// Different content types have different fields

// SQL approach: Painful
// - Posts table with 50 nullable columns
// - Or 20 different tables with complex joins
// - Schema changes require migrations

// MongoDB approach: Natural
const blogPost = {
  type: "article",
  title: "MongoDB Design",
  content: "...",
  author: "John",
  tags: ["database", "nosql"],
  metadata: {
    readTime: 5,
    difficulty: "intermediate"
  }
};

const videoPost = {
  type: "video",
  title: "MongoDB Tutorial",
  url: "https://...",
  duration: 3600,
  transcript: "...",
  metadata: {
    resolution: "1080p",
    format: "mp4"
  }
};

// Both in same collection, query by type
db.posts.find({ type: "article", "metadata.difficulty": "intermediate" });
```

2. High Write Throughput

```javascript
// Scenario: IoT sensor data - millions of writes per second

// SQL approach: Bottleneck
// - Each insert is a transaction
// - Index updates on every write
// - Table locks or row locks contention

// MongoDB approach: Scales
// - Document model batches naturally
// - Sharding distributes writes
// - No schema validation overhead
```

```javascript
// Batch insert (10,000 docs at once)
db.sensorData.insertMany(readings, { ordered: false });

// Sharded by sensor ID for even distribution
sh.shardCollection("iot.sensorData", { sensorId: "hashed" });

// Write concern tuned for performance
db.sensorData.insertOne(reading, { writeConcern: { w: 1 } });
```

3. Nested/Complex Data

```javascript
// Scenario: Product catalog with variants, specs, images

// SQL approach: 5-10 tables with complex joins
// products
// product_variants
// product_specs
// product_images
// product_reviews
// SELECT p.*, v.*, s.*, i.*, r.*
// FROM products p
// LEFT JOIN product_variants v ON p.id = v.product_id
// LEFT JOIN product_specs s ON p.id = s.product_id
// LEFT JOIN product_images i ON p.id = i.product_id
// LEFT JOIN product_reviews r ON p.id = r.product_id
// WHERE p.id = 123

// MongoDB approach: Single document
{
  _id: "prod123",
  name: "Smartphone X",
  brand: "TechCorp",
  specs: {
    screen: "6.1 inch OLED",
    processor: "A15 Bionic",
    ram: "8GB",
    storage: "256GB",
    battery: "4500mAh"
  },
  variants: [
    { color: "black", sku: "X-BLK", price: 999, stock: 50 },
    { color: "white", sku: "X-WHT", price: 999, stock: 30 },
    { color: "gold", sku: "X-GLD", price: 1099, stock: 15 }
  ],
  images: [
    { url: "img1.jpg", primary: true },
    { url: "img2.jpg", primary: false }
  ],
  reviews: [
    { user: "john", rating: 5, text: "Great phone!" }
  ]
}

// One query, all data
db.products.findOne({ _id: "prod123" });
```

4. Rapid Development and Iteration

```javascript
// Scenario: Startup MVP - requirements change daily

// Week 1: Users have name and email
db.users.insertOne({ name: "John", email: "john@example.com" });

// Week 2: Add phone and address
db.users.insertOne({
  name: "Jane",
  email: "jane@example.com",
  phone: "555-1234",
  address: {
    city: "Boston",
    zip: "02101"
  }
});

// Week 3: Add preferences
db.users.updateMany({}, {
  $set: { preferences: { newsletter: true, theme: "light" } }
});

// No downtime, no migrations, no ALTER TABLE
```

5. Geographic Distribution

```javascript
// Scenario: Global application with local data

// MongoDB Atlas global clusters
{
  "globalCluster": true,
  "customZoneMappings": [
    { "zone": "NA", "location": "NORTH_AMERICA" },
    { "zone": "EU", "location": "EUROPE" },
    { "zone": "ASIA", "location": "ASIA_PACIFIC" }
  ]
}

// Data stays in region
sh.addTagRange("db.users", { region: "NA" }, { region: "NA" }, "NA")

// Reads from local region
db.users.find().readPref("nearest")
```

Use SQL When - Deep Analysis

1. Complex Joins Across Multiple Tables

```javascript
```

```
// Scenario: Financial reporting with complex relationships

// SQL shines:
SELECT
    c.name,
    a.account_number,
    t.transaction_date,
    t.amount,
    t.type,
    cat.category_name,
    cat.tax_implications
FROM customers c
JOIN accounts a ON c.id = a.customer_id
JOIN transactions t ON a.id = t.account_id
JOIN categories cat ON t.category_id = cat.id
JOIN tax_rules tr ON cat.id = tr.category_id
WHERE t.transaction_date >= '2024-01-01'
AND t.amount > 1000
AND tr.country = 'US';

// MongoDB approach: Complex
// - Multiple $lookup stages
// - Performance degrades with each join
// - No query optimizer for joins
```

2. Strong Consistency Requirements

javascript

```
// Scenario: Banking system - money transfers

// SQL: ACID transactions across multiple tables
BEGIN TRANSACTION;
    UPDATE accounts SET balance = balance - 100 WHERE id = 1;
    UPDATE accounts SET balance = balance + 100 WHERE id = 2;
    INSERT INTO transfers (from_id, to_id, amount) VALUES (1, 2, 100);
COMMIT;

// MongoDB: Multi-document transactions possible but:
// - Performance overhead
// - Limited to replica sets
// - More complex application code
// - Not as mature as SQL transactions
```

3. Highly Structured, Stable Data

javascript

```
// Scenario: Government system with fixed schema

// SQL: Perfect fit
CREATE TABLE citizens (
    id INT PRIMARY KEY,
    ssn VARCHAR(11) UNIQUE NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    birth_date DATE NOT NULL,
    birth_place VARCHAR(100),
    -- 100+ other fixed fields
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

// Data never changes structure
// Schema enforced at database level
// Reports always expect same columns
```

4. Ad-hoc Reporting and Analytics

```
javascript
// Scenario: Business intelligence dashboard

// SQL: Powerful analytics
SELECT
    DATE_TRUNC('month', order_date) as month,
    product_category,
    COUNT(DISTINCT customer_id) as unique_customers,
    SUM(amount) as revenue,
    AVG(amount) as avg_order_value,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) as median_order
FROM orders o
JOIN products p ON o.product_id = p.id
WHERE order_date >= '2024-01-01'
GROUP BY 1, 2
ORDER BY 1 DESC, 2;

// MongoDB: Possible but more complex
// - Need multiple aggregation stages
// - No percentile function natively
// - More verbose
```

5. Existing SQL Expertise and Tooling

javascript

```
// Scenario: Enterprise with SQL-trained team

// SQL advantages:
// - Team already knows SQL
// - Existing BI tools (Tableau, PowerBI) work
// - ORM support (Hibernate, Entity Framework)
// - Decades of best practices
// - Mature query optimization
```

Hybrid Approach: Polyglot Persistence

Using Both MongoDB and SQL

javascript

```javascript
// Modern architecture: Use the right tool for each job

class HybridArchitecture {
  constructor() {
    this.mongodb = new MongoClient(process.env.MONGODB_URI);
    this.postgres = new Pool(process.env.POSTGRES_URI);
  }

  // Product catalog in MongoDB (flexible schema, nested data)
  async getProduct(productId) {
    return this.mongodb.db('catalog').collection('products')
      .findOne({ _id: productId });
  }

  // Orders in MongoDB (document structure, fast writes)
  async createOrder(orderData) {
    return this.mongodb.db('orders').collection('orders')
      .insertOne(orderData);
  }

  // Financial transactions in PostgreSQL (ACID, complex joins)
  async processPayment(orderId, amount) {
    const client = await this.postgres.connect();
    try {
      await client.query('BEGIN');

      await client.query(
        'INSERT INTO transactions (order_id, amount, status) VALUES ($1, $2, $3)',
        [orderId, amount, 'pending']
      );

      await client.query(
        'UPDATE accounts SET balance = balance - $1 WHERE account_id = $2',
        [amount, orderId]
      );

      await client.query('COMMIT');
    } catch (e) {
```

```
      await client.query('ROLLBACK');
      throw e;
    } finally {
      client.release();
    }
  }

  // Analytics in PostgreSQL (complex reporting)
  async getMonthlyReport(year, month) {
    return this.postgres.query(`
      SELECT
        DATE_TRUNC('month', created_at) as month,
        product_category,
        COUNT(*) as orders,
        SUM(amount) as revenue
      FROM orders
      WHERE EXTRACT(YEAR FROM created_at) = $1
        AND EXTRACT(MONTH FROM created_at) = $2
      GROUP BY 1, 2
    `, [year, month]);
  }
}
```

Migration Strategy: SQL to MongoDB

javascript

```javascript
// When migrating from SQL to MongoDB

// Phase 1: Analyze SQL schema
const sqlSchema = {
  tables: ['users', 'orders', 'order_items', 'products'],
  relationships: [
    'users.id → orders.user_id',
    'orders.id → order_items.order_id',
    'products.id → order_items.product_id'
  ]
};

// Phase 2: Design MongoDB document model
const mongoModel = {
  users: {
    _id: 'from SQL users.id',
    name: 'from users.name',
    email: 'from users.email',
    recentOrders: []  // Will be populated from orders
  },
  orders: {
    _id: 'from orders.id',
    userId: 'from orders.user_id',
    items: [],  // Embedded from order_items + products
    total: 'calculated',
    date: 'from orders.created_at'
  }
};

// Phase 3: Migration script
async function migrateData() {
  const sql = await getSQLConnection();
```

```
  const mongo = await getMongoDB();

  // Migrate users
  const users = await sql.query('SELECT * FROM users');
  for (const user of users) {
    // Get user's orders
    const orders = await sql.query(
      'SELECT * FROM orders WHERE user_id = $1',
      [user.id]
    );

    // For each order, get items with product details
    for (const order of orders) {
      const items = await sql.query(`
        SELECT oi.*, p.name, p.price
        FROM order_items oi
        JOIN products p ON oi.product_id = p.id
        WHERE oi.order_id = $1
      `, [order.id]);

      order.items = items;
    }

    // Create MongoDB document
    await mongo.collection('users').insertOne({
      _id: user.id,
      name: user.name,
      email: user.email,
      recentOrders: orders.slice(0, 5).map(o => ({
        id: o.id,
        date: o.created_at,
        total: o.total
      }))
    });

    // Create orders collection
    await mongo.collection('orders').insertMany(
      orders.map(o => ({
        _id: o.id,
        userId: o.user_id,
        items: o.items,
        total: o.total,
        date: o.created_at
      }))
    );
  }
}
```

Summary: The Complete Mental Model

Real-World Design Skills

Think of yourself as an **architect**:

- **Design scalable schemas** like designing a building foundation - must support future floors
- **Decide embedding vs referencing** like deciding what goes in which room - related things together
- **Optimize slow queries** like finding bottlenecks in traffic - analyze, identify, fix
- **Use aggregation pipelines** like an assembly line - each stage transforms the product
- **Handle production data** like a facility manager - monitor, maintain, plan for growth

MongoDB vs SQL Decision

Think of it as choosing between **different vehicles**:

- **MongoDB** is a SUV - flexible, handles rough terrain, carries varied cargo, easy to reconfigure
- **SQL** is a train - follows fixed tracks, carries standardized containers, extremely efficient at scale
- **Polyglot persistence** is having a fleet - use the right vehicle for each job

The Golden Rules for Real-World Design

1. **Start simple, but plan for growth** - You can always add complexity later
2. **Model for your queries, not your data** - Structure follows access patterns
3. **Working set must fit in RAM** - The cardinal rule of MongoDB performance
4. **Embed unless there's a reason not to** - Referencing is the exception, not the rule
5. **Index aggressively, but wisely** - Every index has a cost
6. **Test with production-scale data** - 1M docs behave differently than 1B
7. **Monitor everything** - You can't optimize what you don't measure

8. **Have a migration plan** - Schemas evolve, data grows, requirements change

The Final Word

At the senior level, you're not just writing code—you're **making trade-offs**:

```text
Performance vs. Flexibility
Consistency vs. Availability
Development Speed vs. Operational Excellence
Today's Simplicity vs. Tomorrow's Scalability
```

Your job is to understand these trade-offs deeply and make the right choice for your specific context. There's no universal "best" - only what's best for your users, your data, and your constraints.